

Neural Networks

Máster en Innovación Tecnológica (MINT)

From logistic regression to neural networks

imanolgranada@gmail.com



1

Why Neural Networks and Deep Learning?

Virtual assistants

Speech recognition and synthesis.

Chatbots

ChatGPT and friends: transformers (chapter 6).

Fake news detection

News aggregation and text classification.

Composing music

Sequence generation (chapter 5).

Image coloring

Image-to-image networks.

Robotics

Perception and control.

Image captioning

A CNN encoder + a language decoder.

YOLO

You Only Look Once: real-time object detection (chapter 4).

Stable Diffusion

Generating new images from text.

RAG

Chatbots that answer from your own documents (chapter 6).

Vehicle damage from photos

Classify parts, segment damage, locate it with boxes: every block of this course, together.

All of them are built from the **same few blocks** you will learn in these six chapters. [YOLO demo video](#)



2

The plan for the six chapters

1 · Neurons and layers

A neuron is a logistic regression. Stack them: a multi-layer perceptron. Why the non-linearity matters.

2 · Forward and backward

How the prediction is computed, and how every weight gets its gradient (backpropagation), with real numbers.

Lab 7 · an MLP from scratch in NumPy

3 · Training

Mini-batches, optimizers (momentum, Adam), learning-rate schedules, dropout, BatchNorm, the layer zoo.

Labs 8, 9 · PyTorch MLP · multiclass images

4 · Images: CNNs

Convolutions, pooling, famous architectures, transfer learning, then segmentation and object detection.

Labs 10, 12, 13 · CNN · fine-tuning a ResNet · YOLO (+ a multi-task demo)

5 · Sequences: RNNs and LSTMs

Embeddings, a network with memory, why it forgets, and the gates that fix it.

Lab 11 · LSTM sentiment analysis

6 · Attention and transformers

Every token looks at every other token. GPT, decoding text, and RAG.

Labs 14, 15 · text generation with GPT-2 · RAG

Every chapter follows the same pattern: a picture, the formula behind it, a small example with numbers, and a lab (a green slide with an **Open in Colab** button) as soon as the theory it needs has been explained.

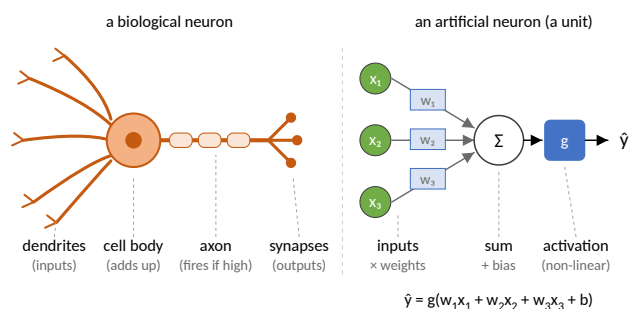
Why are they called Neural Networks?

The brain is composed of a lot of small (and "simple") **neurons**. Each one collects signals from others and "fires" when the total is strong enough.

Neural networks are composed of a lot of small and simple **units**: each one computes a **weighted sum** of its inputs and applies a **non-linear function**.

The power does not come from any single unit, but from **connecting many of them in layers**.

The analogy with biology stops there: what we train is a big differentiable function, with the same tools as in the ML part of the course (a loss and gradient descent).



Problem Statement:

Same as in supervised learning: given a training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$, learn h so that $h(\mathbf{x})$ is a "good" predictor of y .

"good" = minimize a loss function!

- y continuous $\rightarrow$ regression
- y discrete $\rightarrow$ classification
- A combination of both (e.g. a box and its class in object detection)

What changes is the model family:

- Linear / logistic regression: h is linear in $\mathbf{x}$ (or a sigmoid of it).
- Neural network: h is a **composition** of many simple functions:

$$h(\mathbf{x}) = g^{[L]} \left(\dots g^{[2]} \left(g^{[1]} (\mathbf{x} \mathbf{W}^{[1]} + \mathbf{b}^{[1]}) \mathbf{W}^{[2]} + \mathbf{b}^{[2]} \right) \dots \right)$$

The features are no longer designed by hand: the hidden layers **learn** them. The next slides build this formula one piece at a time.

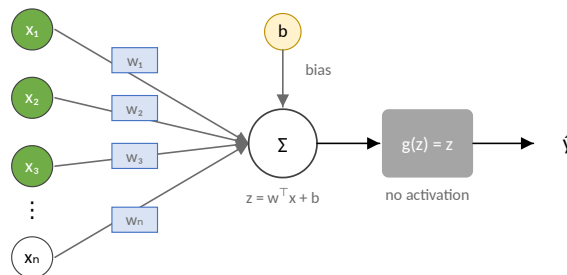
The simplest network: one neuron, no activation

You already know this model: it is **linear regression**.

$$\hat{y} = w_1 x_1 + \dots + w_n x_n + b = \mathbf{w}^\top \mathbf{x} + b$$

- New names for old things: the **weights** $\mathbf{w}$ are the $\theta_1, \dots, \theta_n$ of the ML part, and the **bias** b is θ_0 .
- Train it by minimizing the MSE with gradient descent: exactly what you did in the ML labs.

Every neural network is built from this unit, plus **one extra ingredient**: a non-linear **activation** applied to $z = \mathbf{w}^\top \mathbf{x} + b$.



One neuron with no activation = linear regression

Logistic Regression is a Neural Net:

Approximate y as a non-linear function of $\mathbf{x}$:

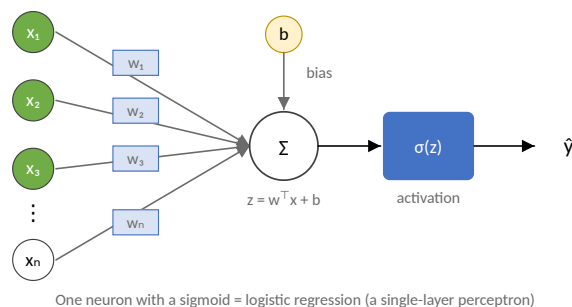
$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function: it squashes any number into $(0, 1)$, a probability.

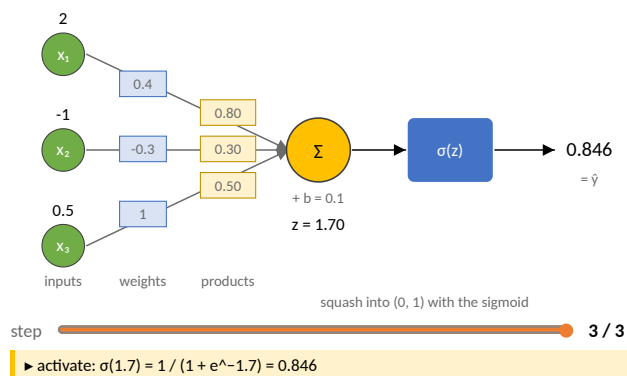
1. A **linear step**: $z = \mathbf{w}^\top \mathbf{x} + b = w_1 x_1 + \dots + w_n x_n + b$.

2. An **activation**: $a = \sigma(z)$.

Single layer perceptron! One neuron = logistic regression. Every unit of a neural network does exactly these two steps.



One neuron, with numbers



Inputs $\mathbf{x} = (2, -1, 0.5)$, weights $\mathbf{w} = (0.4, -0.3, 1.0)$, bias $b = 0.1$.

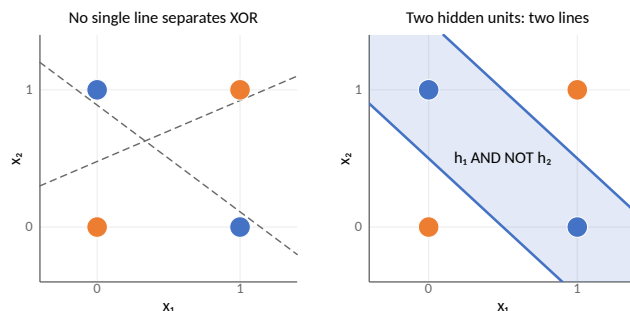
$$z = \sum_i w_i x_i + b = 0.8 + 0.3 + 0.5 + 0.1 = 1.7$$

$$\hat{y} = \sigma(1.7) = \frac{1}{1 + e^{-1.7}} = 0.846$$

Two operations, always the same: a **weighted sum** (the weights are what we learn) and a **squashing** function. Positive weights vote "yes", negative ones vote "no", and the bias sets the threshold.

The whole of deep learning is this, repeated millions of times and arranged cleverly.

The limit of linear models: XOR



Proof that no line works. Suppose $w_1x_1 + w_2x_2 + b$ is positive exactly on the blue points:

- $(0, 0)$ orange: $b \leq 0$.
- $(1, 0)$ and $(0, 1)$ blue: $w_1 + b > 0$ and $w_2 + b > 0$.
- Add them: $w_1 + w_2 + b > -b \geq 0$.
- But $(1, 1)$ is orange: $w_1 + w_2 + b \leq 0$. Contradiction. ■

XOR ("one or the other, not both") is the simplest problem where a single neuron fails. Two ways out: **invent new features** by hand, or **let the network learn them**.

Fix 1: engineer new features by hand



Circle data: class 1 (blue) inside $x_1^2 + x_2^2 < r^2$. No line separates the classes in the (x_1, x_2) plane.

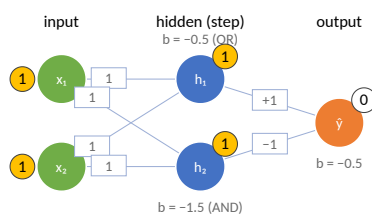
With the new features $z_1 = x_1^2$, $z_2 = x_2^2$ the border becomes $z_1 + z_2 = r^2$: a **straight line**. Logistic regression on (z_1, z_2) solves it.

For XOR the feature x_1x_2 does the job: $\hat{y} = x_1 + x_2 - 2x_1x_2$ is exactly XOR on the four points.

But **you must guess the right features**. For a 224×224 photo, which formula of the 150,528 pixel values means "cat"? Nobody knows.

Fix 2: let the model learn useful features from the data. That is exactly what a **hidden layer** does.

Fix 2: a hidden layer learns the features (XOR by hand)



x_1	x_2	h_1	h_2	$\hat{y}$
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

$\hat{y} = \text{XOR}(x_1, x_2) \checkmark$

$h_1: 1 + 1 - 0.5 = 1.5 \rightarrow 1 \cdot h_2: 1 + 1 - 1.5 = 0.5 \rightarrow 1 \cdot \hat{y}: 1 - 1 - 0.5 = -0.5 \rightarrow 0$

input row 4 / 4

► input (1, 1): both fire and AND cancels OR, $\hat{y} = 0$

Two hidden units, each one draws a line:

- $h_1 = \text{step}(x_1 + x_2 - 0.5)$ is **OR**,
- $h_2 = \text{step}(x_1 + x_2 - 1.5)$ is **AND**.

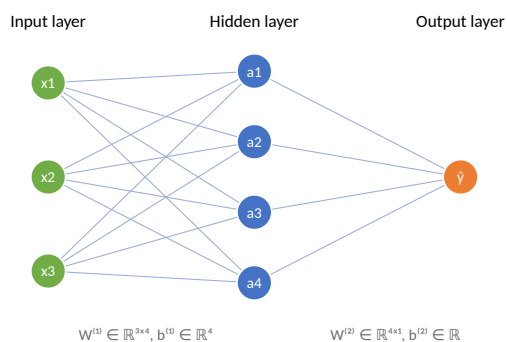
The output is a third unit, on (h_1, h_2) :

$\hat{y} = \text{step}(h_1 - h_2 - 0.5)$, "OR but not AND" = XOR.

In the new coordinates (h_1, h_2) the four points become $(0, 0)$, $(1, 0)$, $(1, 0)$, $(1, 1)$: now a line separates them. The hidden layer **changed the representation**.

Here we chose the weights by hand. Training finds weights like these automatically, by gradient descent (chapter 2). That is what "learning the features" means.

Multi-layer perceptron:



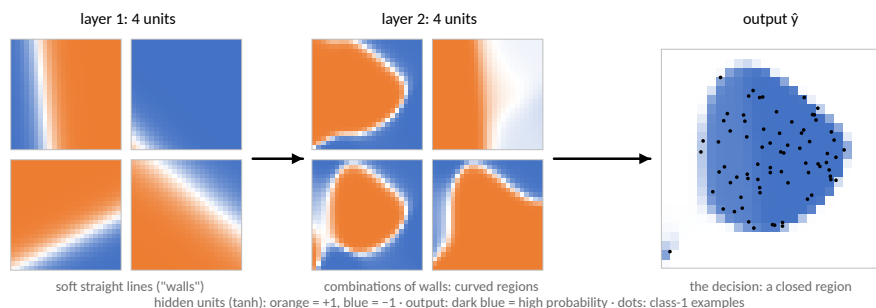
- $a_1 = \sigma(\mathbf{W}_{:,1}^{[1]\top} \mathbf{x} + b_1^{[1]})$
- $a_2 = \sigma(\mathbf{W}_{:,2}^{[1]\top} \mathbf{x} + b_2^{[1]})$
- $a_3 = \sigma(\mathbf{W}_{:,3}^{[1]\top} \mathbf{x} + b_3^{[1]})$
- $a_4 = \sigma(\mathbf{W}_{:,4}^{[1]\top} \mathbf{x} + b_4^{[1]})$
- $h(\mathbf{x}) = \sigma(\mathbf{W}^{[2]\top} \mathbf{a} + b^{[2]})$, with $\mathbf{a} = [a_1, \dots, a_4]$

Column j of $\mathbf{W}^{[1]}$ holds the weights of hidden unit j ; the superscript $[l]$ is the layer number.

Each hidden unit is a logistic regression on the inputs. The output is a logistic regression on the **hidden units**.

Parameters: $3 \cdot 4 + 4 = 16$ in layer 1, $4 \cdot 1 + 1 = 5$ in layer 2. **21** in total. "Fully connected" or "dense": every unit sees every unit of the previous layer.

Going deeper: features of features



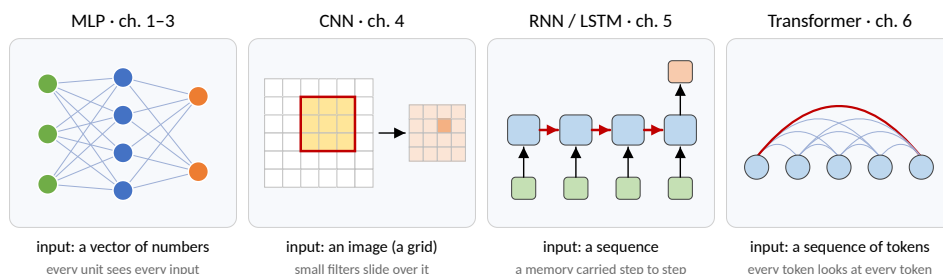
A 2-4-4-1 network (two hidden layers, tanh) trained on the circle data. Each heat map shows one unit's value over the whole plane.

$$\mathbf{a}^{[1]} = g(\mathbf{x}\mathbf{W}^{[1]} + \mathbf{b}^{[1]}), \quad \mathbf{a}^{[2]} = g(\mathbf{a}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]})$$

$$\hat{y} = \sigma(\mathbf{a}^{[2]}\mathbf{W}^{[3]} + \mathbf{b}^{[3]})$$

Layer 1 draws **lines**, layer 2 combines lines into **curved regions**, the output combines regions. More layers give more complex shapes with fewer units. On images: pixels → edges → textures → parts → objects (chapter 4). That is the **deep** in deep learning.

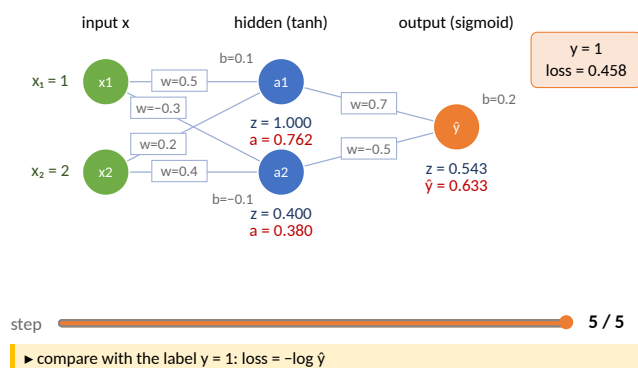
The neural-network family of this course



All of them are layers of **weighted sums + non-linearities**, trained with a loss and gradient descent. What changes is **how the units are wired**, to match the structure of the data.

- **MLP**: no structure assumed. **CNN**: nearby pixels go together, a pattern can appear anywhere.
- **RNN**: order matters, keep a running memory. **Transformer**: any word can relate to any other, all in parallel.

The forward pass, with numbers



One example $\mathbf{x} = (1, 2)$, label $y = 1$. Weights chosen by hand (they would normally be random).

$L = -\log 0.633 = 0.458$. Not terrible, not great. Chapter 2 computes how to change each of the 9 parameters to lower it.

Forward pass = feed the input through the layers, left to right, storing every intermediate value.

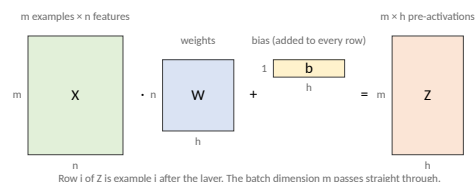
Multi-layer perceptron: Matrix notation

Stack the m examples as rows of $\mathbf{X}$ (so $\mathbf{A}^{[0]} = \mathbf{X}$). For each layer $l = 1, \dots, L$:

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]}, \quad \mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$$

object	shape
$\mathbf{A}^{[l-1]}$	$m \times n_{l-1}$
$\mathbf{W}^{[l]}$	$n_{l-1} \times n_l$
$\mathbf{b}^{[l]}$	$1 \times n_l$ (added to every row)
$\mathbf{Z}^{[l]}, \mathbf{A}^{[l]}$	$m \times n_l$

The whole forward pass is L matrix products and L element-wise functions: exactly what GPUs are fast at.



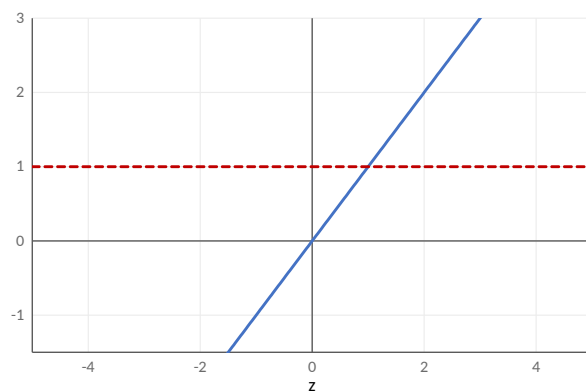
Some books write one example as a column, $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$: the same thing, transposed. The row convention matches PyTorch: `nn.Linear(in_features=n, out_features=h)` stores its weight as $h \times n$ and computes $\mathbf{x}\mathbf{W}^T + \mathbf{b}$.

Activation Layers:

- Apply **non-linearities**, element by element.
- Fast to compute, with fast-to-calculate derivatives (needed for gradient descent).

activation	range	use it for
Sigmoid	(0, 1)	output of binary classification; gates (ch. 5)
Tanh	(-1, 1)	small nets, RNN hidden state
ReLU	[0, ∞)	default for hidden layers (MLP, CNN)
LeakyReLU	$\mathbb{R}$	when many ReLUs "die" (always 0)
GELU	$\approx [-0.17, \infty)$	transformers (a smooth ReLU)
Linear	$\mathbb{R}$	output of regression only

ReLU is not differentiable at 0; we use a sub-derivative (0 or 1). It never matters in practice. Sigmoid and tanh **saturate**: their derivative is almost 0 for large $|z|$ (chapter 2 shows why that hurts). Press → to go through them; their derivatives come next, one by one.



Sigmoid Tanh ReLU LeakyReLU GELU **Linear**

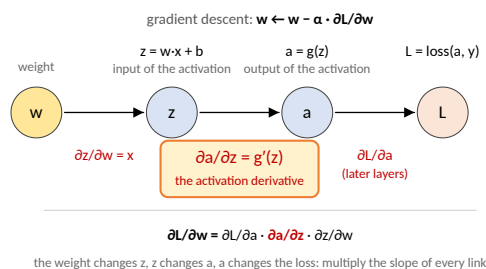
— $g(z)$ — derivative $g'(z)$

derivative

$g'(z) = 1$

► Linear: $g(z) = z$, slope 1. Only for the output of a regression

Why do we need the derivatives of the activations?



Training a network is **gradient descent**. For every weight w (and every bias) we repeat

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$$

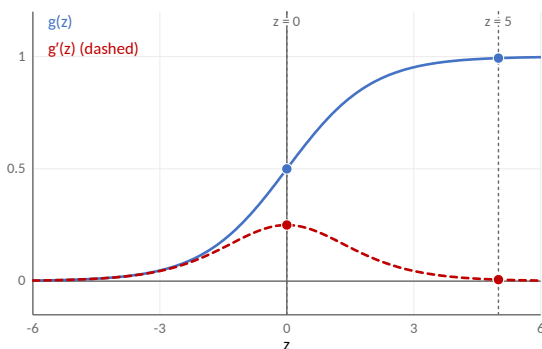
So we need $\frac{\partial L}{\partial w}$ for **every** weight. A weight sits at the start of a chain: it changes z , which changes the output $a = g(z)$, which changes the loss. The chain rule multiplies the slope of each link:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

The middle link is always an activation. Its slope is $\frac{\partial a}{\partial z} = g'(z)$: how much the output a moves when its **input** z is nudged. This is the derivative of the activation function, taken with respect to z . The next slides compute it for each activation.

These derivatives will be important in chapter 2: backpropagation uses them in every layer, so we calculate them once, here.

Derivative of the sigmoid



$$\sigma(z) = \frac{1}{1 + e^{-z}} = (1 + e^{-z})^{-1}$$

write the fraction as a power

$$\sigma'(z) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

chain rule: $(u^{-1})' = -u^{-2} u'$, and for $u = 1 + e^{-z}$ we have $u' = -e^{-z}$

$$= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$$

split the square into two factors

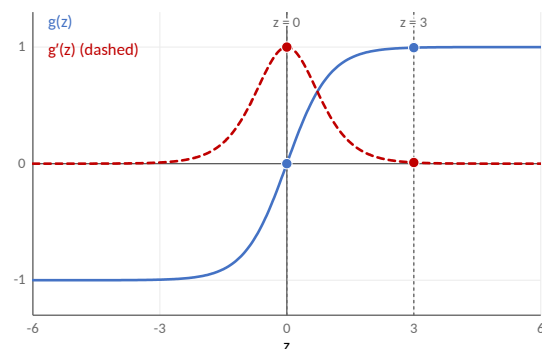
$$\sigma'(z) = \sigma(z) (1 - \sigma(z))$$

the first factor is $\sigma(z)$; the second is $1 - \sigma(z)$, because $1 - \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}}$

z	-5	0	1	5
$\sigma(z)$	0.007	0.5	0.731	0.993
$\sigma'(z)$	0.0066	0.25	0.197	0.0066

The slope is largest at $z = 0$ (0.25) and almost 0 when $|z|$ is large: the sigmoid **saturates**.

Derivative of tanh



1 • **A fraction.** Call the top N and the bottom D :

$$\tanh z = \frac{N}{D}, \quad N = e^z - e^{-z}, \quad D = e^z + e^{-z}$$

2 • **Differentiate the pieces.** The derivative of e^{-z} is $-e^{-z}$, so the two pieces swap:

$$N' = e^z + e^{-z} = D, \quad D' = e^z - e^{-z} = N$$

3 • **Quotient rule,** $\left(\frac{N}{D}\right)' = \frac{N'D - ND'}{D^2}$, with $N' = D$ and $D' = N$:

$$\tanh' z = \frac{D \cdot D - N \cdot N}{D^2}$$

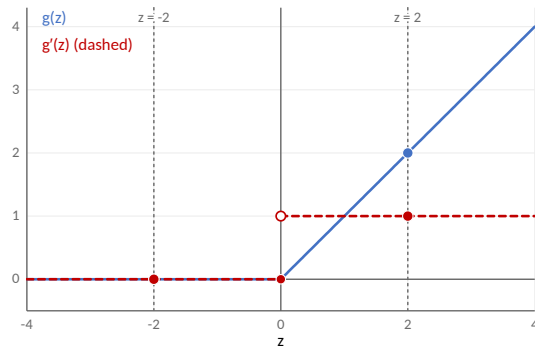
4 • **Split the fraction:** $\frac{D^2}{D^2} - \frac{N^2}{D^2} = 1 - \left(\frac{N}{D}\right)^2$

$$\tanh' z = 1 - \tanh^2 z$$

Numbers. $z = 0$: $\tanh = 0$ and $\tanh' = 1$ (the largest slope). $z = 1$: 0.762 and $1 - 0.762^2 = 0.420$. $z = 3$: 0.995 and 0.0099 , almost flat.

Check with the sigmoid: $\tanh z = 2\sigma(2z) - 1$, so $\tanh' z = 4\sigma'(2z) = 4 \cdot \frac{1 + \tanh z}{2} \cdot \frac{1 - \tanh z}{2} = 1 - \tanh^2 z$. Its slope at 0 is 4 times larger than the sigmoid's.

Derivative of ReLU



$\text{ReLU}(z) = \max(0, z)$ is made of two straight pieces:

$$\text{ReLU}(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases} \quad \text{ReLU}'(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases}$$

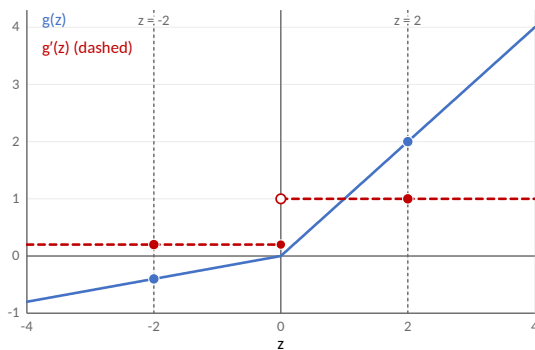
the slope of each piece: a flat line has slope 0, the line $y = z$ has slope 1

At $z = 0$ the two pieces meet in a corner: no derivative. We simply choose one (PyTorch uses 0). It never matters: z is almost never exactly 0.

z	-2	0	3
$\text{ReLU}(z)$	0	0	3
$\text{ReLU}'(z)$	0	0 (chosen)	1

- **Cheap:** a comparison, no exponential.
- **Never shrinks** a gradient it lets through: the slope is exactly 1.
- **The catch:** a unit with $z < 0$ for every example has slope 0 everywhere and stops learning (a "dead" ReLU). The fix is LeakyReLU.

Derivative of LeakyReLU



Same as ReLU, but the left piece is not flat: it has a small slope α .

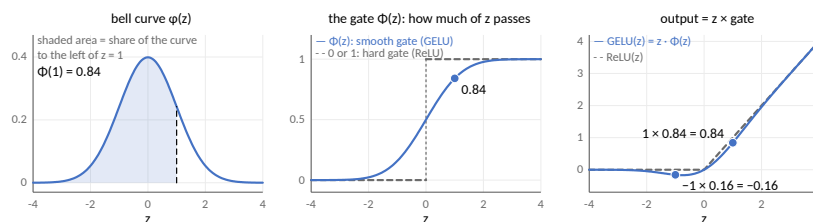
$$\text{LeakyReLU}(z) = \begin{cases} \alpha z & z < 0 \\ z & z > 0 \end{cases} \quad \text{LeakyReLU}'(z) = \begin{cases} \alpha & z < 0 \\ 1 & z > 0 \end{cases}$$

again the slope of each straight piece

z (with $\alpha = 0.2$)	-3	2
$\text{LeakyReLU}(z)$	-0.6	2
$\text{LeakyReLU}'(z)$	0.2	1

- The slope is **never 0**, so a unit can always recover: no dead units.
- α is a small fixed number: 0.01 is the PyTorch default (the figure uses 0.2 so that you can see it).
- $\alpha = 0$ is ReLU, $\alpha = 1$ would be the linear activation.

GELU: z times a smooth gate



ReLU is " z times a hard gate": $\text{ReLU}(z) = z \cdot \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases}$.

The gate is shut for negative z and open for positive z .

GELU keeps the idea, with a smooth gate:

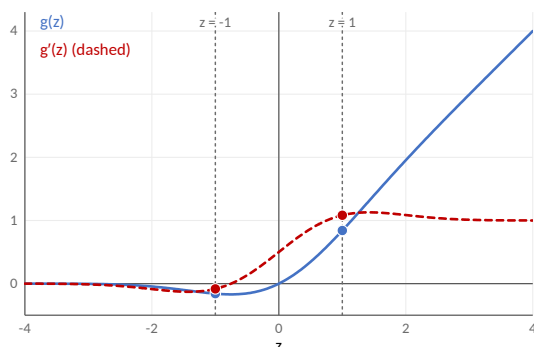
$$\text{GELU}(z) = z \cdot \Phi(z)$$

$\Phi(z)$ (capital phi) is the function that multiplies z . It is the share of the bell curve that lies to the left of z (the shaded area, left plot), so it grows smoothly from 0 to 1.

z	-3	-1	0	1	3
gate $\Phi(z)$	0.001	0.159	0.5	0.841	0.999
$z \cdot \Phi(z)$	-0.004	-0.159	0	0.841	2.996

- z very negative: the gate is almost shut, the output is almost 0.
- z very positive: the gate is almost open, the output is almost z .
- Around 0 the gate is half open: a smooth bend, and a small dip below 0.

Derivative of GELU



$$\text{GELU}(z) = z \cdot \Phi(z)$$

z times the smooth gate $\Phi(z)$ of the previous slide

$$\text{GELU}'(z) = 1 \cdot \Phi(z) + z \cdot \Phi'(z)$$

product rule: $(uv)' = u'v + uv'$

$$\Phi'(z) = \varphi(z) = \frac{e^{-z^2/2}}{\sqrt{2\pi}}$$

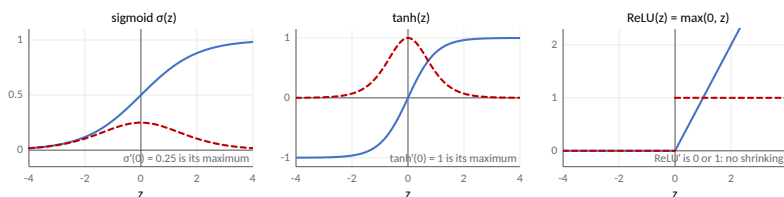
the slope of the area function Φ is the height of the bell curve φ

$$\text{GELU}'(z) = \Phi(z) + z\varphi(z)$$

z	-3	-1	0	1	3
$\text{GELU}(z)$	-0.004	-0.159	0	0.841	2.996
$\text{GELU}'(z)$	-0.012	-0.083	0.5	1.083	1.012

- No corner: the slope goes smoothly from about 0 (very negative z) to about 1 (large positive z), overshooting 1 a little (up to 1.13).
- The figure uses the usual tanh approximation of Φ (as in GPT-2): within 0.0005 of the exact curve.

Derivatives of the activations, at a glance



activation	$a = g(z)$	$g'(z) = \partial a / \partial z$	g' written with a	largest g'
Sigmoid	$\frac{1}{1 + e^{-z}}$	$\sigma(z)(1 - \sigma(z))$	$a(1 - a)$	0.25
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2 z$	$1 - a^2$	1
ReLU	$\max(0, z)$	1 if $z > 0$, else 0	1 if $a > 0$, else 0	1
LeakyReLU	z if $z > 0$, else αz	1 if $z > 0$, else α	1 if $a > 0$, else α	1
GELU	$z \Phi(z)$	$\Phi(z) + z \varphi(z)$	needs z itself	1.13
Linear	z	1	1	1

Two things to remember about the derivatives

1 • The slope $g'(z)$ can be computed from the output a

$g'(z) = \frac{\partial a}{\partial z}$ is the derivative of the output with respect to its input z . In the forward pass we computed z and stored $a = g(z)$. If g' can be written with a , nothing has to be computed again: one multiplication.

activation	g' with the output
sigmoid	$a(1 - a)$
tanh	$1 - a^2$
ReLU	1 if $a > 0$, else 0

Sigmoid unit with $z = 1$: the stored output is $a = 0.731$, so $g' = 0.731 \cdot 0.269 = 0.197$. **Tanh unit** of the tiny network, $a = 0.76$: $g' = 1 - 0.76^2 = 0.42$.

2 • Sigmoid shrinks the gradient, ReLU lets it through

In backpropagation (chapter 2) the gradient is multiplied by one g' per layer. The sigmoid never gives more than 0.25:

layers crossed	1	2	5	10
sigmoid, at most	0.25	0.063	0.001	0.000001
ReLU, active unit	1	1	1	1

After 10 sigmoid layers a gradient is at most a millionth of what it was: the first layers barely learn. A ReLU passes it on **unchanged** wherever the unit is active.

This is the **vanishing gradient**, and the main reason why ReLU is the default activation of hidden layers.

Linear activation layer:

- It is only used for the **last layer** and a **regression** problem.
- Why is it not used in hidden layers?

Proof. With $g(z) = z$ in the hidden layer:

$$\begin{aligned} h(\mathbf{x}) &= (\mathbf{x}\mathbf{W}^{[1]} + \mathbf{b}^{[1]})\mathbf{W}^{[2]} + \mathbf{b}^{[2]} \\ &= \mathbf{x}\underbrace{\mathbf{W}^{[1]}\mathbf{W}^{[2]}}_{\tilde{\mathbf{W}}} + \underbrace{\mathbf{b}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]}}_{\tilde{\mathbf{b}}} \end{aligned}$$

A composition of linear maps is linear: **any number of linear layers collapses into one**. Depth buys nothing without a non-linearity.

This is why every hidden layer has an activation. The non-linearity is what lets a stack of layers build new features instead of one big linear map.

With the numbers of the tiny network (the forward-pass example), if we removed the tanh:

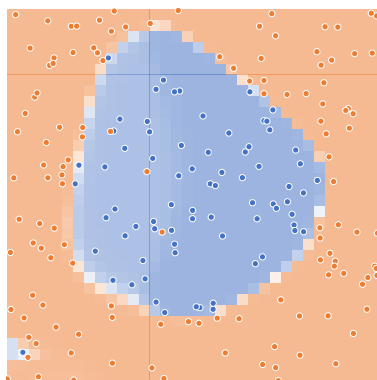
$$\begin{aligned} \tilde{\mathbf{W}} &= \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix} \begin{pmatrix} 0.7 \\ -0.5 \end{pmatrix} = \begin{pmatrix} 0.50 \\ -0.06 \end{pmatrix} \\ \tilde{b} &= 0.1 \cdot 0.7 + (-0.1) \cdot (-0.5) + 0.2 = 0.32 \end{aligned}$$

The two-layer "network" is just the single neuron

$$\hat{y} = 0.50 x_1 - 0.06 x_2 + 0.32.$$

Try it on the next slide: set the activation to **Linear** and add as many units as you like. The boundary stays a straight line.

Depth and non-linearity:



hidden units per layer 6

hidden layers 2

Tanh ReLU Linear

► back to tanh, 2 layers of 6: a rounder region

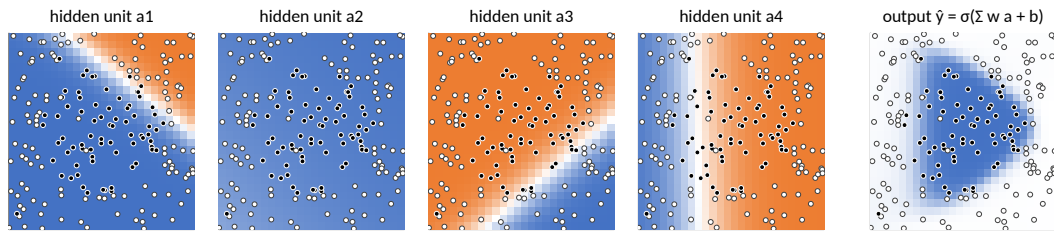
Like the [TensorFlow playground](#): a network trained from scratch here, in your browser, on only x_1, x_2 (no hand-made features).

model	parameters	train accuracy	loss
a neural network	67	0.985	0.067

- 0 hidden units:** logistic regression, a straight line.
- 3-4 units:** a closed curve. The hidden layer learned the features.
- Linear activation:** still a line, however deep.

We can learn non-linear functions without performing feature engineering! 4 % of the labels are flipped on purpose (noise), so 100 % is not the goal. Press → to run the scripted tour.

What did the hidden units learn?

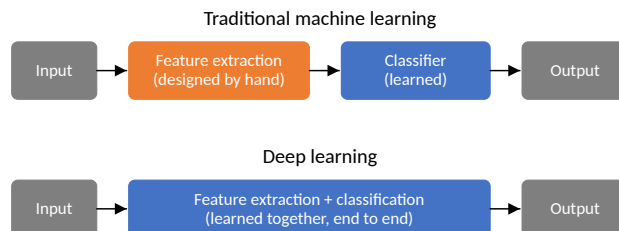


Each hidden unit is a soft half-plane (a tilted sigmoid / tanh in x_1, x_2). The output layer adds them with weights $w = (-5.38, 8.25, 6.08, 6.06)$ and $b = -2.84$: four "walls" combine into a closed region. That is what "learned features" means.
hidden units (tanh): orange = +1, blue = -1 · output: dark blue = high probability · dots: black = inside (class 1), white = outside

The same network as before (4 tanh units, one layer). Each hidden unit on its own is a **logistic regression**: a soft line through the plane. None of them can see a circle.

The output unit is **another logistic regression on the four hidden activations**. Added with the right signs, four half-planes fence in a region. With ReLU the walls are sharp; with more units the fence is rounder.

Different approach:



- **Machine learning**: we design features ($x_1^2, x_1x_2, \sin x...$), then fit a simple model on them.
- **Neural networks**: one hidden layer learns a few features from the raw inputs.
- **Deep learning**: many layers learn features of features: edges → shapes → objects (chapter 4).

playground.tensorflow.org

Deep Learning is not magic:

The applications that you saw at the beginning are not magic. They are simply a combination of:

- Linear combinations.
- Simple non-linear functions.
- A stack of the previous two.

Keep calm and linear algebra. Keep calm and optimize.

The most complex computer-vision networks are just a combination of many "small and easy blocks" that you can understand and even program!

The main ideas behind finding the best weights are the same tools that you already have from machine learning: a loss, its gradient, and gradient descent.

We will explain and understand small blocks, then build complex nets by abstracting each block (like LEGO).

Output layer and loss: which pair?

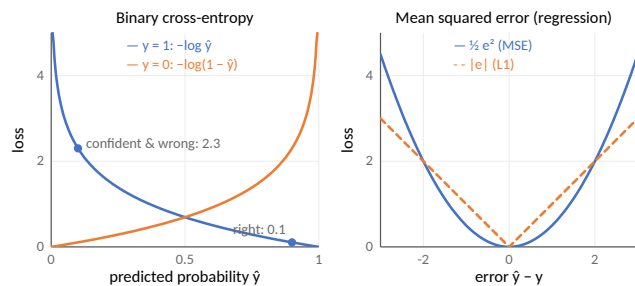
The last layer and the loss go together, exactly as in the GLMs of the ML part:

Problem type	Example	Last-layer activation	Loss function	PyTorch
Binary classification	spam / not spam	sigmoid	binary cross-entropy	<code>nn.BCEWithLogitsLoss</code>
Multiclass, single-label	digit 0-9	softmax	categorical cross-entropy	<code>nn.CrossEntropyLoss</code>
Multiclass, multi-label	tags of a photo	sigmoid (per class)	binary cross-entropy	<code>nn.BCEWithLogitsLoss</code>
Regression, any value	house price	none (linear)	MSE (or L1 / Huber)	<code>nn.MSELoss</code>
Regression, values in 0-1	box coordinates	sigmoid	MSE or binary cross-entropy	<code>nn.MSELoss</code>

PyTorch is the library we will use for the labs, and the standard one in deep learning. The last column lists its loss classes. We will talk about it more later.

The multiclass rows (softmax, one-hot targets, categorical cross-entropy) are explained later in this chapter, together with where these pairs come from and how PyTorch uses them.

What a loss looks like



Cross-entropy punishes confident mistakes very hard: predicting 0.1 for a positive costs 2.3, predicting 0.01 costs 4.6. Being right and confident costs almost nothing.

The loss of the dataset is the **average** over examples. Gradient descent lowers that average; it cannot make every example perfect.

MSE squares the error: big errors dominate, outliers pull hard. **L1** and **Huber** (quadratic near 0, linear far away) are more robust: used for box regression in detection.

Rule: pick the loss from the type of y , not from taste.

The most useful derivative: sigmoid + cross-entropy

One example, label $y \in \{0, 1\}$, prediction $\hat{y} = \sigma(z)$, loss

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

1 • Derivative with respect to the prediction:

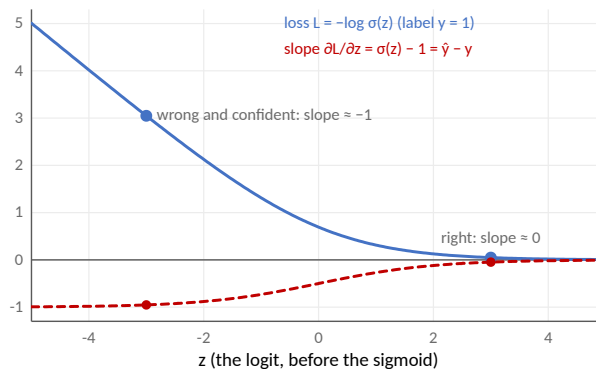
$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$$

2 • The slope of the sigmoid, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, with

$$\hat{y} = \sigma(z): \frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$$

3 • Chain rule: the two $\hat{y}(1 - \hat{y})$ cancel:

$$\frac{\partial L}{\partial z} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \cdot \hat{y}(1 - \hat{y}) = \hat{y} - y$$



"Prediction minus target": large when the network is confidently wrong, about 0 when it is right. The same happens for softmax + cross-entropy (coming up) and for a linear output + $\frac{1}{2}$ MSE. Chapter 2 starts from here.

More than two classes

Binary: two classes

Is this email spam? **One** output is enough: $\hat{y} = P(\text{spam})$, a number in $(0, 1)$. The other class gets $1 - \hat{y}$.

Multiclass: K classes, one is right

What is in this photo: a *cat*, a *dog*, a *car*, a *tree* or a *boat*? We need K numbers: $P(\text{cat}), \dots, P(\text{boat})$. All ≥ 0 , and adding up to 1.

- Digits 0–9 ($K = 10$) · iris species ($K = 3$) · damage severity: minor, moderate, severe ($K = 3$) · the next word of a sentence ($K \approx 50\,000$).
- **Multi-label** is different: several classes can be true at once (the tags of a photo). Each tag then gets its own independent sigmoid.

The plan: **one output neuron per class**, and a way to turn the K scores into K probabilities that add up to 1.

The classes as one-hot vectors

A label is a name. How do we write it as numbers? PyTorch counts classes from 0.

label	integer	one-hot vector
cat	0	(1, 0, 0, 0, 0)
dog	1	(0, 1, 0, 0, 0)
car	2	(0, 0, 1, 0, 0)
tree	3	(0, 0, 0, 1, 0)
boat	4	(0, 0, 0, 0, 1)

Integers are compact, but they invent an order and distances: "car = 2" looks like twice a dog. None of that is true.

One-hot: K entries, a 1 in the position of the true class and 0 elsewhere. No order, every class equally far from the others.

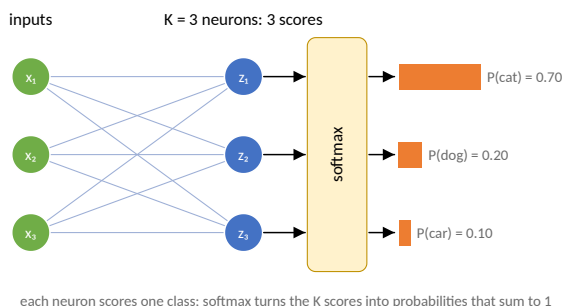
A one-hot vector is also a **probability distribution**: all the probability on the true class. The network will output another one, $\hat{y}$, and training pushes it towards the target y (true class: dog):

	cat	dog	car	tree	boat
target y	0	1	0	0	0
prediction $\hat{y}$	0.02	0.90	0.05	0.01	0.02

Both are vectors of K numbers that add up to 1, so they can be compared entry by entry.

For a whole dataset the targets form a matrix $\mathbf{Y}$ with m rows (one per example) and K columns. Each row has a single 1.

A layer of neurons: one score per class



K classes $\rightarrow K$ neurons side by side, each with its own weights. Each one computes the **score** of its class, called a **logit**:

$$z_k = \mathbf{w}_k^\top \mathbf{x} + b_k, \quad k = 1, \dots, K$$

All the weights together form a matrix $\mathbf{W}$ of size $n \times K$, and the whole layer is $\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}$.

Still linear. The border between two classes is where $z_k = z_j$, i.e. $(\mathbf{w}_k - \mathbf{w}_j)^\top \mathbf{x} + (b_k - b_j) = 0$: a straight line. One layer, however wide, cannot draw curved borders.

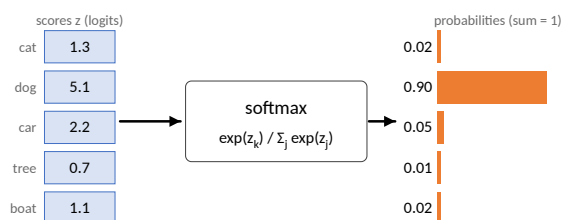
The scores are not probabilities: they can be negative and they do not add up to 1. One more block turns them into K probabilities: the **softmax**. (You met this model in the ML part as softmax regression.)

Softmax activation layer:

- Turns the K scores (logits) of the last layer into K **probabilities**: all positive, adding up to 1.
- It is applied to the whole **vector** of scores at once, not to each number separately like ReLU or the sigmoid.
- For problems where exactly one class is right.
- The output is a "soft" version of the one-hot target, and the cross-entropy loss compares the two.

$$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

$$L = - \sum_k y_k \log \hat{y}_k = - \log \hat{y}_{\text{true class}}$$



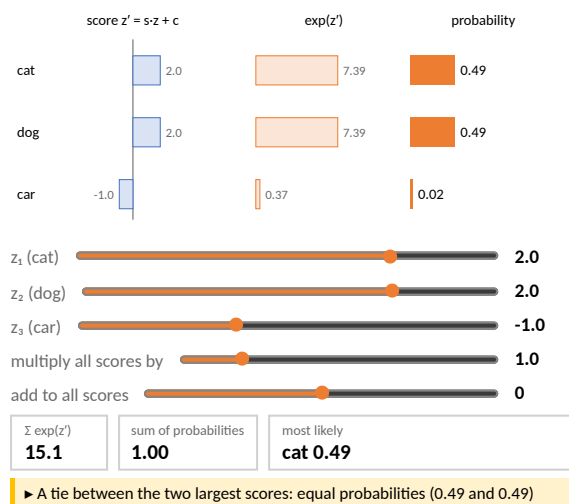
The exponential makes the biggest score dominate (5.1 $\rightarrow$ 0.90) but never sends the others to exactly 0.

Softmax, step by step with numbers

	cat	dog	car	tree	boat	sum
scores z_k	1.3	5.1	2.2	0.7	1.1	
1 • exponentiate: e^{z_k}	3.67	164.02	9.03	2.01	3.00	181.73
2 • divide by the sum	0.02	0.90	0.05	0.01	0.02	1.00

- **Exponentiate.** e^z is always positive, even for a negative score, and it **stretches the gaps**: dog is 2.9 points of score above car, but $e^{2.9} \approx 18$ times above it in e^z (164.02 against 9.03).
- **Divide by the sum** (181.73): the five numbers now add up to 1. For dog: $\frac{164.02}{181.73} = 0.90$. (Rounded to two decimals here.)
- **The loss.** If the true class is dog, the cross-entropy is $-\ln 0.90 = 0.10$: small. If it had been cat: $-\ln 0.02 = 3.9$: large. The softmax gradient calculation reuses these probabilities.

Softmax: playing with the scores



What softmax does to the scores:

- **Probabilities, always:** positive and adding up to 1, whatever the scores, negative ones included.
- **The order is kept:** the largest score gets the largest probability, so the arg-max of the scores is the predicted class.
- **Only differences matter:** adding the same number to every score changes nothing (slider "add"). In code the maximum score is subtracted first, so e^z cannot overflow.
- **The scale sets the confidence:** large scores give a nearly one-hot answer, small ones a nearly uniform one. In chapter 6 the scores are divided by a **temperature T** : the same thing with $s = 1/T$.
- **Two classes = sigmoid:** for $K = 2$, $\hat{y}_1 = \sigma(z_1 - z_2)$. Scores (2, 1): $\frac{e^2}{e^2 + e^1} = 0.73 = \sigma(1)$.

Softmax - gradient calculation:

Step 1. Substitute the softmax into the cross-entropy:

$$L = - \sum_j y_j \log \frac{e^{z_j}}{\sum_k e^{z_k}} = - \sum_j y_j \left(z_j - \log \sum_k e^{z_k} \right)$$

Since $\mathbf{y}$ is one-hot ($\sum_j y_j = 1$, and $y_i = 1$ for the correct class i), this simplifies to:

$$L = - \left(z_i - \log \sum_k e^{z_k} \right)$$

Step 2. Differentiate with respect to z_j . Two cases:

Case 1: $j = i$ (correct class).

$$\frac{\partial L}{\partial z_i} = - \left(1 - \frac{e^{z_i}}{\sum_k e^{z_k}} \right) = \hat{y}_i - 1$$

Case 2: $j \neq i$ (incorrect classes).

$$\frac{\partial L}{\partial z_j} = - \left(0 - \frac{e^{z_j}}{\sum_k e^{z_k}} \right) = \hat{y}_j$$

Vectorized form, for all classes:

$$\frac{\partial L}{\partial z_j} = \hat{y}_j - y_j$$

With the example above and true class "dog": gradient = (0.02, 0.90 - 1, 0.05, 0.01, 0.02): push the dog score up, all the others slightly down.

Where the losses come from: maximum likelihood

Say how likely each label is, according to the network. The loss is **minus the log** of the probability of the label that really happened. Three kinds of label, three losses:

label	the network says	probability of the true label	minus the log = the loss
a real number	a bell curve centred on $\hat{y}$	$\propto \exp \left(- \frac{(y - \hat{y})^2}{2\sigma^2} \right)$	$\frac{1}{2} (y - \hat{y})^2 + \text{constant}$: the MSE
0 or 1	Bernoulli: $P(1) = \hat{y}$	$\hat{y}^y (1 - \hat{y})^{1-y}$	$-y \log \hat{y} - (1 - y) \log(1 - \hat{y})$: the BCE
one of K	categorical: $P(k) = \hat{y}_k$	$\prod_k \hat{y}_k^{y_k} = \hat{y}_{\text{true}}$	$-\sum_k y_k \log \hat{y}_k$: the cross-entropy

- Over a dataset the likelihood is a **product** over examples. The log turns it into a sum, and the minus turns "maximize" into "minimize".
- Numbers: label 1 and $\hat{y} = 0.8$ costs $-\ln 0.8 = 0.22$; with $\hat{y} = 0.1$ it costs $-\ln 0.1 = 2.30$. Label 3 and $\hat{y} = 2.5$ costs $\frac{1}{2} \cdot 0.5^2 = 0.125$.
- Same recipe as in the ML part: least squares and logistic regression were maximum likelihood too.

The matching pairs: the gradient is prediction – target

Pair each output layer with its loss, and the derivative with respect to the logit z is always the same simple error:

output layer	loss	$\partial L / \partial z$
linear	$\frac{1}{2}(\hat{y} - y)^2$	$\hat{y} - y$
sigmoid	binary cross-entropy	$\hat{y} - y$
softmax	categorical cross-entropy	$\hat{y}_k - y_k$ for each class

The last two are the derivations we did for sigmoid + cross-entropy and for softmax + cross-entropy. For the first: $\hat{y} = z$, so

$$\frac{\partial}{\partial z} \frac{1}{2} (z - y)^2 = z - y.$$

$$\frac{\partial J}{\partial \mathbf{Z}^{[L]}} = \frac{1}{m} (\mathbf{A}^{[L]} - \mathbf{Y})$$

For a mini-batch: predictions minus targets, divided by m . This is the **first line of backpropagation** (chapter 2).

A mismatched pair is worse. Sigmoid with the squared error: the derivative gets an extra factor $\hat{y}(1 - \hat{y})$.

label $y = 1$, logit $z = -5$ ($\hat{y} = 0.0067$)	
sigmoid + BCE	$\hat{y} - y = -0.993$
sigmoid + $\frac{1}{2}$ MSE	$(\hat{y} - y) \hat{y}(1 - \hat{y}) = -0.0066$

A confidently wrong prediction gets a gradient **150 times smaller**: it learns slowly exactly when it should learn fast. Cross-entropy cancels the sigmoid's flat tails.

Losses in PyTorch take the logits

task	the model ends with	loss	targets
binary	raw logit z	<code>nn.BCEWithLogitsLoss</code>	floats 0. / 1.
multiclass	raw logits $\mathbf{z}$, $[B, K]$	<code>nn.CrossEntropyLoss</code>	class indices, $[B]$
multiclass	<code>LogSoftmax</code>	<code>nn.NLLLoss</code>	class indices, $[B]$
regression	the value $\hat{y}$	<code>nn.MSELoss</code>	real numbers

- `CrossEntropyLoss` = softmax + log + `NLLLoss` in one step. `BCEWithLogitsLoss` = sigmoid + BCE in one step. `NLLLoss` expects **log-probabilities**, not probabilities.
- No softmax or sigmoid at the end of the model** when you use them. Applying softmax first, and then `CrossEntropyLoss`, applies it twice: it still trains, but worse.
- Probabilities, only when you need them (prediction time):
`logits.softmax(dim=1), torch.sigmoid(z)`.

Why logits are safer. Computed in two steps, the exponentials overflow or underflow:

case	two steps	one step
logits (1000, 0, 0)	<code>exp(1000)</code> overflows: nan	log-probabilities (0, ~1000, ~1000)
logit -200, label 1	<code>BCELoss(sigmoid)</code> : 100 (capped)	<code>BCEWithLogitsLoss</code> : 200

The one-step versions subtract the largest logit before exponentiating (the log-sum-exp trick).

```
logits = model(x)          # raw scores [B, K]
loss = nn.CrossEntropyLoss()(logits, y)
probs = logits.softmax(dim=1) # to read answers
```

Universal approximation

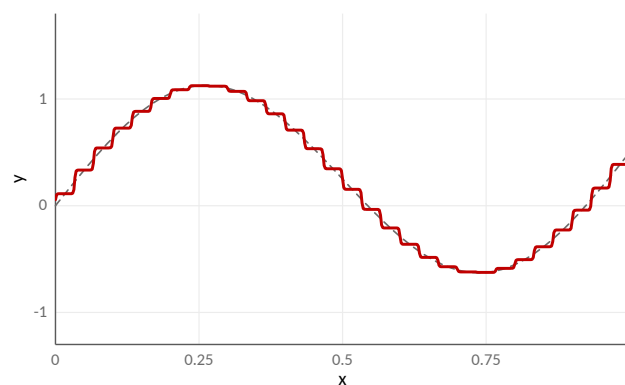
A network with **one hidden layer** and enough units can approximate any continuous function on a bounded interval.

A constructive picture. Two steep sigmoids make a step up and a step down:

$$\text{bump}_{[a,b]}(x) = \sigma(k(x-a)) - \sigma(k(x-b))$$

Put a bump of height $f(\text{centre})$ on each small interval and add them: the output layer does the adding. More units, finer bumps.

It says such weights **exist**, not that gradient descent finds them, nor how many units are needed. In practice, depth is far more efficient than width.



bumps 30

hidden units used
60

► thirty bumps (60 hidden units): close to the curve, except at the two ends

The loss is no longer convex

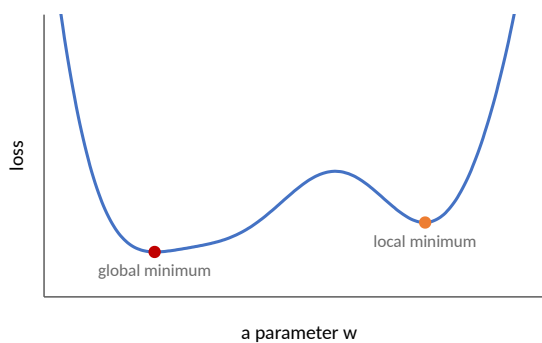
Symmetry argument. Swap two hidden units (their incoming and outgoing weights). The network computes **exactly the same function**, so the loss is the same.

So if $\mathbf{w}^*$ is a minimum, the swapped $\mathbf{w}^{**}$ is another one. Their average $\frac{1}{2}(\mathbf{w}^* + \mathbf{w}^{**})$ gives both units the same weights: generally a **worse** network.

A convex function would satisfy

$$J\left(\frac{\mathbf{w}^* + \mathbf{w}^{**}}{2}\right) \leq \frac{1}{2}J(\mathbf{w}^*) + \frac{1}{2}J(\mathbf{w}^{**}). \text{ It fails: not convex. } \blacksquare$$

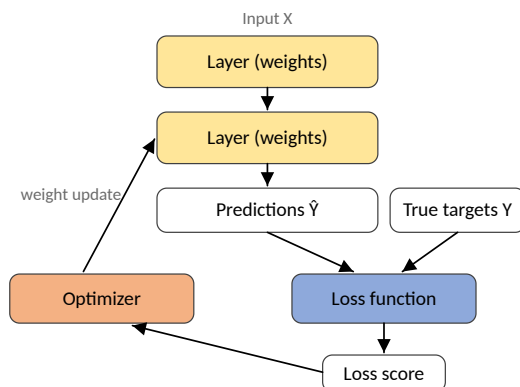
- Many equivalent minima, plus saddle points and plateaus.
- The start (random initialization) matters. Large networks usually find good minima anyway.
- We will need better optimizers than vanilla gradient descent (chapter 3).



Learning workflow:

- Start with a labelled dataset.
- Design an **architecture** (layers, units, activations).
- Decide a **loss function** (from the type of y).
- Decide an **optimizer** (chapter 3).
- Apply an iterative algorithm (gradient descent): forward, loss, backward, update, repeat.
- The engine of neural networks: **gradient-based optimization!**
 - Note: we do not mention the word "convex" anymore.

The first step to perform gradient-based optimization is to find the **gradients**: next chapter.



Vocabulary of chapter 1

neuron / unit	weighted sum of its inputs + bias, then an activation. One neuron = logistic regression.	forward pass	computing $\hat{y}$ from $\mathbf{x}$, layer by layer, storing every $\mathbf{Z}$ and $\mathbf{A}$.
weights, bias	$\mathbf{w}$ and b : the θ of the ML part. What training changes.	one-hot	a vector with a 1 at the true class and 0 elsewhere: the target of a multiclass problem.
layer	a group of units that share the same inputs. Dense / fully connected: $\mathbf{Z} = \mathbf{A}\mathbf{W} + \mathbf{b}$.	logits	the output layer's $\mathbf{Z}$ before softmax / sigmoid. What the PyTorch losses expect.
activation	the element-wise non-linearity: ReLU (hidden), sigmoid / softmax / none (output).	loss	how wrong $\hat{y}$ is; minus a log-likelihood. Averaged over the batch.
hidden layer	any layer between input and output. Its activations are learned features .	softmax	turns K scores into K probabilities that add up to 1.
		MLP	multi-layer perceptron: a stack of dense layers with activations.
		deep learning	many layers: features of features, learned end to end.

Lab 7 (an MLP from scratch in NumPy: XOR, forward pass, backpropagation) comes at the end of chapter 2, once we know how to compute the gradients. Theory notes for this chapter: [notes 00 · Neural networks and the MLP](#).

Neural Networks

Máster en Innovación Tecnológica (MINT)

Next: Forward and backward propagation

imanolgranada@gmail.com

Neural Networks

Máster en Innovación Tecnológica (MINT)

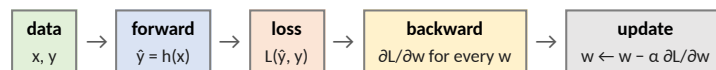
Forward and backward propagation

imanolgranada@gmail.com



1

Where we are



Chapter 1 gave us the **forward** pass and the **loss**. Gradient descent needs one more thing: the derivative of the loss with respect to **every** weight and bias (in the formulas, w stands for any of them).

A small MLP has thousands of parameters; GPT-2 has 124 million. We need a method that gets **all** the gradients cheaply and mechanically.

Backpropagation = the chain rule, applied from the loss backwards, reusing what the forward pass already computed.

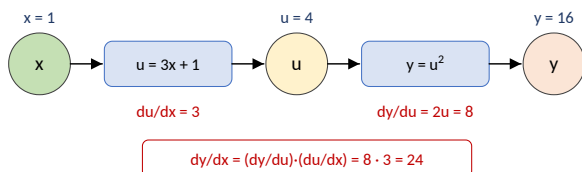
This chapter: a derivatives refresher, the chain rule on a graph with numbers, one neuron with numbers, the tiny network with numbers, then the general formulas and their code, then two practical topics (initialization, vanishing gradients). It ends with Lab 7.



2

Refresher: a derivative is a sensitivity

the derivative predicts what a small nudge does



check: $x = 1.01 \rightarrow u = 4.03 \rightarrow y = 16.2409$, so y grew by $0.24 = 24 \times 0.01$ ✓

step 3 / 3

► check with a nudge of 0.01: y changes by $0.24 = 24 \times 0.01$

Derivative $\frac{dy}{dx}$: how much y changes when x is nudged a little, $\Delta y \approx \frac{dy}{dx} \Delta x$.

Chain rule: if y depends on u and u depends on x , the sensitivities **multiply**:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Partial derivative $\frac{\partial L}{\partial w}$: the same, for one input while all the others are held fixed. The **gradient** $\nabla_w L$ is the vector of all of them.

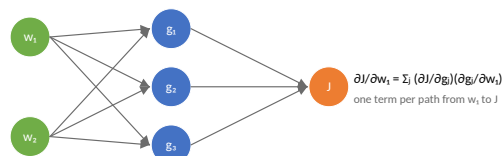
A neural network is a long chain of simple functions. Backprop = the chain rule, applied along the chain from the loss back to each weight.

Gradient Calculation:

- We could write the network as one very complex function and find all derivatives by hand.
 - This is very complex and inefficient.
- We will use a trick named **back-propagation**.
 - Modern DL libraries (TensorFlow / PyTorch) construct **computational graphs** that do exactly this.

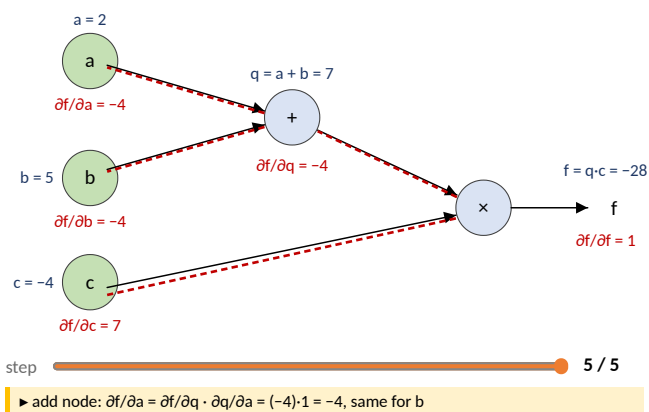
When a parameter reaches the loss through **several paths**, the chain rule adds one term per path. If $g_j = g_j(w_1, \dots, w_p)$ and $J = J(g_1, \dots, g_k)$:

$$\frac{\partial J}{\partial w_i} = \sum_{j=1}^k \frac{\partial J}{\partial g_j} \frac{\partial g_j}{\partial w_i}$$



$$\frac{df(g(x))}{dx} = \underbrace{\frac{df(g(x))}{dg(x)}}_{\text{outer, at the inner}} \cdot \underbrace{\frac{dg(x)}{dx}}_{\text{inner}}$$

A computation is a graph: $f = (a + b) \cdot c$



Every node does one simple operation. **Forward:** compute each node from its inputs, left to right.

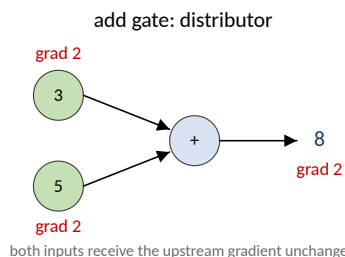
Backward: start with $\partial f / \partial f = 1$ and walk right to left. At each node,

gradient of an input = (gradient of the output) $\times$ (local derivative)

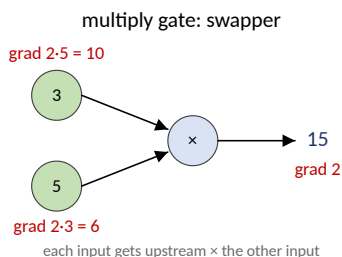
Meaning of the result: increase a by a tiny ϵ and f changes by -4ϵ . Check: $a = 2.01 \Rightarrow f = 7.01 \cdot (-4) = -28.04$. ✓

Each node needs only its **own** local derivative and the number coming from the right. No node has to know the whole formula. That is why it scales to millions of parameters and any architecture.

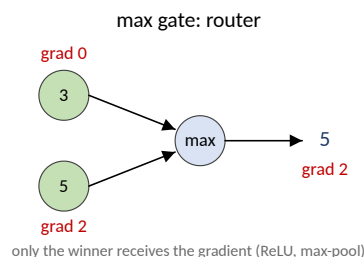
Three gates to remember



Add (biases, residual connections): the local derivative is 1 for each input, so the upstream gradient is **copied** to both branches.



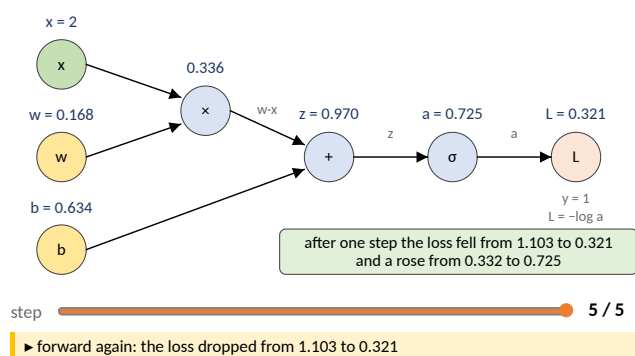
Multiply (weight $\times$ input): each input's gradient is the upstream gradient times the **other** input. A weight's gradient is proportional to the activation feeding it: $\partial L / \partial w = \delta \cdot a$.



Max (ReLU, max-pooling): the gradient goes entirely to the input that won; the others get exactly 0.

Every layer in this course is built from these plus the element-wise functions (sigmoid: local derivative $a(1 - a)$; tanh: $1 - a^2$; derived in chapter 1).

One neuron, forward and backward with numbers



$x = 2, w = -0.5, b = 0.3$, label $y = 1$, loss $L = -\log a$ (cross-entropy).

$z = 0.970, a = 0.725, L = 0.321$. One step of gradient descent, and the neuron went from 33 % to 73 % on this example.

Blue numbers were computed left to right and stored. Every red number reuses a blue one.

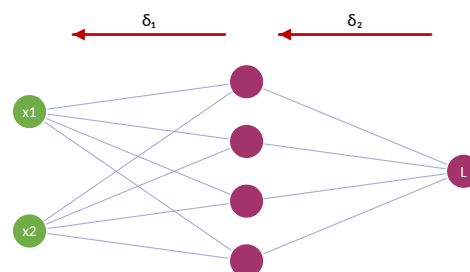
Why backpropagation is efficient

Naive way: nudge each of the P parameters and re-run the network: P forward passes. A small network has 10^5 parameters; GPT-2 has 1.2×10^8 .

Backpropagation = the chain rule applied from the loss backwards, **reusing** the partial results of the layer above.

- One forward pass: store every $\mathbf{Z}^{[l]}$ and $\mathbf{A}^{[l]}$.
- One backward pass: every gradient, at roughly the cost of two forward passes.

This is "reverse-mode automatic differentiation". It is why training networks with billions of weights is possible at all. Side effect: training needs several times the memory of inference, because all the activations are kept.



The error starts at the loss and travels backward: output $\rightarrow \delta_2 \rightarrow$ hidden $\rightarrow \delta_1 \rightarrow$ input

Forward and backward propagation:

Key notation	Shape
$\mathbf{X}$: input matrix, m samples	$m \times n_{\text{input}}$
$\mathbf{W}^{[1]}, \mathbf{b}^{[1]}$: input $\rightarrow$ hidden	$n_{\text{input}} \times n_{\text{hidden}},$ $1 \times n_{\text{hidden}}$
$\mathbf{Z}^{[1]}$: hidden pre-activation	$m \times n_{\text{hidden}}$
$\mathbf{A}^{[1]} = g(\mathbf{Z}^{[1]})$: hidden activation	$m \times n_{\text{hidden}}$
$\mathbf{W}^{[2]}, \mathbf{b}^{[2]}$: hidden $\rightarrow$ output	$n_{\text{hidden}} \times n_{\text{output}},$ $1 \times n_{\text{output}}$
$\mathbf{Z}^{[2]}, \mathbf{A}^{[2]} = g(\mathbf{Z}^{[2]})$: output (predictions)	$m \times n_{\text{output}}$
L : the loss (any differentiable function)	scalar

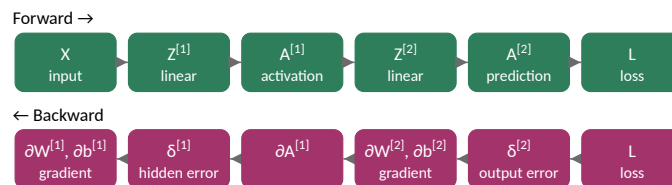
A network with one hidden layer, the same example for all the derivations. The superscript $[l]$ is the layer, as in chapter 1 and in Lab 7.

Forward: compute and **store** every intermediate result from $\mathbf{X}$ to L .

Backward: go from L back to $\mathbf{X}$, turning each stored result into a gradient.

Next slide: walk through both, one step at a time. Then the same walk with the numbers of the tiny network.

Forward and backward propagation: step by step



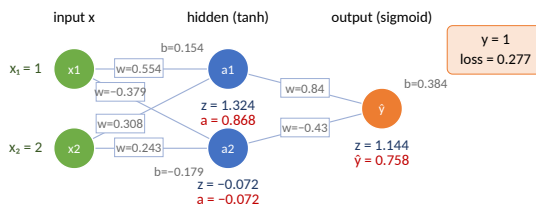
Step backward 6/6

Gradient with respect to $\mathbf{W}^{[1]}$ and $\mathbf{b}^{[1]}$. Same shape as for $\mathbf{W}^{[2]}$, one layer earlier. The whole path from $\mathbf{W}^{[1]}$ to L in one line; every bracket is a term we already have:

$$\frac{\partial L}{\partial \mathbf{W}^{[1]}} = \frac{\partial L}{\partial \mathbf{A}^{[2]}} \underbrace{\frac{\partial \mathbf{A}^{[2]}}{\partial \mathbf{Z}^{[2]}} \frac{\partial \mathbf{Z}^{[2]}}{\partial \mathbf{A}^{[1]}} \frac{\partial \mathbf{A}^{[1]}}{\partial \mathbf{Z}^{[1]}} \frac{\partial \mathbf{Z}^{[1]}}{\partial \mathbf{W}^{[1]}}}_{\delta^{[1]}} = \mathbf{X}^\top \delta^{[1]}$$

$\partial \mathbf{Z}^{[1]} / \partial \mathbf{W}^{[1]} = \mathbf{X}$ plays the role $\mathbf{A}^{[1]}$ played before. And $\frac{\partial L}{\partial \mathbf{b}^{[1]}} = \sum_{i=1}^m \delta_i^{[1]}$.

The tiny network: backward with numbers



updated parameters ($\alpha = 0.5$)

$W^{(1)} = [0.554, -0.379]$ $b^{(1)} = (0.154, -0.179)$ $W^{(2)} = (0.84, -0.43)$

$[0.308, 0.243]$ $b^{(2)} = 0.384$

new loss 0.277 (was 0.458)

step 7 / 7

► forward again with the new weights

Same network and example as chapter 1: $\mathbf{x} = (1, 2)$, $y = 1$, $\hat{y} = 0.633$, loss **0.458**.

New loss **0.277**, down from **0.458**. With a mini-batch, every gradient is the **average** of these numbers over the examples.

Backpropagation: the derivation

The **error signal** of layer l is the derivative of the loss with respect to $\mathbf{Z}^{[l]}$, the input of the activation: $\delta^{[l]} = \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \in \mathbb{R}^{m \times n_l}$. Each fact below is one more link of the chain on the right.

(a) **At the output.** Two links, $\mathbf{Z}^{[L]} \rightarrow \mathbf{A}^{[L]} \rightarrow J$: **already calculated in chapter 1**.

$$\delta^{[L]} = \frac{\partial J}{\partial \mathbf{A}^{[L]}} \odot g'(\mathbf{Z}^{[L]}) = \frac{1}{m} (\mathbf{A}^{[L]} - \mathbf{Y})$$

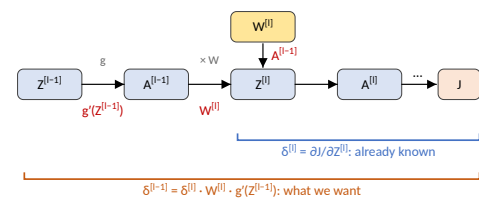
(b) **To the weights.** One more link, $\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]}$: the multiply gate, "times the other input".

$$\frac{\partial J}{\partial \mathbf{W}^{[l]}} = \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \cdot \frac{\partial \mathbf{Z}^{[l]}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} \cdot \mathbf{A}^{[l-1]} \rightarrow \mathbf{A}^{[l-1]\top} \delta^{[l]}$$

(c) **To the previous layer.** Two more links: through $\mathbf{W}^{[l]}$, and through the activation.

$$\delta^{[l-1]} = \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \cdot \frac{\partial \mathbf{Z}^{[l]}}{\partial \mathbf{A}^{[l-1]}} \cdot \frac{\partial \mathbf{A}^{[l-1]}}{\partial \mathbf{Z}^{[l-1]}}$$

$$\delta^{[l-1]} = (\delta^{[l]} \mathbf{W}^{[l]\top}) \odot g'(\mathbf{Z}^{[l-1]})$$



In the formulas: blue = terms we already have, orange = the new links, g' = the activation derivative of chapter 1.

Sanity check with shapes. $\mathbf{A}^{[l-1]\top} \delta^{[l]}$ is $(n_{l-1} \times m)(m \times n_l) = n_{l-1} \times n_l$, the shape of $\mathbf{W}^{[l]}$. $\delta^{[l]} \mathbf{W}^{[l]\top}$ is $m \times n_{l-1}$, the shape of $\mathbf{Z}^{[l-1]}$. If you forget the formulas, the shapes almost force them.

Link to the ML classes. With one layer ($\mathbf{A}^{[0]} = \mathbf{X}$): $\nabla_{\mathbf{W}} J = \frac{1}{m} \mathbf{X}^\top (\hat{\mathbf{Y}} - \mathbf{Y})$. Exactly the gradients of linear and logistic regression: you already knew how to start!

Forward and backward propagation: the algorithm

Forward →

Start: $\mathbf{A}^{[0]} = \mathbf{X}$

For each layer $l = 1, \dots, L$:

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]}, \quad \mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$$

Loss at the last layer: $J(\mathbf{A}^{[L]}, \mathbf{Y})$. Keep every $\mathbf{Z}^{[l]}, \mathbf{A}^{[l]}$ in memory.

← Backward

Start at the output (matching loss):

$$\delta^{[L]} = \frac{1}{m} (\mathbf{A}^{[L]} - \mathbf{Y})$$

For each layer $l = L, \dots, 1$:

$$\frac{\partial J}{\partial \mathbf{W}^{[l]}} = \mathbf{A}^{[l-1]\top} \delta^{[l]}, \quad \frac{\partial J}{\partial \mathbf{b}^{[l]}} = \sum_i \delta_i^{[l]}$$

$$\delta^{[l-1]} = (\delta^{[l]} \mathbf{W}^{[l]\top}) \odot g'^{[l-1]}(\mathbf{Z}^{[l-1]})$$

Update parameters with gradient descent (learning rate α):

$$\mathbf{W}^{[l]} := \mathbf{W}^{[l]} - \alpha \frac{\partial J}{\partial \mathbf{W}^{[l]}}, \quad \mathbf{b}^{[l]} := \mathbf{b}^{[l]} - \alpha \frac{\partial J}{\partial \mathbf{b}^{[l]}}$$

Three matrix products per layer in the backward pass. Any layer type (conv, attention...) just needs its own version of these three lines.

The same algorithm in NumPy

```
# X: (m, n0), Y: (m, 1); sigmoid everywhere, as in Lab 7
# ---- forward: keep every Z and A ----
Z1 = X @ W1 + b1; A1 = sigmoid(Z1) # (m, n1)
Z2 = A1 @ W2 + b2; A2 = sigmoid(Z2) # (m, 1)
loss = -np.mean(Y*np.log(A2) + (1-Y)*np.log(1-A2))

# ---- backward: from the loss to the first layer ----
delta2 = (A2 - Y) / m # (a) [2]
dW2 = A1.T @ delta2 # (b) (n1, 1)
db2 = delta2.sum(axis=0, keepdims=True)
delta1 = (delta2 @ W2.T) * A1 * (1 - A1) # (c) sigma' = a(1 - a)
dW1 = X.T @ delta1 # (b) (n0, n1)
db1 = delta1.sum(axis=0, keepdims=True)

# ---- update ----
W1 -= lr * dW1; b1 -= lr * db1
W2 -= lr * dW2; b2 -= lr * db2
```

Every line of the backward pass is one of the three facts (a), (b), (c). Nothing else is needed.

Read it with the shapes: $\mathbf{A1.T} @ \text{delta2}$ is $(n_1 \times m)(m \times 1)$, the shape of $\mathbf{W2}$. The $/ m$ in delta2 turns the sums over examples into averages.

The derivative of the sigmoid reuses the stored activation: $\mathbf{A1} * (1 - \mathbf{A1})$. With tanh it would be $1 - \mathbf{A1}^2$, with ReLU $(\mathbf{Z1} > 0)$.

Repeat these few lines on mini-batches and the network learns. That is Lab 7.

Gradient checking, and autograd

A backprop implementation can be subtly wrong and still "train". Check it against a **numerical derivative**, one parameter at a time:

$$\frac{\partial J}{\partial w} \approx \frac{J(w + \varepsilon) - J(w - \varepsilon)}{2\varepsilon}, \quad \varepsilon \approx 10^{-5}$$

The centred difference has error $O(\varepsilon^2)$, much better than the one-sided $O(\varepsilon)$.

Compare with the **relative error** $\frac{\|g_{bp} - g_{num}\|}{\|g_{bp}\| + \|g_{num}\|}$: below 10^{-7} is correct; above 10^{-3} is a bug.

Try it on the tiny network: $J(W_1^{[2]} + \varepsilon) - J(W_1^{[2]} - \varepsilon)$ over 2ε gives -0.280 , the number of the slide before.

Automatic differentiation in PyTorch does all of this for you:

```
logits = model(X)          # forward: builds the graph
loss = loss_fn(logits, y)
optimizer.zero_grad()      # .grad accumulates: reset
loss.backward()            # backward: fills p.grad
optimizer.step()           # update
```

- Every tensor with `requires_grad=True` records the operations applied to it (the graph).
- `backward()` runs exactly the algorithm of the previous slides, for any graph.
- `with torch.no_grad():` for evaluation: no graph, less memory.

Gradient checking is still the way to test your own layers, and it is what the NumPy lab does.



Universidad
de Navarra

15

Weight Initialization:

How would you initialize your weights?

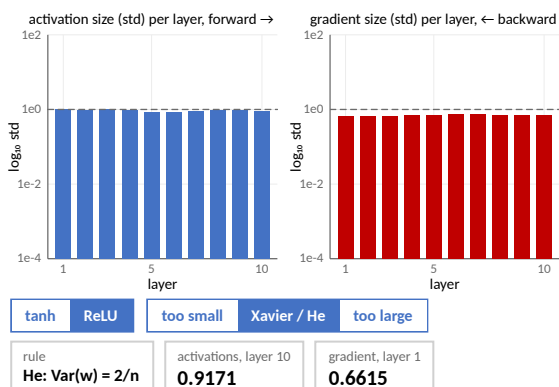
- **All zeros (or all equal)?** Every unit in a layer computes the same thing, gets the same δ , the same update, and stays identical forever. Random initialization **breaks the symmetry**.
- A deep linear chain: $\hat{y} = \mathbf{W}^{[L]} \mathbf{W}^{[L-1]} \dots \mathbf{W}^{[1]} \mathbf{x}$. With every $\mathbf{W} = 1.5 \mathbf{I}$: $\hat{y} = 1.5^L x$, **exploding**. With $0.5 \mathbf{I}$: $0.5^L x$, **vanishing**.

How large? For zero-mean independent weights:

$\text{Var}(z_k) = n_{in} \text{Var}(w) \mathbb{E}[a^2]$. Through L layers these factors **multiply**: we want each to be 1.

- **tanh**: $\text{Var}(w) = \frac{1}{n_{in}}$ (Xavier/Glorot: $\frac{2}{n_{in} + n_{out}}$).
- **ReLU** zeroes half the signal, $\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2} \mathbb{E}[z^2]$, so $\text{Var}(w) = \frac{2}{n_{in}}$ (He / Kaiming). Biases usually start at 0. PyTorch's `nn.Linear` default is smaller: uniform $\pm 1/\sqrt{n_{in}}$, $\text{Var}(w) = \frac{1}{3n_{in}}$.

[deeplearning.ai: initialization](https://deeplearning.ai/initialization)



10 layers of 100 units, 200 random inputs. The dashed line is "size 1".

► ReLU needs He ($2/n$), and then both stay of order 1 too (0.92 and 0.66)



Universidad
de Navarra

16

Vanishing and exploding gradients

Unroll the backward formula from the output down to layer l :

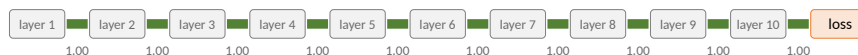
$$\delta^{[l]} = \left(\dots (\delta^{[L]} \mathbf{W}^{[L]\top} \odot g'(\mathbf{Z}^{[L-1]})) \dots \mathbf{W}^{[l+1]\top} \right) \odot g'(\mathbf{Z}^{[l]})$$

The gradient reaching an early layer is a **product** of $L - l$ factors "weight matrix $\times$ activation derivative". Products of many numbers shrink or blow up **exponentially**.

sigmoid, 10 layers: each factor ≤ 0.25



ReLU + He init: each factor ≈ 1



Line width = size of the gradient that reaches each layer, starting at the loss (right) and going back to layer 1 (left).

Sigmoid has $\sigma' \leq \frac{1}{4}$: after 10 layers the gradient can be $4^{-10} \approx 10^{-6}$. Early layers barely learn: **vanishing gradients**. Large weights: **exploding** (loss becomes NaN).

Remedies: ReLU activations · proper initialization · normalization layers (BatchNorm / LayerNorm, ch. 3) · **residual connections** $\mathbf{a}^{[l+1]} = \mathbf{a}^{[l]} + F(\mathbf{a}^{[l]})$, whose Jacobian contains the identity (ch. 4) · gradient clipping (ch. 5).

Vocabulary of chapter 2

derivative	sensitivity: how much the output moves when an input is nudged.	autograd	PyTorch's automatic backprop: <code>loss.backward()</code> fills every <code>.grad</code> .
computational graph	the network written as nodes of simple operations. PyTorch builds it during the forward pass.	gradient checking	comparing backprop with finite differences to catch bugs.
chain rule	gradient of an input = upstream gradient $\times$ local derivative; sum over paths.	Xavier / He init	random weights with variance $1/n_{\text{in}}$ (tanh) or $2/n_{\text{in}}$ (ReLU) so signals keep their size.
backpropagation	the chain rule run from the loss backwards, reusing stored forward values.	vanishing / exploding	gradients that shrink or grow exponentially with depth, because they are long products.
error signal $\delta^{[l]}$	$\partial J / \partial \mathbf{Z}^{[l]}$: how much the loss would change if the layer's pre-activations moved.		

Theory notes for this chapter: [notes 01 · Backpropagation](#) (chain rule on graphs, the derivation, gradient checking, initialization).

LAB 7

An MLP from scratch with NumPy

Build and train the network of chapters 1–2 with nothing but NumPy: every formula of these slides becomes a line of code.

- 1 XOR: see logistic regression fail, fix it with a hand-made feature.
- 2 Write the forward pass, the activations and the binary cross-entropy.
- 3 Derive and code backpropagation: $\delta^{[2]}$, $\partial \mathbf{W}^{[2]}$, $\delta^{[1]}$, $\partial \mathbf{W}^{[1]}$, with a shape check.
- 4 Verify it with **gradient checking**.
- 5 Train with mini-batch gradient descent; plot the decision boundary and what the hidden units learned.
- 6 Experiments: width, learning rate, random restarts, zero initialization.

[CO Open in Colab](#)

notebook Lab 7 - MLP from scratch.ipynb

you need chapter 1 (neuron, MLP, activations, BCE) and chapter 2 (backprop, gradient checking, initialization)

runs on CPU, no GPU needed

next Lab 8 (the same MLP in PyTorch) comes in chapter 3, after optimizers and mini-batches

Neural Networks

Máster en Innovación Tecnológica (MINT)

Next: Training neural networks

imanolgranada@gmail.com

Neural Networks

Máster en Innovación Tecnológica (MINT)

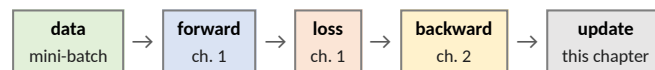
Training: mini-batches, optimizers, dropout, BatchNorm

imanolgranada@gmail.com



1

Where we are



We can compute every gradient. Three questions remain:

1. **How much data per step?** Whole dataset, one example, or a mini-batch.
2. **How to turn a gradient into a step?** Vanilla gradient descent is slow and fragile on non-convex losses: optimizers (momentum, RMSProp, Adam) and learning-rate schedules.
3. **How to avoid overfitting?** Weight decay, dropout, data augmentation, early stopping; and layers that make training stable (BatchNorm).

All of it is "knobs" on the same loop: forward, loss, backward, update. None of it changes the model's formula.

Two labs in this chapter: Lab 8 (the MLP again, now in PyTorch) right after the optimizers, and Lab 9 (classifying clothes images) at the end. The chapter closes with the layer zoo, a debugging recipe and a cheat-sheet of defaults.



2

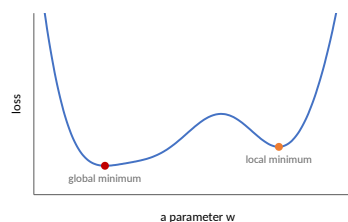
Gradient Descent for NN:

To optimize the weights of a neural network we apply gradient descent, **but**:

- Multi-layer NN are **not convex**.
 - Small NN → seed it at random points.
 - Large NN have more freedom and will likely escape from potential local-minima traps (empirical observation).
 - Optimizers → different from vanilla gradient descent.
- Some activation functions are not differentiable.
 - Only at some points (e.g. 0 for ReLU).
 - We still have sub-derivatives.

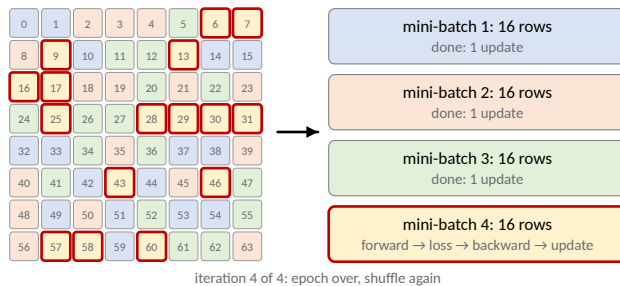
[Why does gradient descent work so well on non-convex nets? · ReLU and backprop](#)

In high dimensions most flat points are **saddle points**, not bad local minima: some directions go up, others go down. Noise in the gradient (mini-batches) and momentum help to leave them.



Vocabulary: epoch, mini-batch, iteration

shuffle, then cut into 4 mini-batches of 16



step 5/5

► iteration 4: the epoch is over; shuffle again and repeat

mini-batch a few examples (16–256) processed together, as one matrix.

iteration / step one gradient update, computed on one mini-batch.

epoch one full pass over the training set: $m/|B|$ iterations.

Example: 50,000 images, batch 64 → 782 iterations per epoch; 20 epochs → 15,640 updates.

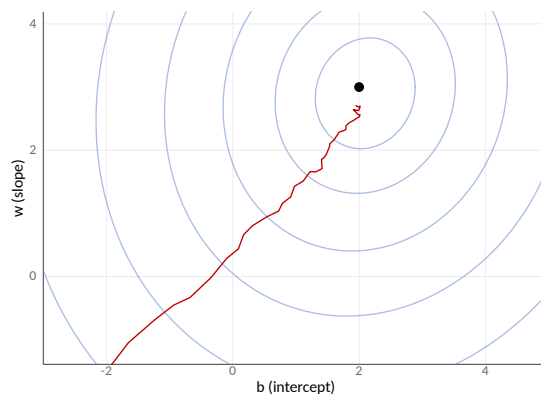
In PyTorch: `DataLoader(dataset, batch_size=64, shuffle=True)` does the shuffling and cutting; the inner `for xb, yb in loader` is one epoch.

Gradient Descent variations:

- **Batch gradient descent:** feed the entire dataset, average the cost, calculate derivatives and update. Used in LinReg and LogReg (convex problems).
- **Stochastic gradient descent:** feed 1 sample, calculate the cost and derivatives, and update. Many more updates, not always towards the minimum (which can be good with non-convexity).
- **Mini-batch gradient descent:** feed a mini-batch of samples, average the cost, calculate derivatives and update.
 - The same benefits as stochastic, but the calculation for the mini-batch is **vectorized**: speed.
 - Typically 16, 32, 64 or 128 (computers are prepared to parallelize this).

$$\mathbf{w} := \mathbf{w} - \alpha \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\mathbf{w}} \ell(\mathbf{x}^{(i)}, y^{(i)}; \mathbf{w})$$

Here $\mathbf{w}$ is the vector of all the weights and biases. The mini-batch gradient is an unbiased but noisy estimate of the full one: its variance shrinks like $1/|\mathcal{B}|$.



Batch Stochastic Mini-batch (16)

what happens

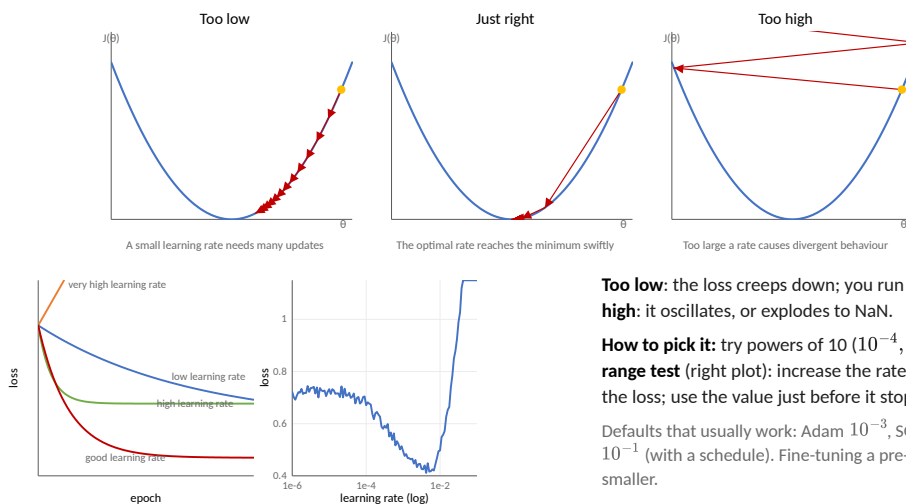
16 samples per step: the practical compromise

updates

39

► mini-batch of 16: 39 steps, smooth enough, cheap enough

The learning rate is the most important knob



Too low: the loss creeps down; you run out of patience. **Too high:** it oscillates, or explodes to NaN.

How to pick it: try powers of 10 (10^{-4} , 10^{-3} , 10^{-2}), or run a **range test** (right plot): increase the rate every step and watch the loss; use the value just before it stops falling steeply.

Defaults that usually work: Adam 10^{-3} , SGD with momentum 10^{-1} (with a schedule). Fine-tuning a pre-trained net: 10–100× smaller.

A tool for optimizers: the exponential moving average

Mini-batch gradients are **noisy**. Smooth them with a running average that forgets slowly:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t, \quad v_0 = 0$$

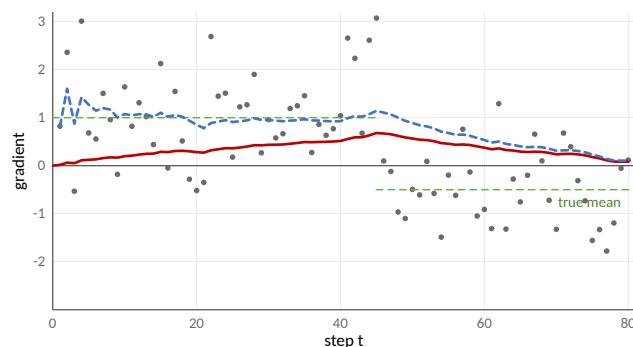
Unrolled: $v_t = (1 - \beta)(g_t + \beta g_{t-1} + \beta^2 g_{t-2} + \dots)$. Recent values weigh more; the weights β^k fade after about $\frac{1}{1-\beta}$ steps ($\beta = 0.9 \rightarrow 10$ steps, $0.99 \rightarrow 100$).

Bias at the start. Because $v_0 = 0$, the first values are too small:

$v_1 = (1 - \beta)g_1$ is only 10 % of g_1 for $\beta = 0.9$. Dividing by $1 - \beta^t$ fixes it:

$$\hat{v}_t = \frac{v_t}{1 - \beta^t} \quad (\hat{v}_1 = g_1)$$

Momentum averages g ; RMSProp averages g^2 ; Adam does both, with this correction.



β **0.98**

averaging window ≈ 50 steps	raw v_t 0.02	corrected $\hat{v}_t$ 0.82
--------------------------------	--------------------------	--------------------------------------

— gradients g_t — EMA v_t — bias-corrected $\hat{v}_t$

► $\beta = 0.98$: very smooth but slow, and far too small at the start unless corrected (blue)

Optimizers - Idea: momentum

In a narrow valley plain gradient descent zig-zags across and crawls along. Idea: **keep a velocity**.

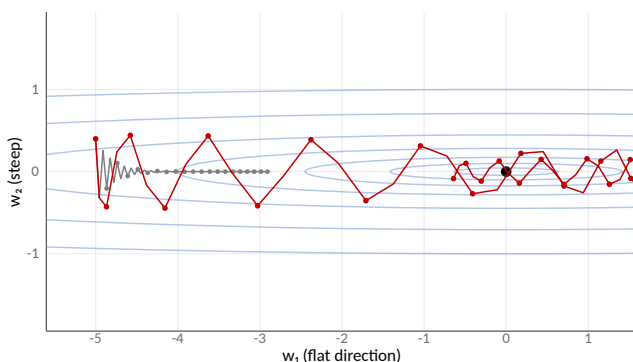
$$\mathbf{v} \leftarrow \beta \mathbf{v} + \nabla_{\mathbf{w}} J, \quad \mathbf{w} \leftarrow \mathbf{w} - \alpha \mathbf{v}$$

$\mathbf{v}$ is a running sum of past gradients,

$$\mathbf{v}_t = \mathbf{g}_t + \beta \mathbf{g}_{t-1} + \beta^2 \mathbf{g}_{t-2} + \dots$$

- **Same sign every step** (the valley floor): $\mathbf{v} \rightarrow \mathbf{g}/(1 - \beta)$, i.e. **up to 10× longer steps** for $\beta = 0.9$.
- **Sign flipping** (across the valley): consecutive gradients partly cancel, $\mathbf{v} \rightarrow \mathbf{g}/(1 + \beta)$, so the sideways speed stays small.
- **The price**: like a heavy ball, it overshoots and oscillates a little before settling. Too much momentum oscillates for a long time.

PyTorch: `SGD(params, lr= $\alpha$ , momentum=0.9)`. Some books write $(1 - \beta)\nabla J$ instead of ∇J (the EMA of the previous slide): the same method with α rescaled.



momentum β **0.95**

no momentum · loss after 60 steps 0.0422	momentum · loss after 60 steps 0.0092	max speed-up $1/(1-\beta)$ ×20.0
--	---	--

► $\beta = 0.95$: too much momentum: it overshoots and keeps oscillating

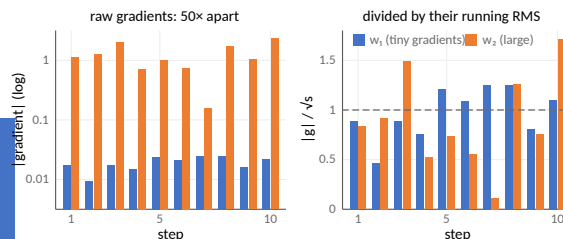
Optimizers - RMSProp: a step size per parameter

Gradients of different parameters can be **1000× apart** (a bias of the last layer vs a weight of the first). One learning rate cannot suit both.

RMSProp divides each parameter's step by the typical size of its own gradient:

$$\mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \mathbf{g} \odot \mathbf{g}, \quad \mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\mathbf{g}}{\sqrt{\mathbf{s}} + \epsilon}$$

- $\mathbf{s}$ is an **EMA of the squared gradient**, element by element; $\sqrt{\mathbf{s}}$ is the root-mean-square: the typical gradient size of each parameter.
- Tiny gradients (≈ 0.02) are divided by ≈ 0.02 , large ones (≈ 1) by ≈ 1 : **every parameter moves about α per step**.
- $\epsilon \approx 10^{-8}$ only avoids dividing by zero. Typical $\beta_2 = 0.9-0.999$.



Optimizers - Adam = momentum + RMSProp

Keep both averages, correct their start, and step:

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t && \text{(momentum: EMA of } \mathbf{g}) \\ \mathbf{s}_t &= \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 && \text{(RMSProp: EMA of } \mathbf{g}^2) \end{aligned}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \beta_2^t}, \quad \mathbf{w}_t = \mathbf{w}_{t-1} - \alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon}$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\alpha = 10^{-3}$. The most popular method: it usually works without tuning anything but α .

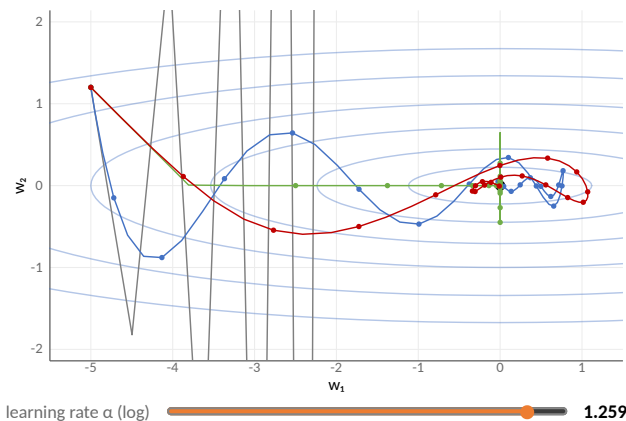
Why the bias correction? Both averages start at 0. At $t = 1$:

$\mathbf{m}_1 = 0.1 \mathbf{g}_1$ and $\mathbf{s}_1 = 0.001 \mathbf{g}_1^2$, far too small. After the correction $\hat{\mathbf{m}}_1 = \mathbf{g}_1$, $\hat{\mathbf{s}}_1 = \mathbf{g}_1^2$, so the **first step is $\alpha \cdot \text{sign}(\mathbf{g}_1)$** : every parameter moves by about α .

The factor matters for about 50 steps for $\mathbf{m}$ ($0.9^{50} \approx 0.005$) but for thousands of steps for $\mathbf{s}$ ($0.999^{5000} \approx 0.007$).

Method	Idea	to tune
Momentum	smooths the direction, speeds up flat valleys	α , β
RMSProp	per-parameter step size	α , β_2
Adam	both, bias-corrected	α (rest: defaults)

Optimizers - Examples:



► large ($\alpha = 1.26$): GD diverges out of the valley; momentum and the adaptive methods still converge

A narrow valley: the loss is 25× more curved along w_2 than along w_1 . 60 steps each, from the same start.

• gradient descent - final loss
1143.1348

• momentum
0.0026

• RMSProp
0.0086

• Adam
0.0000

GD zig-zags across the narrow direction and crawls along the long one; push α above 1 and it diverges. Adam and RMSProp rescale both directions to a similar step size.

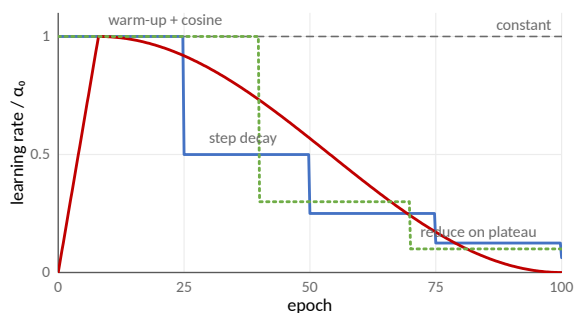
Which optimizer, which learning rate?

Optimizer	PyTorch	Typical α	Used for	Notes
SGD + momentum	<code>SGD(lr, momentum=0.9)</code>	0.01–0.1 + schedule	CNNs on images (ResNet...)	often the best final accuracy, needs tuning
Adam	<code>Adam(lr)</code>	10^{-3}	almost everything, first try	robust to the choice of α
AdamW	<code>AdamW(lr, weight_decay=0.01)</code>	10^{-4} – 10^{-3}	transformers, fine-tuning	Adam with correct weight decay
RMSProp	<code>RMSprop(lr)</code>	10^{-3}	RNNs, reinforcement learning	Adam without momentum

Rule of thumb. Start with **Adam**, $\alpha = 10^{-3}$, batch 32–64. If it trains, try halving / doubling α once. If a paper you are reproducing uses SGD + momentum, copy its schedule.

Bigger batches need bigger learning rates (the gradient is less noisy): a common recipe scales α linearly with the batch size. And **fine-tuning** a pre-trained network needs a much smaller α than training from scratch.

Learning-rate schedules and the training loop



Large steps early to move fast, small steps late to settle. **Warm-up** avoids huge first updates while Adam's statistics are still poor (standard for transformers). **Reduce on plateau** divides α when the validation loss stalls.

```
model = MLP()
opt = torch.optim.AdamW(model.parameters(),
                        lr=1e-3, weight_decay=1e-2)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(
    opt, T_max=epochs)
for epoch in range(epochs):
    model.train()  # training mode
    for xb, yb in train_loader:  # mini-batches
        loss = loss_fn(model(xb), yb)
        opt.zero_grad()
        loss.backward()  # backprop
        opt.step()  # update
    sched.step()  # once per epoch
    model.eval()  # evaluation mode
    with torch.no_grad():
        val_loss = evaluate(model, val_loader)
```

Anatomy of a model: shapes through the layers



B = batch size, passes through untouched. Total: 203,530 parameters, 98.7 % of them in the first Linear.

```
model = nn.Sequential(
    nn.Flatten(),          # [B,1,28,28] -> [B,784]
    nn.Linear(784, 256),   # 784*256 + 256 params
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(256, 10),    # logits, one per class
)
loss_fn = nn.CrossEntropyLoss() # softmax inside
```

Every layer is a function **tensor in** $\rightarrow$ **tensor out**. To design or debug a network, write the shape after each layer: the first number is always the batch size.

`model(xb).shape` and `sum(p.numel() for p in model.parameters())` are the two commands you will type most.

Layers with parameters: `Linear`, `Conv2d`, `Embedding`, `BatchNorm`, `LSTM`... Layers without: `ReLU`, `Dropout`, `Flatten`, `MaxPool2d`.

LAB 8

The MLP again, now with PyTorch

The same data and network as Lab 7, rebuilt with the tools you will use from now on. Autograd replaces the hand-written backward pass.

- 1 Tensors, and **autograd**: check that `loss.backward()` gives the gradients you derived by hand.
- 2 `nn.Linear` and `nn.Module`: an MLP with ReLU hidden layers.
- 3 The loss: why `BCEWithLogitsLoss` takes logits.
- 4 A training loop by hand, then with **optimizers**: SGD, momentum, Adam.
- 5 `Dataset` and `DataLoader`: mini-batches and shuffling, plus a validation set.
- 6 Extras: `nn.Sequential`, saving and loading a model, predicting, running on a GPU.

[CO Open in Colab](#)

notebook Lab 8 - MLP with PyTorch.ipynb

you need Lab 7, autograd (chapter 2), mini-batches and optimizers (this chapter so far)

runs on CPU

tip in Colab, *File* → *Save a copy in Drive* keeps your changes

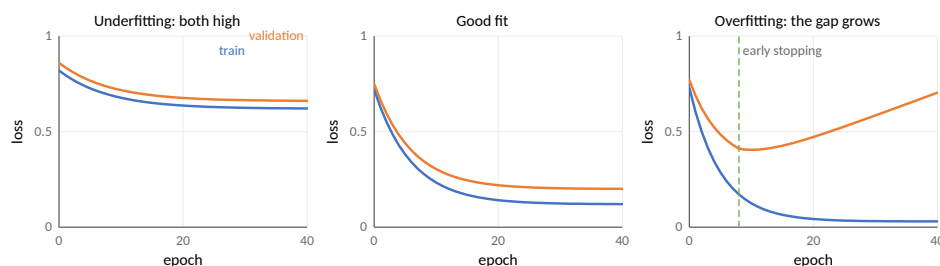


Universidad
de Navarra

15

Overfitting vs underfitting in NN:

In neural networks we should always verify that the model is generalizing well. It is quite easy to overfit your data: **plot the learning curves** (train and validation loss per epoch).



If your model under-fits your data:

- Make your model deeper (or wider).
- Try new architectures.
- Train longer, tune the learning rate.

If your model overfits your data:

- Use more data (also data augmentation).
- Simplify your architecture (depth or number of units).
- Apply L1 and/or L2 regularization to the weights; apply dropout; early stopping. Combine many of the above.



Universidad
de Navarra

16

Weight decay: L2 regularization for networks

Add a penalty on the size of the weights to the loss (biases are usually left out):

$$J_{\text{reg}} = J + \frac{\lambda}{2} \|\mathbf{w}\|^2 \Rightarrow \nabla J_{\text{reg}} = \nabla J + \lambda \mathbf{w}$$

Plug it into the gradient-descent step:

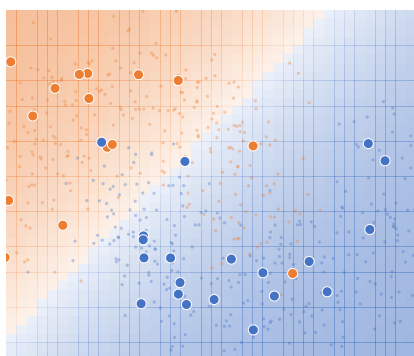
$$\mathbf{w} \leftarrow \mathbf{w} - \alpha(\nabla J + \lambda \mathbf{w}) = (1 - \alpha\lambda) \mathbf{w} - \alpha \nabla J$$

Every step first **shrinks** the weights by a factor $1 - \alpha\lambda$ (e.g. 0.9999): hence the name **weight decay**. Only weights that the data keeps pushing stay large.

- **Why it helps:** large weights make steep, wiggly functions that can pass through every noisy point; small weights give smooth ones. Same as Ridge in the ML part (a Gaussian prior on $\mathbf{w}$).
- **L1** ($\lambda \|\mathbf{w}\|_1$, gradient $\lambda \text{sign}(\mathbf{w})$) pushes small weights to exactly 0: sparse networks. Less common in deep learning.
- **AdamW.** In Adam, the $\lambda \mathbf{w}$ added to the gradient gets divided by $\sqrt{s}$, so large-gradient weights are barely decayed. AdamW applies the $(1 - \alpha\lambda)$ shrink **directly** to the weights, outside the adaptive step. The standard for transformers.

PyTorch: `SGD(params, lr, weight_decay=1e-4)` · `AdamW(params, lr, weight_decay=1e-2)` . Next slide: see it work.

Weight decay in action



weight decay λ  0.1

► $\lambda = 0.1$: too much: the border is too simple (underfitting)

A network with two hidden layers of 24 tanh units (about 700 weights) trained in the browser on only **40 noisy points** (big dots). The small faint dots are 600 **new** points from the same distribution.

train accuracy	accuracy on new data	size of the weights $\ \mathbf{w}\ $	diagnosis
0.925	0.810	2.1	underfitting

- No penalty: the network is big enough to **memorize** the 40 points, noise included.
- More λ : smaller weights, a smoother border, better on new data... up to a point.
- Too much λ : the model is too constrained to follow the real shape.

λ is a hyperparameter: choose it on the validation set, never on the test set.

More regularization: data augmentation and early stopping

Data augmentation. Random, **label-preserving** transformations of the training inputs each epoch (flips, crops, rotations, colour jitter; noise for audio). The model rarely sees the same input twice: like having more data, and it builds in the invariances. Pictures in chapter 4.

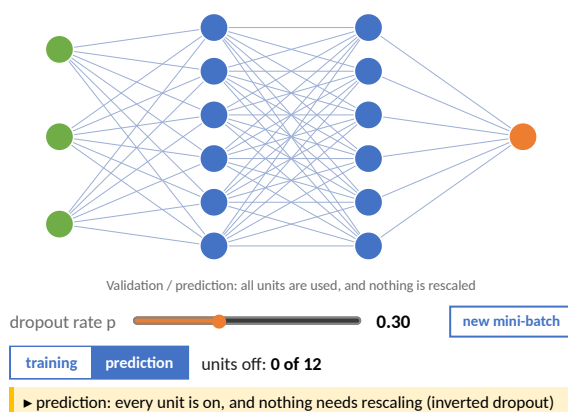
Only use transformations that don't change the label: a horizontal flip is fine for clothes, not for digits (a mirrored 7 is not a 7).

Early stopping. Monitor the validation loss, keep the best checkpoint, stop after `patience` epochs without improvement. Training time acts as a regularizer: the weights start small and grow as training goes on.

On the "Overfitting vs underfitting" slide, the green dashed line marks where early stopping would stop.

In practice: combine a few of them. Weight decay + augmentation + early stopping is a strong default; add dropout for large dense layers.

Dropout:



Inactivate some neurons in **training** (helps with regularization), controlled by a "rate" p . All neurons are used in validation / prediction!

Inverted dropout: survivors are scaled up during training, so the expected activation is unchanged:

$$\tilde{a}_j = \frac{r_j}{1-p} a_j, \quad r_j \sim \text{Bernoulli}(1-p)$$

$$\mathbb{E}[\tilde{a}_j] = \frac{1-p}{1-p} a_j = a_j$$

At prediction time the layer is simply switched off: every a_j passes unchanged (`model.eval()`).

- A unit cannot rely on any other unit being present: more robust, redundant features.
- Like training a huge **ensemble** of thinned networks that share weights (compare bagging).
- Typical p : 0.1–0.5, on dense layers. Validation loss below training loss? Normal with dropout: training runs a handicapped network.

Batchnorm layer:

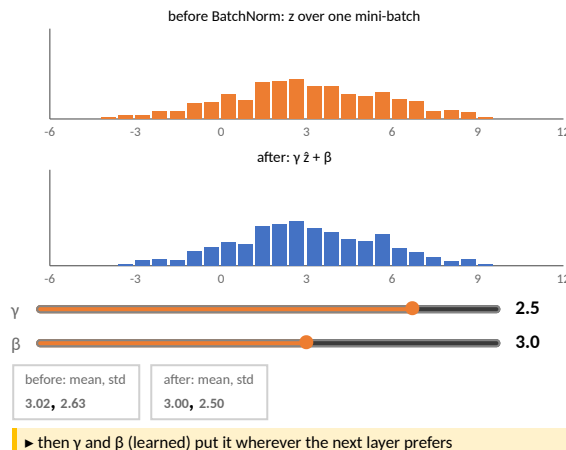
Helps correct training that is slow or unstable. In gradient-based algorithms it's a good idea to put all your data on a common scale; maybe also normalizing **inside** the network would be better!

A batch-normalization layer looks at each batch as it comes in, first normalizing it with its own mean and standard deviation, then putting it on a new scale with **two trainable parameters** per feature:

$$\mu_B = \frac{1}{m} \sum_i z_i, \quad \sigma_B^2 = \frac{1}{m} \sum_i (z_i - \mu_B)^2$$

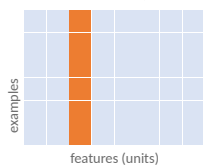
$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{z}_i + \beta$$

- Higher learning rates, faster training, mild regularization (batch noise).
- **Train vs eval:** batch statistics in training; running averages at test time (`model.eval()`). Forgetting `eval()` is a classic bug.
- Where? Usually `Linear/Conv → BatchNorm → ReLU`. In CNNs, one γ, β per **channel**.



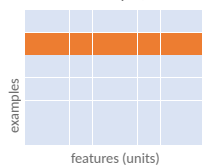
BatchNorm vs LayerNorm: which numbers are normalized?

BatchNorm: one feature, over the batch



orange = the numbers whose mean and variance are computed (then γ, β per feature)

LayerNorm: one example, over its features



BatchNorm: for each feature, mean and variance **over the mini-batch**. Needs a reasonably large batch and behaves differently in train / eval. Standard in CNNs.

LayerNorm: for each example, mean and variance **over its own features**. Independent of the batch and of the other examples, identical in train and eval. Standard in transformers (chapter 6), where sequences have different lengths and batches can be small.

Both end with the same learnable $\gamma \hat{z} + \beta$. For images, BatchNorm2d normalizes each channel over batch $\times$ height $\times$ width.

Other layers: the building blocks

Layer	PyTorch	What it does	params?
Fully connected	<code>nn.Linear</code>	$\mathbf{Z} = \mathbf{A}\mathbf{W} + \mathbf{b}$	yes
Activation	<code>nn.ReLU</code> , <code>nn.GELU</code> , <code>nn.Sigmoid</code>	element-wise non-linearity	no
Dropout	<code>nn.Dropout(p)</code>	switches units off in training	no
Batch / layer norm	<code>nn.BatchNorm1d/2d</code> , <code>nn.LayerNorm</code>	normalize, then scale by γ and shift by β	2 per feature
Convolution, pooling	<code>nn.Conv2d</code> , <code>nn.MaxPool2d</code>	local, shared filters for images (chapter 4)	conv yes, pool no
Flatten	<code>nn.Flatten</code>	tensor $\rightarrow$ vector, before the dense head	no
Embedding	<code>nn.Embedding</code>	token id $\rightarrow$ learned vector (chapters 5–6)	yes
Recurrent	<code>nn.LSTM</code> , <code>nn.GRU</code>	sequences with a hidden state (chapter 5)	yes
Attention	<code>nn.MultiheadAttention</code>	mixes information between positions (chapter 6)	yes
Residual connection	<code>x + f(x)</code> in forward	a direct path for the gradient	no

Plus **gradient clipping** (`clip_grad_norm_`) against exploding gradients. Any layer is fine as long as it is differentiable: autograd does the rest.

Debugging recipe: what the loss curve tells you

Symptom	Likely cause	What to try
Loss is NaN / explodes	learning rate too high, exploding gradients, <code>log(0)</code>	lower α 10 $\times$, clip gradients, use the "WithLogits" losses
Loss flat from the start	α too low, dead ReLUs, bug (wrong labels, no shuffling, forgot <code>zero_grad</code>)	overfit a single mini-batch first : it must reach ~ 0 loss
Train loss $\approx$ chance level ($\log K$)	inputs not normalized, labels misaligned with inputs	check <code>x.mean()</code> , <code>x.std()</code> , look at a few (x, y) pairs
Train good, validation bad	overfitting	augmentation, dropout, weight decay, smaller net, more data, early stopping
Both bad, and close	underfitting	bigger net, train longer, higher α , better features / architecture
Great in training, terrible at test time	forgot <code>model.eval()</code> (BatchNorm / dropout), different preprocessing	same transforms at train and test; call <code>eval()</code>
Validation loss below training loss	dropout / augmentation only act in training	normal; compare accuracy instead

Golden rule: make the pipeline work on a tiny problem (one batch, a small net) before scaling anything up.

Cheat-sheet: sensible defaults

Knob	Default
optimizer	Adam / AdamW, $\alpha = 10^{-3}$
batch size	32–64 (as large as memory allows for big models)
epochs	until validation loss stops improving (early stopping)
hidden activation	ReLU (GELU in transformers)
initialization	PyTorch default (He / Xavier-like)
schedule	cosine or step decay; warm-up for transformers

Knob	Default
input scaling	standardize features; images to [0, 1] then mean/std
weight decay	10^{-4} (SGD) · 10^{-2} (AdamW)
dropout	0.2–0.5 on dense layers
normalization	BatchNorm in CNNs, LayerNorm in transformers
gradient clipping	norm 1.0 for RNNs / transformers
what to log	train + validation loss and metric, every epoch

These get you 90 % of the way on most problems. Tune the learning rate first, then regularization, then architecture.

Vocabulary of chapter 3

mini-batch, epoch	a few examples per update; one full pass over the training set.
learning rate α	the step size; the most important hyperparameter.
moving average (EMA)	$v \leftarrow \beta v + (1 - \beta)g$: a smoothed signal over about $1/(1 - \beta)$ steps.
momentum	a velocity that accumulates gradients: faster along valleys, less zig-zag.
RMSProp, Adam	per-parameter step sizes from the average g^2 ; Adam adds momentum and bias correction.
schedule, warm-up	changing α during training: small at first, large, then decaying.

overfitting	good on training data, bad on new data; the validation curve goes up.
weight decay (L2)	penalty $\frac{\lambda}{2} \ \mathbf{w}\ ^2$: every step shrinks the weights by $1 - \alpha\lambda$.
dropout	randomly switch units off in training; all on at prediction time.
early stopping	keep the checkpoint with the best validation loss.
BatchNorm, LayerNorm	normalize over the batch / over the features, then scale and shift (γ, β).
train / eval mode	<code>model.train()</code> / <code>model.eval()</code> : switches dropout and BatchNorm behaviour.

Theory notes for this chapter: [notes 02 · Training: optimization and regularization](#) (SGD, momentum, Adam, schedules, weight decay, dropout, BatchNorm).

LAB 9

Multiclass images with an MLP (Fashion-MNIST)

A real dataset: 70,000 grayscale images of clothes, 10 classes.
Everything from chapters 1–3 in one pipeline.

- 1 Train / validation / test splits, and pre-processing: scale, standardize, flatten.
- 2 From sigmoid to **softmax**, and the categorical cross-entropy (with the proofs of chapter 1).
- 3 Softmax regression vs an MLP; a training loop with **early stopping**.
- 4 Evaluation: accuracy, confusion matrix, per-class metrics, the errors it makes.
- 5 Experiments with width, depth and **dropout**.
- 6 The limit of MLPs: shift the pictures a few pixels and watch the accuracy fall. That motivates chapter 4.

[CO Open in Colab](#)

notebook Lab 9 - Multiclass_classification.ipynb
you need softmax and cross-entropy (ch. 1), the training loop, Adam and regularization (this chapter), Lab 8
result MLP ≈ 88 % test accuracy, softmax regression ≈ 84 %
runs on CPU is fine; a GPU makes it faster (Runtime → Change runtime type)

Neural Networks

Máster en Innovación Tecnológica (MINT)

Next: Convolutional neural networks

imanolgranada@gmail.com

Neural Networks

Máster en Innovación Tecnológica (MINT)

CNNs, transfer learning, segmentation and detection

imanolgranada@gmail.com



1

Where we are

Part A · Convolutions

- images are grids of numbers
- from a dense layer to a convolution
- filters, channels, output size
- pooling, receptive field

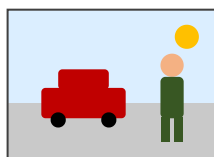
Part B · Architectures

- a basic CNN, layer by layer · **Lab 10**
- AlexNet, VGG, ResNet, 1×1 convs
- transfer learning · **Lab 12**
- multi-task learning · **demo**

Part C · Beyond classification

- semantic / instance segmentation
- U-Net, IoU and Dice
- object detection, YOLO
- NMS and mAP · **Lab 13**

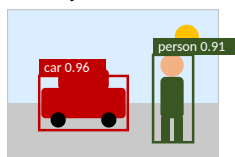
Classification



"car, person"

one label (or a few) for the whole image

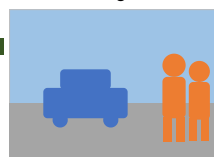
Object detection



a box + class + score per object

where is each thing?

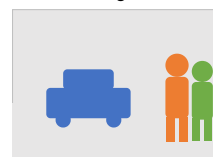
Semantic segmentation



a class for every pixel

sky · road · car · person

Instance segmentation



a mask per object

this person vs that person

same input, same convolutional backbone: what changes is the output layer and the loss



2

Part A Convolutions

- images are grids of numbers
- the convolution, step by step
- filters, channels, output size
- pooling and receptive field

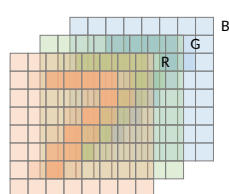
An image is a grid of numbers

grayscale: one number per pixel

0	0	0	0	0	0	0	0
0	51	230	255	255	230	51	0
0	0	0	0	77	255	102	0
0	0	0	77	255	128	0	0
0	0	51	255	153	0	0	0
0	0	204	230	0	0	0	0
0	77	255	77	0	0	0	0
0	0	0	0	0	0	0	0

8×8 matrix: 0 = black, 255 = white (or scaled to 0–1)

colour: three stacked grids (channels)



shape [3, H, W] in PyTorch (channels first)

a batch of images: [B, 3, H, W]

Neural networks are not restricted to vectors of features. They also accept **formatted data**:

- 1D: audio, temporal signals.
- 2D: black and white images.
- 3D: coloured pictures (channels × height × width).
- 4D: coloured video.

Mini-batches add an extra dimension 0: [batch, channels, H, W]. For a single image at prediction time, reshape it to a batch of 1: `x.unsqueeze(0)`.

Why not flatten the image into an MLP?

- **Too many parameters.** A 224×224 RGB image is 150,528 numbers. One dense layer of 1,000 units: **150 million** weights.
- **No notion of position.** A cat in the top-left and the same cat in the bottom-right share nothing: the MLP must learn the pattern at every position separately. Lab 9 showed it: shift the images a few pixels and the MLP's accuracy collapses.
- **No notion of neighbourhood.** Neighbouring pixels are related, far ones much less: after flattening, pixel (0,0) is as "close" to (0,1) as to (223,223).

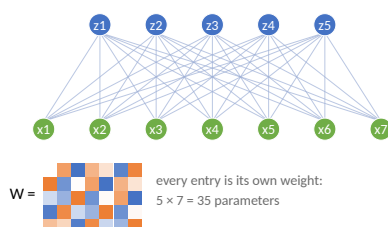
Two ideas fix all three problems:

1. **Local connections:** each unit looks at a small patch (3×3), not at the whole image.
2. **Weight sharing:** the same small set of weights is used at every position.

Together they give the **convolutional layer**. A 3×3 conv from 3 to 64 channels has **1,792** parameters, whatever the image size.

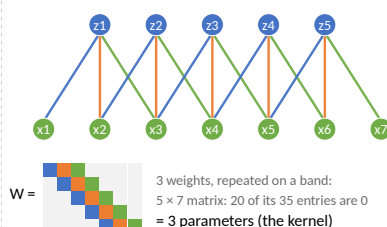
From a dense layer to a convolution

dense layer: each output sees every input



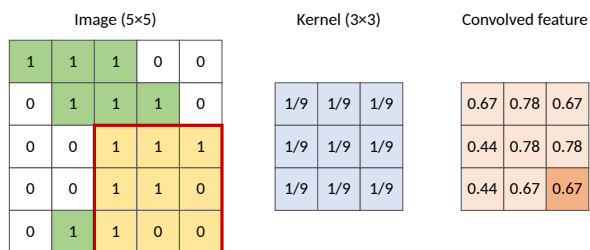
A 1-D picture of the two ideas. **Dense:** $z_o = \sum_{i=1}^7 W_{oi} x_i$, 35 independent weights. **Convolution:** each output only sees 3 neighbours (**local**), always with the same three weights k_1, k_2, k_3 (**shared**):
 $z_o = k_1 x_o + k_2 x_{o+1} + k_3 x_{o+2}$.

convolution: local + the same weights everywhere



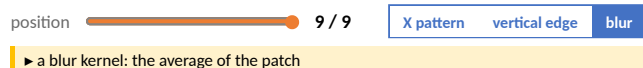
A convolution **is** a dense layer whose weight matrix is forced to be banded and repeated: most entries are 0 and the rest are copies of the kernel. Far fewer parameters, and the same pattern detector runs at every position. In 2-D the kernel is a small square (3×3) that slides over rows and columns.

Convolutional layers - CNN:



$$\text{output} = \sum \text{image} \times \text{kernel} = 1 \cdot 1/9 + 1 \cdot 1/9 + 1 \cdot 1/9 + 1 \cdot 1/9 + 1 \cdot 1/9 + 1 \cdot 1/9 + 1 \cdot 1/9 = 0.67$$

The 9 kernel weights are the parameters we learn. The same kernel slides over the whole image.

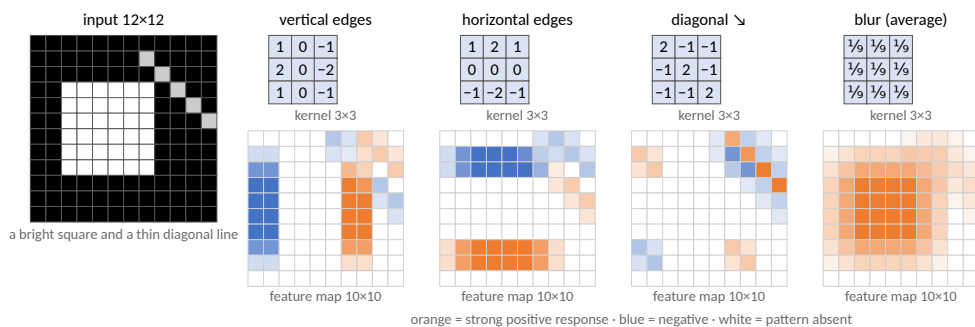


- The same filter (small, few parameters) is applied throughout the image.
 - Parameter 1 → **kernel size** ((3,3) in the example).
 - Parameter 2 → **stride**, the "jump" between calculations ((1,1) here).
 - Parameter 3 → **padding** (add zeros in the border).
- The stride and the padding determine the size of the output matrix.
- A filter "learns" certain features of the input: an edge, a corner, a texture.

$$Z_{ij} = \sum_{u=0}^{F-1} \sum_{v=0}^{F-1} X_{i+u, j+v} K_{uv} + b$$

Multiply the patch by the kernel element by element and add everything up. Strictly this is cross-correlation (no kernel flip); since the kernel is learned, nobody cares.

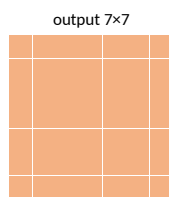
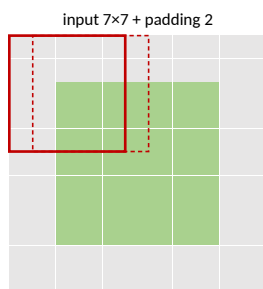
A filter is a pattern detector



The output of a convolution is a **feature map**: large where the image locally **looks like the kernel**, near zero elsewhere. A dot product is a similarity score.

Hand-made kernels like these (Sobel) were the classic computer-vision features. A CNN **learns** its kernels by backprop: the 9 numbers are parameters like any other weight.

Convolutional layers - output size



$$H_{\text{out}} = \left\lfloor \frac{H + 2P - F}{S} \right\rfloor + 1$$

output
7 × 7

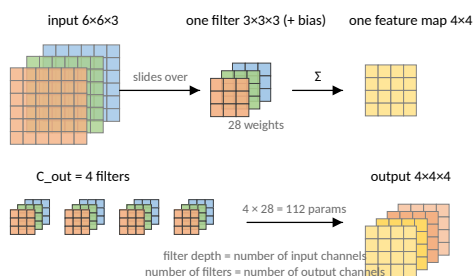
Why? With padding the input is $H + 2P$ wide. The first window starts at 0 and the last must start at most at $H + 2P - F$. Starting points every S steps:
 $\lfloor (H + 2P - F) / S \rfloor + 1$ of them.

- "same" padding: $P = (F - 1) / 2$ with $S = 1$ keeps $H_{\text{out}} = H$.
- Stride 2 halves the size: down-sampling.
- Example: $H = 224, F = 7, P = 3, S = 2 \rightarrow \lfloor 223 / 2 \rfloor + 1 = 112$ (ResNet's first layer).



► 5×5 kernel needs padding 2 to keep the size

Convolutional layers - many filters



$$\# \text{params} = (F \cdot F \cdot C_{\text{in}} + 1) \cdot C_{\text{out}}$$

3×3 conv, 64 → 128 channels: $(9 \cdot 64 + 1) \cdot 128 = 73,856$ parameters, whatever the image size.

- The input has C_{in} channels (3 for RGB). Each filter is $F \times F \times C_{\text{in}}$: the **third dimension matches** the previous layer's automatically.
- C_{out} filters → an output with C_{out} channels ("feature maps").
- PyTorch: `nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, stride=1, padding=1)`.

Weight sharing ⇒ translation equivariance. Shift the input by (a, b) , $X'_{ij} = X_{i-a, j-b}$:

$$Z'_{ij} = \sum_{u,v} X_{i-a+u, j-b+v} K_{uv} = Z_{i-a, j-b}$$

The output shifts by the same amount: **a pattern is detected wherever it appears**. Pooling then adds a little invariance to small shifts. ■

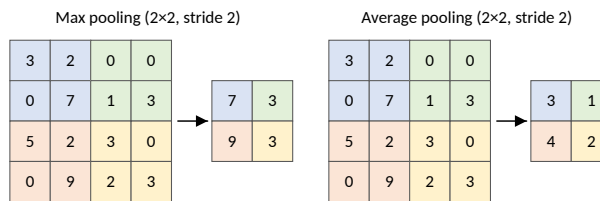
Pooling layers - CNN:

Down-sampling can be done with the stride: stride (2,2) divides the size by 2 at every conv layer.

A more robust and common approach is a **pooling layer** (after the non-linear activation).

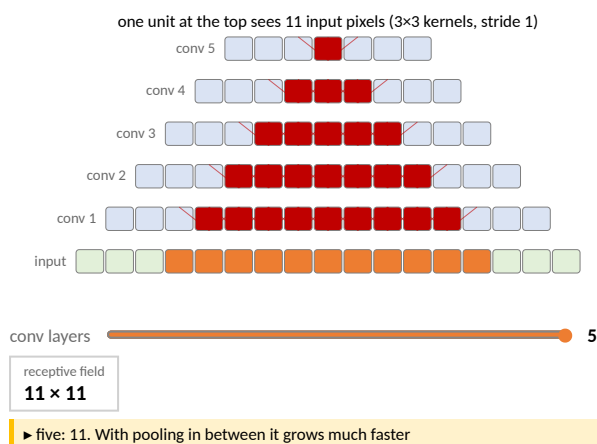
The pooled feature map is a summarized version of the features detected in the input. Max pooling asks "is the pattern present somewhere in this region?", which tends to be more informative than its average presence.

- No parameters.
- Halves height and width: 4× less computation for the next layer.
- Backward: the gradient goes only to the max (the "router" gate of chapter 2).



A summarized, down-sampled version of the features: 4× fewer numbers, no parameters.

Receptive field: how much of the image does a unit see?



Each 3×3 layer adds 2 pixels. A **stride** or a **pooling** multiplies how much every following layer adds:

$$r_l = r_{l-1} + (F_l - 1) \prod_{i < l} S_i$$

Example: conv3 → pool2 → conv3 → pool2 gives 3, 4, 8, 10 pixels. ResNet-50: several hundred: the last layer sees the whole image.

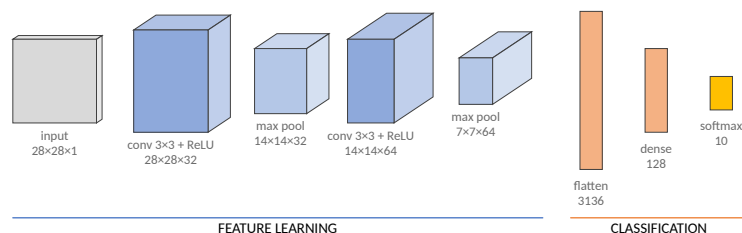
Stacking: layers close to the input learn low-level features (edges); deeper layers combine them into shapes and objects, because they see a larger patch.

Part B

Architectures and transfer learning

- a basic CNN, layer by layer · Lab 10
- AlexNet, VGG, ResNet
- transfer learning · Lab 12
- multi-task learning · demo

Basic CNN architecture:



```
model = nn.Sequential(
    nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), # 28×28×32
    nn.MaxPool2d(2), # 14×14×32
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), # 14×14×64
    nn.MaxPool2d(2), # 7×7×64
    nn.Flatten(), # 3136
    nn.Linear(3136, 128), nn.ReLU(),
    nn.Linear(128, 10)) # logits
```

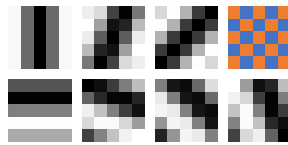
Pooling combined with more filters! The spatial size shrinks while the number of channels grows. Then flatten and classify with dense layers.

layer	params
conv 1→32	$(9 \cdot 1 + 1) \cdot 32 = 320$
conv 32→64	$(9 \cdot 32 + 1) \cdot 64 = 18,496$
dense 3136→128	$3136 \cdot 128 + 128 = 401,536$
dense 128→10	$128 \cdot 10 + 10 = 1,290$

95 % of the weights sit in the first dense layer: that is why modern nets use global average pooling instead (a few slides ahead).

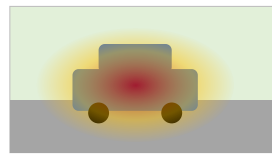
Looking inside a CNN

first-layer filters of a trained CNN (schematic)



oriented edges at several angles
and colour contrasts (Gabor-like)

class activation map: where did it look?

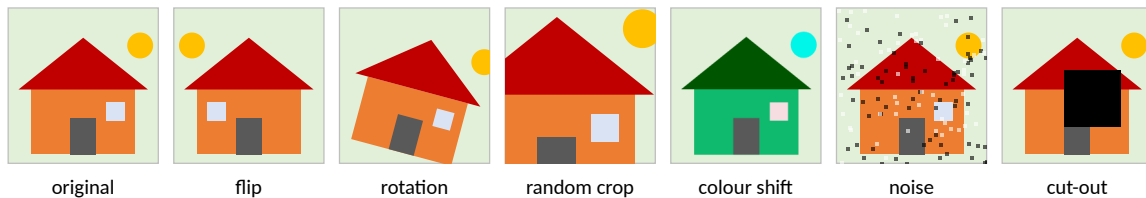


Grad-CAM for "car": the pixels that
pushed the car score up

The first layer of any CNN trained on photos learns **oriented edges and colour blobs**, much like the early visual cortex. That is why those layers transfer so well to new tasks.

Grad-CAM: take the gradient of a class score with respect to the last feature maps, weight the maps, and overlay them on the image. If a "wolf" classifier lights up on the snow, it learned a shortcut.

Data augmentation:



Each epoch, the model sees a slightly different version of every image, and the label stays the same.

- **Flip, rotation, crop:** the object can appear anywhere, at any small angle.
- **Colour shift, noise:** tolerance to lighting, camera quality.
- **Cut-out (information loss):** parts of the object can be hidden.

Only transformations that keep the label! Applied on the fly, a new random version each epoch, to the **training** set only. For detection and segmentation, the **boxes and masks must be transformed too**.

albumentations.ai · `torchvision.transforms.v2`

LAB 10

CNNs for image classification (Fashion-MNIST)

The same clothes images as Lab 9, now with a convolutional network, and a direct comparison with the MLP.

- 1 The convolution **from scratch**; padding, stride and the output size; channels and the parameter count.
- 2 Translation equivariance, pooling and the receptive field, checked in code.
- 3 Build and train a small CNN; look at its learned filters and feature maps.
- 4 **Shifted images**: the CNN keeps working where the MLP of Lab 9 fails.
- 5 Fight overfitting with data augmentation and dropout; a stronger CNN with more filters and BatchNorm.

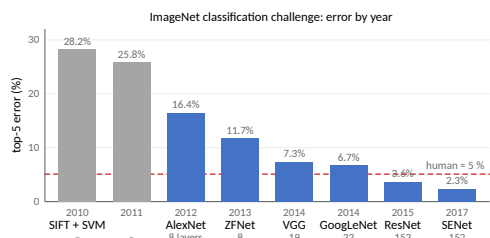
[CO Open in Colab](#)

notebook lab 10 - CNNs for image classification.ipynb

you need Part A of this chapter, the basic CNN, augmentation; dropout and BatchNorm (ch. 3); Lab 9

runs on CPU works; a GPU is faster (Runtime → Change runtime type → T4 GPU)

Famous architectures: AlexNet, VGG, ResNet



AlexNet (2012)

First big CNN on GPUs, with ReLU and dropout. Won ImageNet by a huge margin (16 % vs 26 % error). 60 M parameters.

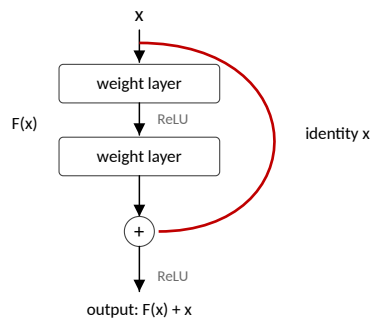
VGG (2014)

Only 3×3 convs and 2×2 max-pools. Two 3×3 layers see a 5×5 patch with 18 weights instead of 25, plus an extra non-linearity. VGG16, VGG19.

ResNet (2015)

A 56-layer plain CNN did worse than a 20-layer one, in **training** too: not overfitting, but optimization. Skip connections fixed it: 152 layers.

Residual connections



Instead of learning $H(\mathbf{x})$ directly, learn a **correction** $F(\mathbf{x})$ and add the input back:

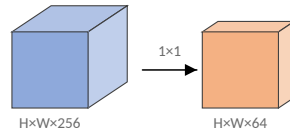
$$\mathbf{h} = F(\mathbf{x}) + \mathbf{x} \Rightarrow \frac{\partial \mathbf{h}}{\partial \mathbf{x}} = \frac{\partial F}{\partial \mathbf{x}} + \mathbf{I}$$

- The Jacobian always contains the identity: the gradient reaches early layers even if F 's gradient is tiny (the "add gate" copies it).
- If a block is not useful, it can learn $F \approx 0$ and become the identity: a deeper net is at least as good as a shallower one.
- The same trick is in every transformer block (chapter 6).

Inception modules instead run 1×1 , 3×3 and 5×5 convolutions in parallel and concatenate them, for features of different sizes.

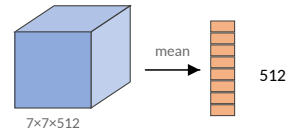
Two useful tricks: 1×1 convolutions and global pooling

1×1 convolution: mix channels, keep space



a dense layer applied at every pixel, across its 256 channel values
 $256 \cdot 64 + 64$ params

global average pooling: one number per map



replaces Flatten + big dense layers
 no parameters, any input size
 (ResNet, EfficientNet)

1×1 conv: a dense layer applied at every pixel, across channels. Used to shrink 256 channels to 64 before an expensive 3×3 conv (the "bottleneck" of ResNet-50), and as the classifier of segmentation heads.

Global average pooling: average each feature map to one number. A $7 \times 7 \times 512$ tensor becomes 512 numbers, then one `Linear(512, K)`. Few parameters, and the network accepts any image size.

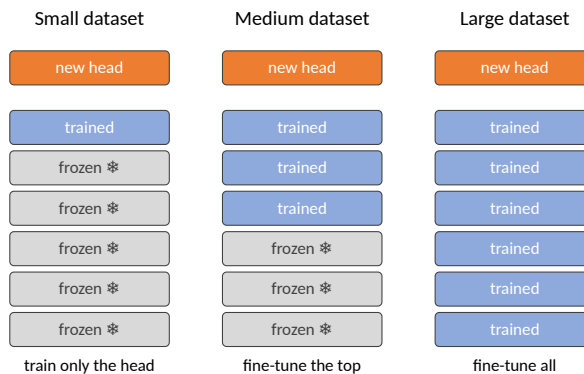
Transfer learning:

You can use the same neural net as Google, but you do not have the same amount of data!

The first conv layers learn to extract general features from images (edges, textures), so you can reuse that knowledge in new problems:

- Download a trained net (with the learned weights), e.g. a ResNet trained on ImageNet (1.2 M images, 1,000 classes).
- Fix (**freeze**) the first layers, where the complex feature engineering happens.
- Replace or randomly initialize the last fully connected layers: they learn the more specific, "high-level" features of your problem.
- Train these (fewer) weights with your (smaller) dataset, with a small learning rate. Feed the images normalized **as in pre-training** (ImageNet mean and std).

```
m = torchvision.models.resnet18(weights="DEFAULT")
for p in m.parameters(): p.requires_grad = False
m.fc = nn.Linear(m.fc.in_features, n_classes)
```



LAB 12

Fine-tuning a ResNet: pre-trained or from scratch?

The same ResNet-18 on 27,000 satellite images (EuroSAT, 10 land-use classes), started from random weights and from ImageNet weights. ImageNet has never seen a forest from above: does the knowledge transfer?

- 1 What an ImageNet network says about satellite images; its first-layer filters next to random ones.
- 2 **Zero training:** ImageNet features + logistic regression.
- 3 With 500 labelled images: **from scratch, frozen backbone + new head, fine-tuning** (last block, then everything, with discriminative learning rates).
- 4 Look inside: which weights moved, t-SNE of the features, confusion matrices.
- 5 **How many labels do you need?** Accuracy against training-set size.
- 6 Three ways to break a fine-tune: a learning rate too large, no ImageNet normalization, BatchNorm drifting.

Open in Colab

notebook lab 12 - Fine-tuning ResNet pretrained vs scratch.ipynb

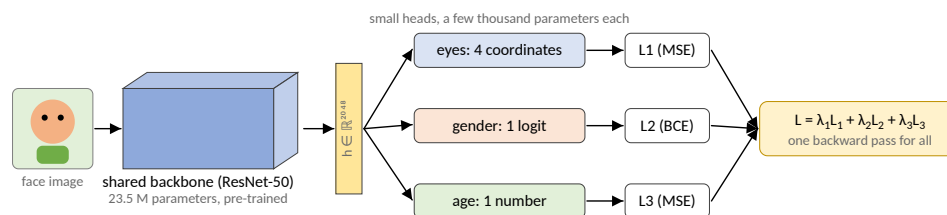
you need transfer learning and ResNet (this part), the training loop, schedules and BatchNorm (ch. 3)

result 500 labelled images: from scratch ≈ 81 %, fine-tuned ≈ 93 %

runs on GPU (T4 in Colab), 10 to 15 minutes

data EuroSAT, ≈ 90 MB, downloaded by torchvision

Multi-task learning: one backbone, several heads



one forward pass gives three predictions; the backbone must learn features that serve all three tasks

Train **one network for several related tasks**: a shared backbone $\mathbf{h} = f_w(\mathbf{x})$ and a small head per task, $\hat{y}_k = g_k(\mathbf{h})$. The loss is a weighted sum,

$$L = \sum_k \lambda_k L_k \Rightarrow \nabla_w L = \sum_k \lambda_k \nabla_w L_k$$

so the backbone receives gradient from every task. Benefits: one forward pass for three outputs, and each task acts as a regularizer for the others.

The catch: loss scales. At the start the eye MSE (coordinates in $[0, 1]$) is about $6 \cdot 10^{-5}$, the gender BCE about 0.69, the age MSE (in years²) about 73. With all $\lambda_k = 1$ the age term dominates and the eyes are not trained at all.

The fix: standardize each regression target, $z = (y - \mu)/\sigma$ with training-set statistics, so every loss starts near 1. For an MSE this is the same as choosing $\lambda = 1/\sigma^2$.

It is the same recipe as a detector (next part): one backbone, a box head and a class head, a weighted sum of losses.

Less boilerplate: PyTorch Lightning

```
class FaceNet(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.trunk = resnet50(weights="IMAGENET1K_V2")
        self.trunk.fc = nn.Identity()  # 2048 features
        self.eyes = nn.Linear(2048, 4)
        self.gender = nn.Linear(2048, 1)
        self.age = nn.Linear(2048, 1)

    def training_step(self, batch, idx):
        h = self.trunk(batch["image"])
        loss = (mse(self.eyes(h), batch["eyes_z"])
                + bce(self.gender(h).squeeze(1), batch["gender"])
                + mse(self.age(h).squeeze(1), batch["age_z"]))
        self.log("train/loss", loss)
        return loss  # no backward(), no step()

    def configure_optimizers(self):
        return torch.optim.AdamW(self.parameters(), lr=1e-4)

trainer = L.Trainer(max_epochs=20, gradient_clip_val=1.0,
                    callbacks=[EarlyStopping(monitor="val/loss")])
trainer.fit(model, datamodule=dm)
```

The training loop of chapter 3 is always the same: `zero_grad`, `backward`, `step`, move data to the GPU, switch `train()` / `eval()`, `log`, save the best checkpoint.

Lightning writes that loop for you. You only fill in the parts that change: the model, `training_step` (return the loss), `validation_step`, and `configure_optimizers`.

The `Trainer` adds GPU support, gradient clipping, early stopping and checkpointing with one argument each.

It does not make the model better; it removes a family of silent bugs (a forgotten `zero_grad`, metrics averaged wrongly, `eval()` never called).

DEMO

Multi-task face analysis

A worked solution to the multi-task problem on real faces: **one ResNet-50 backbone, several heads**. A demo, not a lab: there are no exercises, just two versions of the solution to read and run.

- 1 **Plain PyTorch**: transfer learning with a ResNet-50, eye positions as a regression problem, a hand-written training loop with gradient clipping and checkpointing, the predicted eyes drawn on test faces.
- 2 **Lightning**: the same faces with **three tasks** at once, eye positions (regression), gender (classification) and age (regression): a baseline first, standardized targets, a weighted loss, a `Trainer` with early stopping.

C0 Demo: plain PyTorch**C0** Demo: Lightning

notebooks `demo - multitask face analysis plain PyTorch.ipynb`, `demo - multitask face analysis Lightning.ipynb`

you need Lab 12, multi-task learning and Lightning (the two previous slides)

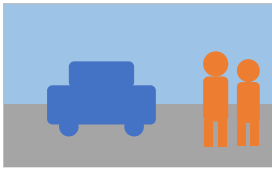
runs on GPU (T4 in Colab)

Part C Segmentation and object detection

- classify every pixel: U-Net
- IoU and Dice
- boxes: YOLO
- NMS and mAP · Lab 13

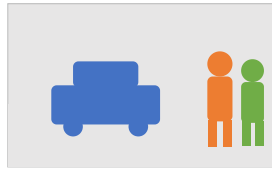
Segmentation: three flavours

Semantic: what class is this pixel?



both persons share one colour: "person"

Instance: which object is this pixel?



each person its own mask; background ignored

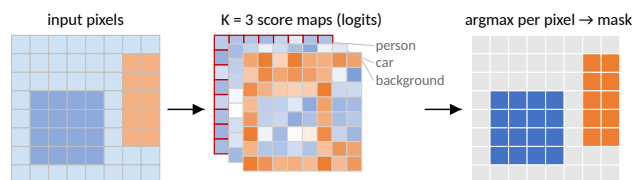
Panoptic: both at once



stuff (sky, road) by class, things (car, persons) by instance

Task	Output	Typical model	Example
Semantic	one class per pixel, $[K, H, W]$	U-Net, DeepLab, SegFormer	road / car / sky for self-driving; damage / no damage
Instance	one mask + class per object	Mask R-CNN, YOLO-seg	count and separate each scratch or each cell
Panoptic	both: "stuff" by class, "things" by instance	Mask2Former	full scene understanding

Semantic segmentation = classify every pixel



output tensor $[K, H, W]$: for each pixel a softmax over the K classes, trained with cross-entropy on every pixel
shown: the "person" map is outlined in red (orange = high score, blue = low)

show score map **person**
▶ the person score map

Classification: one softmax per **image**. Segmentation: one softmax per **pixel**. Same loss, just averaged over $H \times W$ positions:

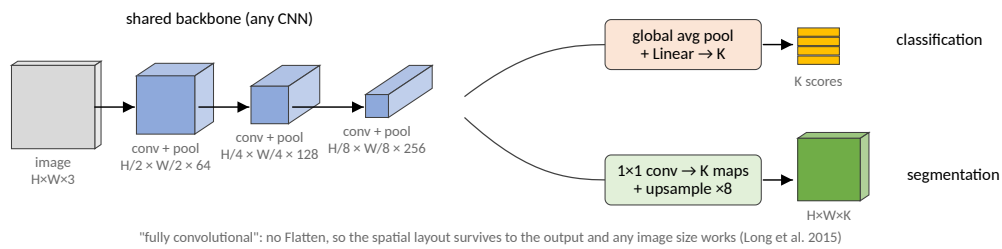
$$J = -\frac{1}{HW} \sum_{i,j} \log \hat{p}_{i,j}(y_{i,j})$$

The labels are masks: an integer image with the class of every pixel (painted by hand, expensive!).

PyTorch: `nn.CrossEntropyLoss()` accepts logits $[B, K, H, W]$ and targets $[B, H, W]$ directly.

Problem: the backbone shrinks the image 32x. We need a way to go back up to full resolution.

Fully convolutional networks

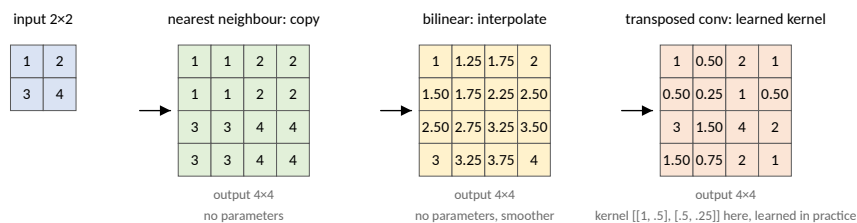


Classification head: pool everything into one vector, lose the "where".

Segmentation head: keep the spatial grid, predict K scores at each cell with a 1×1 conv, then **upsample** to the input size.

Upsampling $\times 32$ in one go gives blobby masks. Better: go up **gradually**, and reuse the high-resolution features of the early layers. That is the U-Net.

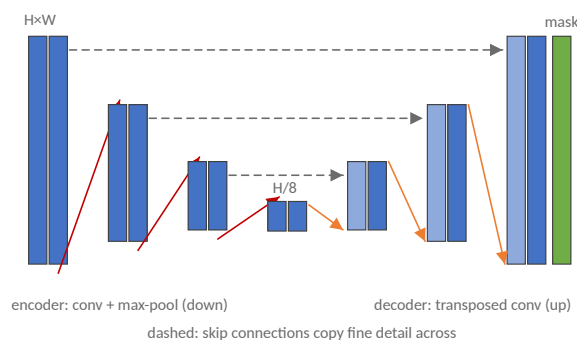
Going up: nearest, bilinear, transposed convolution



- **Nearest / bilinear:** fixed rules, no parameters. Usually followed by a normal conv to refine (`nn.Upsample`).
- **Transposed convolution:** each input value "paints" a copy of a **learned** kernel, scaled by the value, onto the output; overlaps add up (`nn.ConvTranspose2d`).

Same parameters as a conv: kernel size, stride, padding. With stride 2 it doubles H and W : exactly the mirror of a stride-2 conv, and its backward pass is a normal convolution.

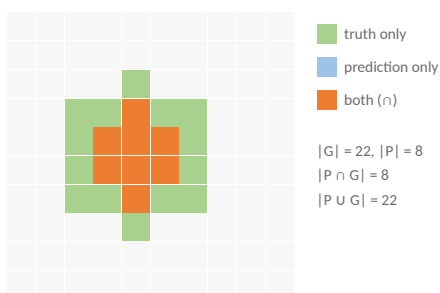
U-Net: encoder, decoder, skip connections



- **Encoder** (left, down): conv + pooling, like any CNN, often a pre-trained ResNet. Understands *what* is in the image.
- **Decoder** (right, up): upsampling + conv. Recovers *where*.
- **Skip connections** (dashed): copy the encoder's feature maps at each resolution and concatenate them to the decoder's. They bring back the fine detail (exact borders) that the bottleneck lost.

Output: K channels at full resolution, one per class. Born in medical imaging (2015), still the default for segmentation, and the core of image generators like Stable Diffusion.

Measuring masks: IoU and Dice



$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|}, \quad \text{Dice} = \frac{2|P \cap G|}{|P| + |G|}$$

P = predicted pixels, G = ground-truth pixels. Both are 1 for a perfect mask and 0 for no overlap.

Why not pixel accuracy? If damage covers 2 % of the image, predicting "no damage" everywhere is 98 % accurate and useless. IoU ignores the easy background.

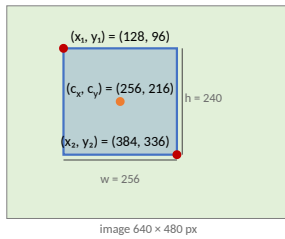
mIoU: the IoU of each class, averaged. The standard segmentation metric.

Dice loss = $1 - \text{Dice}$ computed on the soft probabilities, often added to the cross-entropy to fight class imbalance.



► too small, but inside: every predicted pixel is correct, still IoU is low

Object detection: what and where



three equivalent formats

xyxy: $(x_1, y_1, x_2, y_2) = (128, 96, 384, 336)$

xywh: $(x_1, y_1, w, h) = (128, 96, 256, 240)$

xcywh: $(c_x, c_y, w, h) = (256, 216, 256, 240)$

normalized (YOLO labels)

divide by image width and height:

$(0.40, 0.45, 0.40, 0.50)$

independent of resizing; one line per object:

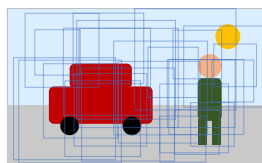
class cx cy w h → "2 0.40 0.45 0.40 0.50"

- **Classification:** what is in the image?
- **Localization:** what, and where? One box (x, y, w, h) regressed by the network, plus the class: a loss that combines regression and classification.
- **Detection:** all the objects, a box, a class and a confidence for each. Unknown number of objects!

Most bugs in detection code are box-format bugs: always check which convention a library and a dataset use.

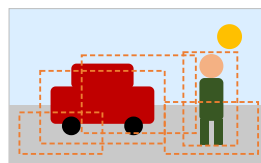
Three generations of detectors

Sliding window (2000s)



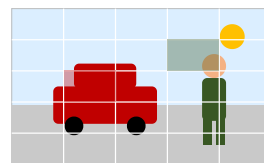
crops:
every position and size goes through a classifier, 10^4 – 10^5 per image
slow

Two-stage: R-CNN family (2014–)



proposals:
~1000 likely regions, then classify and refine each box
accurate, slower

One-stage: YOLO, SSD, RetinaNet (2016–)

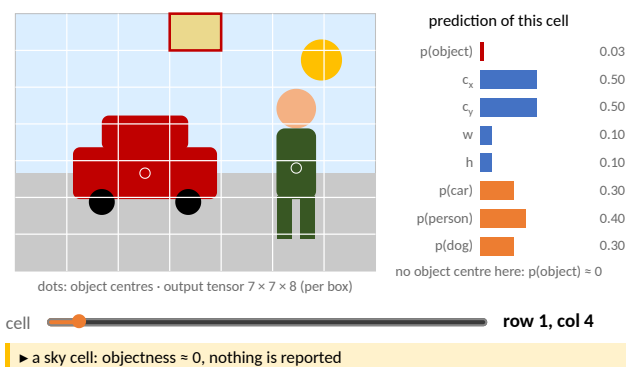


grid:
every cell predicts boxes and classes in one forward pass
real-time

all of them reuse a classification CNN as the backbone; they differ in how they turn features into boxes

Two-stage detectors are more accurate on small objects; one-stage ones are fast enough for video. Modern YOLO versions closed most of the accuracy gap, which is why they are the practical default. Transformers also detect: DETR predicts a **set** of boxes directly.

YOLO: every grid cell predicts



The backbone turns the image into an $S \times S$ grid of feature vectors. A 1×1 conv head predicts, **for every cell at once**:

- an **objectness** score: is there an object whose centre is here?
- the **box**: centre offset inside the cell, width, height;
- **class probabilities**.

One forward pass = all boxes. That is "You Only Look Once", and why it runs in real time.

This is the original YOLO idea. Modern YOLOs (v8, 11) use 3 grids of different sizes (small, medium, large objects), predict the box as distances from the cell centre to its 4 sides, and drop the objectness score: the class scores themselves say "nothing here".

Training a detector: the loss

A detector is a **multi-task** network: a box regression and a classification at every cell. For YOLO11 (the lab):

$$J = \underbrace{\lambda_{\text{box}} (1 - \text{CIoU})}_{\text{box overlap}} + \underbrace{\lambda_{\text{cls}} \text{BCE}}_{\text{class scores}} + \underbrace{\lambda_{\text{dfl}} \text{DFL}}_{\text{box sides}}$$

- **Box**: $1 - \text{IoU}$ (CIoU adds penalties for the distance between centres and the aspect ratio). Unlike an L1 on pixels, it does not depend on the object's size.
- **Class**: an independent sigmoid + BCE per class, at **every** cell; most cells must say "no class" (all scores low).
- **DFL** (distribution focal loss): each side distance is predicted as a small distribution over 16 bins, for more precise borders.

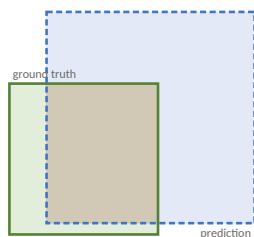
Ultralytics' defaults: $\lambda_{\text{box}} = 7.5$, $\lambda_{\text{cls}} = 0.5$, $\lambda_{\text{dfl}} = 1.5$. Older YOLOs (v1-v5) also had an objectness term.

Assignment decides which predictions are responsible for which true object (the "positives"): the cell containing the centre (YOLOv1), the anchor boxes with the best IoU (YOLOv3-5), or a learned score mixing IoU and class confidence (YOLOv8-11, the **task-aligned assigner**). The box and DFL terms only use the positives.

It is the same recipe as the multi-task face demo: one backbone, several heads, a weighted sum of losses.

Class imbalance is huge (thousands of background cells for a few objects): focal loss and careful weighting exist for that.

IoU for boxes: when is a prediction correct?



$$\text{IoU}(B_p, B_g) = \frac{\text{area}(B_p \cap B_g)}{\text{area}(B_p \cup B_g)}$$

A prediction **matches** a true object if the class is right and their IoU is at least a threshold (often 0.5). Matched: true positive. Unmatched prediction: false positive. Unmatched truth: false negative.

IoU 0.5 is stricter than it sounds: shifting a square by a third of its side already gets you there.

shift prediction  1.0

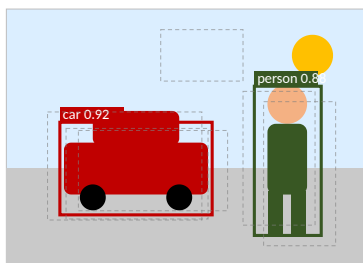
scale prediction  1.4

IoU
0.306

at threshold 0.5
not a match

► too large: also punished

Non-maximum suppression, step by step



done: one box per object

candidates, sorted by score

- ✓ car 0.92
- ✓ person 0.88
- ✗ car 0.81 IoU > 0.5 with a kept box
- ✗ car 0.75 IoU > 0.5 with a kept box
- ✗ person 0.62 IoU > 0.5 with a kept box
- ✗ car 0.40 IoU > 0.5 with a kept box
- ✗ person 0.30 IoU > 0.5 with a kept box
- car 0.15 below threshold

Neighbouring cells all see the same object, so the raw output has **many near-duplicate boxes**. Post-processing:

1. Drop boxes below a confidence threshold (e.g. 0.25).
2. Sort the rest by score.
3. Keep the best; remove every other box of the same class with IoU above ~0.5 with it.
4. Repeat with the next remaining box.

```
keep = torchvision.ops.nms(boxes, scores, 0.5)
```

Two people hugging can be wrongly merged: the IoU threshold is a trade-off.

step  4 / 4

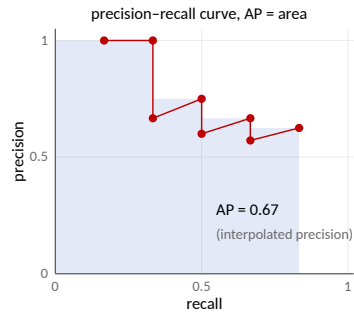
► nothing left to process: one box per object

Evaluating detectors: precision, recall, mAP

predictions, sorted by score

score	IoU ≥ 0.5?	prec.	recall
0.95	TP	1.00	0.17
0.90	TP	1.00	0.33
0.82	FP	0.67	0.33
0.75	TP	0.75	0.50
0.66	FP	0.60	0.50
0.60	TP	0.67	0.67
0.45	FP	0.57	0.67
0.30	TP	0.63	0.83

6 true objects in total
each row: one more box accepted



- Sort all predictions of a class by score. Walk down the list: each one is a TP or an FP.
- Running **precision** = TP / predictions so far; **recall** = TP / true objects.
- AP**: area under the precision-recall curve (made monotone first: the shaded staircase).
- mAP**: the mean of AP over classes. **mAP@0.5** uses IoU ≥ 0.5; **mAP@0.5:0.95** (COCO) averages over 10 thresholds, rewarding precise boxes.

Same precision and recall as in the logistic-regression classes: AP is the detection version of the area under a curve.

LAB 13

Object detection with YOLO

Detect buffaloes, elephants, rhinos and zebras: first understand every piece by coding it, then fine-tune a real detector.

- Box formats and **IoU** from scratch.
- How the YOLO11 head works: the grids, the box as 4 distances, the loss and the assigner.
- NMS** and **mAP** from scratch, checked against the library.
- Baseline with the COCO-pretrained model, then **fine-tune YOLO11n** on the African Wildlife dataset.
- Evaluate on the test set, look at the errors, measure the speed, export to ONNX and build a small Gradio app.

[CO Open in Colab](#)

notebook lab 13 - Object detection with YOLO.ipynb
you need transfer learning (Part B, Lab 12), and detection, IoU, NMS, mAP (this part)
runs on a GPU is strongly recommended (T4 in Colab)
data ≈100 MB of images and YOLO11n weights, downloaded by Ultralytics

Vision tasks at a glance

Task	Output per image	Loss	Metric	Model / library
Classification	K probabilities	cross-entropy	accuracy, F1	ResNet, EfficientNet, ViT · torchvision, timm
Semantic segmentation	$K \times H \times W$	pixel CE + Dice	mIoU	U-Net, DeepLab · segmentation_models_pytorch
Object detection	list of (box, class, score)	box (IoU) + class (+ objectness in older YOLOs)	mAP@0.5(:0.95)	YOLO, Faster R-CNN · ultralytics
Instance segmentation	list of (mask, class, score)	detection + mask BCE	mask mAP	Mask R-CNN, YOLO-seg

Vehicle damage from photos uses all four: classify the part, detect each damage with a box, segment its exact extent, and a shared pre-trained backbone underneath.

Vocabulary of chapter 4

kernel / filter	a small $F \times F \times C_{in}$ block of weights slid over the input.	transfer learning	reuse a pre-trained backbone; train a new head.
feature map	one output channel: where the filter's pattern appears.	multi-task learning	one backbone, one head per task, $L = \sum_k \lambda_k L_k$.
stride, padding	step between positions; zeros added at the border.	semantic / instance	class per pixel / mask per object.
pooling	max or mean over windows: down-sampling without parameters.	U-Net	encoder + decoder + skip connections, for masks.
receptive field	the input region that can affect one unit.	IoU, Dice	overlap of prediction and truth, from 0 to 1.
backbone	the pre-trained feature extractor shared by all vision tasks.	NMS	removes duplicate boxes of the same object.
residual connection	$\mathbf{h} = F(\mathbf{x}) + \mathbf{x}$: lets the gradient skip layers.	mAP	area under precision-recall, averaged over classes.

Theory notes for this chapter: [notes 03 · CNNs, transfer and multi-task learning](#).

Neural Networks

Máster en Innovación Tecnológica (MINT)

Next: RNNs and LSTMs

imanolgranada@gmail.com



Neural Networks

Máster en Innovación Tecnológica (MINT)

Recurrent neural networks and LSTMs

imanolgranada@gmail.com



1

Recurrent Neural Networks: why?

If we want to build a NN for **sequential data**, a CNN may not be the best fit: speech, text, the S&P 500, sensor logs...

- Sequences have **different lengths**: an MLP needs a fixed-size input.
- The **order** matters: "not good, bad" $\neq$ "not bad, good".
- What happened earlier affects what comes next: we need **memory**.
- The same pattern can appear at any time step: we want to **share weights** across time, as a CNN shares them across space.

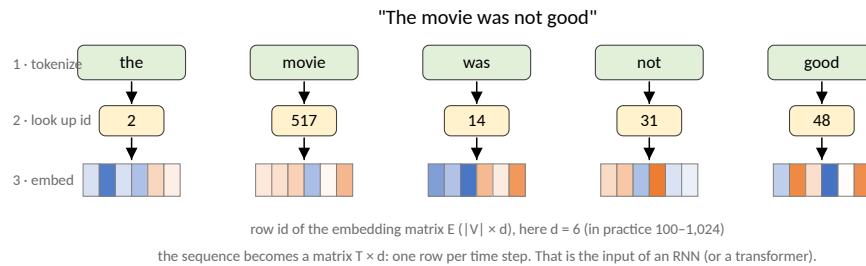
data	one step x_t	task
text	a word (embedding)	sentiment, translation, next word
speech	a 10 ms frame of audio features	transcription
finance	price, volume of one day	forecast tomorrow
machines	sensor readings each second	detect a failure

Recurrent neural networks: networks that read one step at a time and carry a **hidden state**, a running summary of everything seen so far.



2

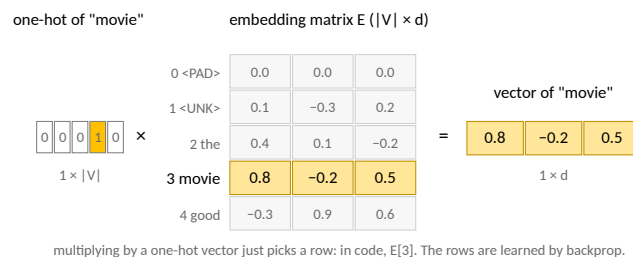
From text to vectors



Tokenize: words, sub-words or characters. Special tokens: `<PAD>` to fill short sequences, `<UNK>` for unknown words, `<START>`. **Index:** one integer per token in a vocabulary of size $|V|$.

Embed: each id becomes a vector of d learned numbers (next slide). A sentence of T tokens becomes a $T \times d$ matrix, one row per time step: the input of an RNN.

An embedding is a lookup table



Each token id k owns one row $\mathbf{e}_k$ of a matrix $\mathbf{E} \in \mathbb{R}^{|V| \times d}$. Written as a one-hot row vector $\mathbf{o}_k$ (a single 1 in position k), the lookup is a matrix product, $\mathbf{e}_k = \mathbf{o}_k \mathbf{E}$: it picks row k . In code nobody multiplies: `E[ids]`.

$\mathbf{E}$ starts random and is **learned by backprop** like any other weight; only the rows of the tokens in the batch get a gradient. `nn.Embedding(|V|, d)`: input $[B, T]$ integers, output $[B, T, d]$ floats. Typical sizes: $|V| = 10,000$ –50,000, $d = 50$ –1,024.

Word embeddings - word2vec

- **One-hot** $\mathbf{o}_w \in \{0, 1\}^{|V|}$: very sparse, high-dimensional, and every pair of words is equally far apart. No similarity information.
- **Embedding**: a dense vector $\mathbf{e}_w \in \mathbb{R}^d$, $d \ll |V|$ (e.g. 50–1,024). Similar words end up close.

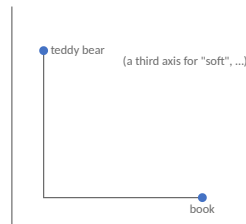
$$\mathbf{e}_w = \mathbf{o}_w \mathbf{E} \quad (\text{row } w \text{ of } \mathbf{E})$$

$\mathbf{E}$ is learned like any other weight: by the task itself, or beforehand with **target/context likelihood models**:

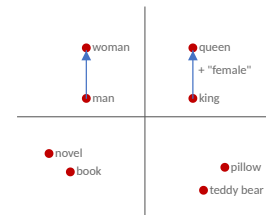
- **word2vec**: predict the words around a word (skip-gram), or the word from its neighbours (CBOW).
- Words used in similar contexts get similar vectors: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$.

The same idea gives embeddings for sentences and documents: the basis of semantic search and RAG (chapter 6).

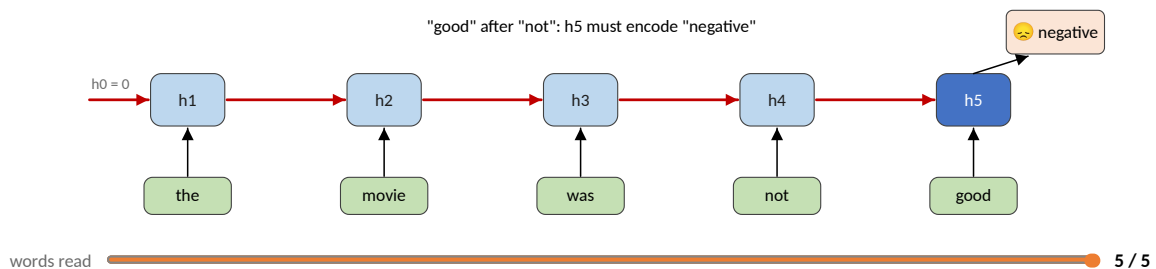
one-hot: every word equally far



embedding: similar words are close



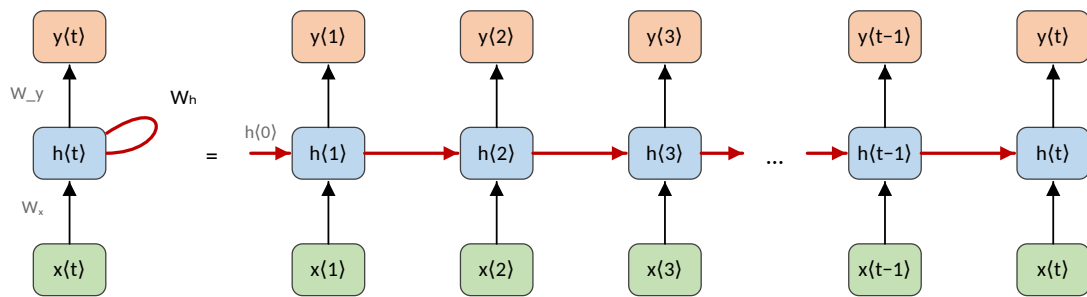
The idea: read one word at a time, keep a summary



Like reading a sentence and keeping a mental note: after each word you **update** the note using the word and the previous note. The note is the **hidden state** $\mathbf{h}_t$, a vector of, say, 128 numbers.

The update rule (the "cell") is the **same** at every step: one set of weights, any sentence length. At the end, $\mathbf{h}_T$ summarizes the whole review; a dense layer turns it into "positive / negative".

Typical Recurrent Neural Networks:



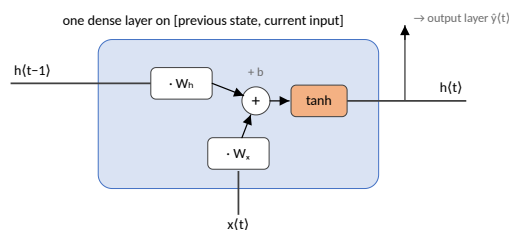
unrolled in time: the same weights W_x , W_h , W_y at every step

$$\mathbf{h}^{(t)} = g_1(\mathbf{x}^{(t)} \mathbf{W}_x + \mathbf{h}^{(t-1)} \mathbf{W}_h + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}^{(t)} = g_2(\mathbf{h}^{(t)} \mathbf{W}_y + \mathbf{b}_y)$$

$\mathbf{x}^{(t)}$: feature vector at time t (e.g. a new word of a sentence). $\hat{\mathbf{y}}^{(t)}$: output at time t (e.g. is this word an entity?). g_1 is usually $\tanh$. The hidden state $\mathbf{h}$ carries the memory. Unrolled, an RNN is just a very deep network whose layers share their weights.

Inside the cell



The cell is a **dense layer with two inputs**. Equivalently, concatenate them:

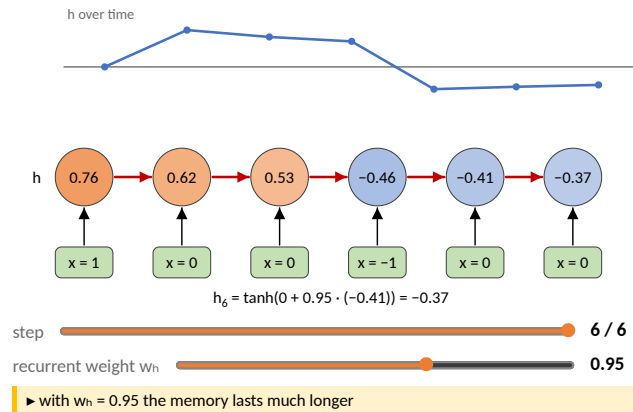
$$\mathbf{h}_t = \tanh([\mathbf{h}_{t-1}, \mathbf{x}_t] \mathbf{W} + \mathbf{b}), \quad \mathbf{W} = \begin{pmatrix} \mathbf{W}_h \\ \mathbf{W}_x \end{pmatrix}$$

object	shape
$\mathbf{x}_t$ (one step, batch B)	$B \times d$
$\mathbf{h}_t$	$B \times d_h$
$\mathbf{W}_x$	$d \times d_h$
$\mathbf{W}_h$	$d_h \times d_h$

Parameters: $d d_h + d_h^2 + d_h$, whatever the length. With $d = 100$, $d_h = 128$: 29,312 (PyTorch keeps two bias vectors: 29,440).

```
rnn = nn.RNN(input_size=100, hidden_size=128, batch_first=True)
out, h_T = rnn(x)          # x: [B, T, 100] → out: [B, T, 128]
```

An RNN with numbers



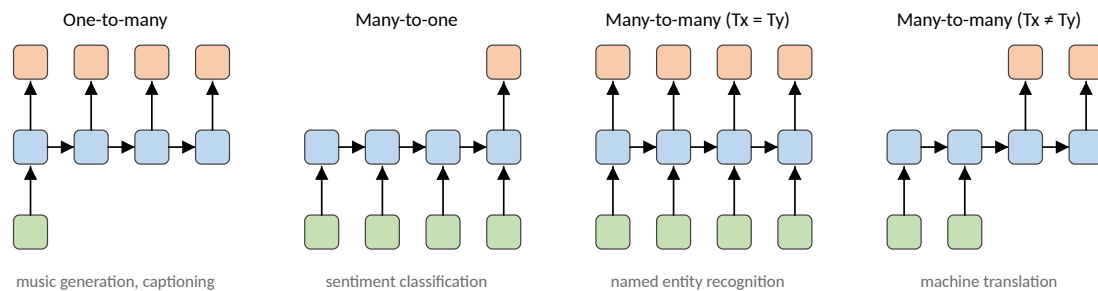
A one-number RNN: $h_t = \tanh(1 \cdot x_t + w_h h_{t-1})$, $h_0 = 0$, input sequence 1, 0, 0, -1, 0, 0.

With $w_h = 0.5$, each step without input **multiplies the memory by about 0.5**: after 5 quiet steps, $0.5^5 \approx 3\%$ of it is left.

Try $w_h > 1$: the state does not forget, but it is pulled back to a fixed point (0.86 for $w_h = 1.5$): the input -1 lowers it (0.84 → 0.25) but cannot flip its sign. Only a narrow range in between works: the core problem of plain RNNs.

The same multiplication by w_h happens to the **gradient** going backward in time.

Recurrent Neural Networks - Applications:



Type	Example	How
One-to-many	music or text generation, image captioning	feed each output back as the next input
Many-to-one	sentiment of a review	classify from the last hidden state
Many-to-many, same length	named entity recognition, part-of-speech	one output per input token
Many-to-many, different length	machine translation, speech-to-text	encoder reads everything, decoder writes

Training: loss and backpropagation through time

Loss function: summed over the time steps (or only at the last one, for many-to-one):

$$L(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{t=1}^{T_y} L(\hat{\mathbf{y}}^{(t)}, \mathbf{y}^{(t)})$$

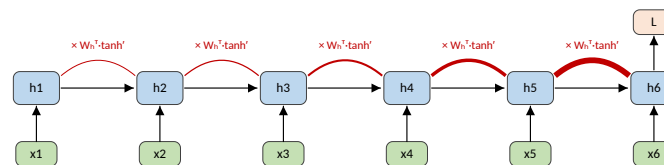
How do we learn? Unroll the network and run ordinary backprop on the unrolled graph: **backpropagation through time** (BPTT).

$$\frac{\partial L}{\partial \mathbf{W}_h} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{W}_h} \Big|_{\text{step } t}$$

$\mathbf{W}_h$ is used at every step, so its gradient is the **sum** of the per-step contributions (shared weights → gradients add up, like the "add gate").

Advantages	Drawbacks
Can process inputs of any length	Computation is slow (sequential)
Model size doesn't grow with the input length	Difficult to access information from a long time ago
Computation takes historical information into account	Cannot consider future inputs for the current state
Weights are shared across time	

Recurrent Neural Networks: vanishing gradients



the gradient from the loss is multiplied by the same factor at every step back: after k steps, $-(\|W_h\| \cdot \tanh^2)^k$

With one number per step, $h_t = \tanh(w h_{t-1} + u x_t)$. The chain rule for one step back:

$$\frac{\partial h_t}{\partial h_{t-1}} = \underbrace{(1 - h_t^2)}_{\tanh'} \cdot w$$

Through $T - k$ steps the factors multiply:

$$\frac{\partial h_T}{\partial h_k} = \prod_{t=k+1}^T (1 - h_t^2) w = w^{T-k} \prod_{t=k+1}^T (1 - h_t^2)$$

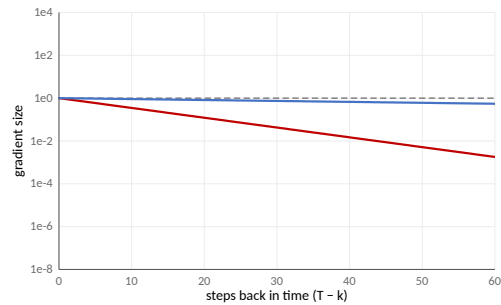
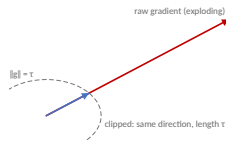
- $0 < 1 - h_t^2 \leq 1$, so every step back multiplies the gradient by at most $|w|$.
- $|w| < 1$: the signal from $T - k$ steps back shrinks **exponentially** (vanishing). The network cannot learn long-range dependencies: the gradient never tells it they matter.
- $|w| > 1$: it can blow up (exploding) → gradient clipping.
- With vectors, w becomes $\mathbf{W}_h^\top$ and $1 - h_t^2$ a diagonal matrix: $\|\partial J / \partial \mathbf{h}_k\| \leq \|\partial J / \partial \mathbf{h}_T\| \|\mathbf{W}_h\|^{T-k}$.

How fast does the signal fade?

Plain RNN: each step back multiplies the gradient by roughly $\|\mathbf{W}_h\| \cdot \tanh'$. LSTM (next slides): along its memory path, only by the forget gate f .

- RNN factor 0.8, 50 steps: $0.8^{50} \approx 10^{-5}$.
- LSTM with $f = 0.98$: $0.98^{50} \approx 0.36$.

Exploding gradients have a cheap fix: **clip** the gradient norm (below). Vanishing ones need a new architecture.



RNN: $\|\mathbf{W}_h\| \cdot \tanh'$ 0.90

LSTM: forget gate f 0.99

RNN, 50 steps back
5.2e-3

LSTM, 50 steps back
6.1e-1

— RNN — LSTM memory path

► LSTM with forget gate 0.99 keeps 60 % after 50 steps; the RNN with 0.9 keeps 0.5 %

GRU and LSTMs: gates

To remedy this, several versions of **gates** have been designed, each with its own purpose, but all with a similar structure:

$$\Gamma = \sigma(\mathbf{x}^{(t)} \mathbf{W} + \mathbf{h}^{(t-1)} \mathbf{U} + \mathbf{b}) \in (0, 1)$$

A gate is a sigmoid: a **soft switch** between 0 (block) and 1 (let through), applied element-wise by multiplication. Each gate has its own weights and looks at the current input and the previous state to decide.

signal	gate $\sigma(z)$	gate $\odot$ signal	
2.00	0.98	1.96	let through
-1.50	0.02	-0.03	blocked
0.80	0.50	0.40	halved
3.00	0.88	2.64	mostly through

Type of gate	Role	Used in
Update gate Γ_u	How much should the past matter now?	GRU
Relevance / reset gate Γ_r	Drop previous information?	GRU
Forget gate Γ_f	Erase a memory cell or not?	LSTM
Input gate Γ_i	Write new content into memory?	LSTM
Output gate Γ_o	How much to reveal of a cell?	LSTM

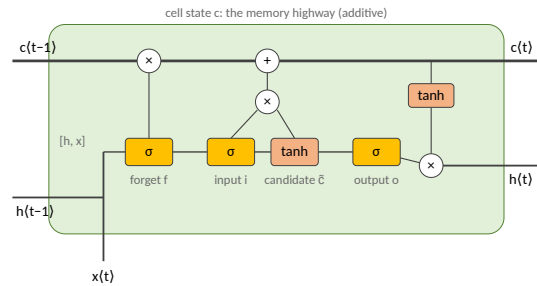
Analogy: a **notebook** (the cell state) with three rules: what to erase (forget), what to write (input), what to read aloud (output).

The LSTM cell

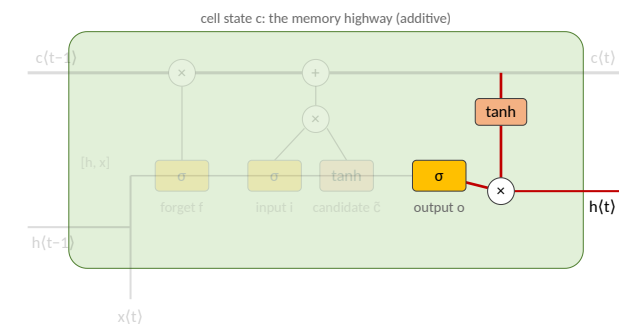
$$\begin{aligned} \mathbf{f}_t &= \sigma([\mathbf{h}_{t-1}, \mathbf{x}_t] \mathbf{W}_f + \mathbf{b}_f) \\ \mathbf{i}_t &= \sigma([\mathbf{h}_{t-1}, \mathbf{x}_t] \mathbf{W}_i + \mathbf{b}_i) \\ \tilde{\mathbf{c}}_t &= \tanh([\mathbf{h}_{t-1}, \mathbf{x}_t] \mathbf{W}_c + \mathbf{b}_c) \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \\ \mathbf{o}_t &= \sigma([\mathbf{h}_{t-1}, \mathbf{x}_t] \mathbf{W}_o + \mathbf{b}_o) \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \end{aligned}$$

$\mathbf{f}_t$ forget, $\mathbf{i}_t$ input, $\mathbf{o}_t$ output gates; $\tilde{\mathbf{c}}_t$ candidate content. The memory update is a **sum**, highlighted in blue.

Two states travel along time: the **cell** $\mathbf{c}_t$ (long-term memory) and the **hidden** $\mathbf{h}_t$ (what is shown to the next layer). Parameters: 4 blocks, $4(d_h(d_h + d) + d_h)$.



The LSTM, one gate at a time



stage 4 / 4

► 4 · output: what part of the memory to show as h

4 · Output gate. Which part of the memory to show, to the next layer and to the next step:

$$\mathbf{o}_t = \sigma([\mathbf{h}_{t-1}, \mathbf{x}_t] \mathbf{W}_o + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

The memory can hold something for a long time without showing it ($\mathbf{o} \approx 0$) and reveal it exactly when it is needed.

One LSTM step, with numbers

$$c(t-1) = 0.8$$

$$\text{keep: } f \cdot c(t-1) = 0.9 \cdot 0.8 = 0.720$$

$$\text{write: } i \cdot \tilde{c} = 0.3 \cdot (-0.5) = -0.150$$

$$\text{new memory } c(t) = 0.72 + (-0.15) = 0.570$$

$$\text{output: } h(t) = o \cdot \tanh(c(t)) = 0.7 \cdot \tanh(0.57) = 0.361$$

the output gate decides how much of the memory to show

step  4 / 4

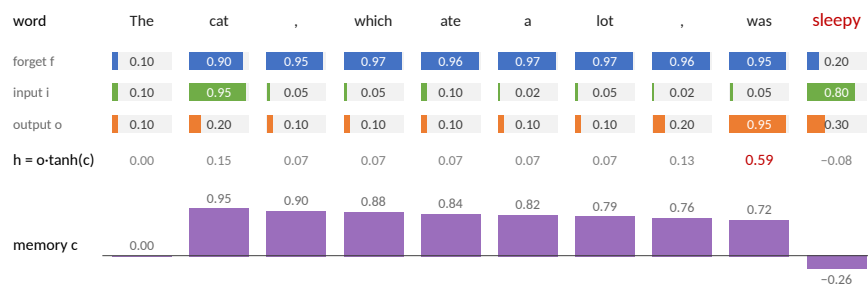
One memory cell, old value $c_{t-1} = 0.8$. The gates (computed from $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$) came out as $f = 0.9$, $i = 0.3$, $o = 0.7$, candidate $\tilde{c} = -0.5$.

$$c_t = 0.9 \cdot 0.8 + 0.3 \cdot (-0.5) = 0.57$$

$$h_t = 0.7 \cdot \tanh(0.57) = 0.36$$

The memory is **mostly kept** and **slightly corrected**. With $f = 1$, $i = 0$ it would be copied unchanged, for as many steps as needed.

The LSTM reading a sentence



"sleepy": the forget gate closes (the subject is no longer needed) and new content is written

words read  10 / 10

"The cat, which ate a lot, **was** sleepy": to choose **was** rather than **were**, the network must remember that the subject is singular across a whole sub-clause. The gate values are illustrative; a trained LSTM learns this kind of behaviour, spread over many cells.

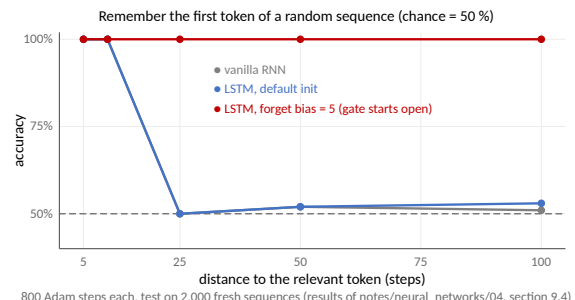
Why the gradient survives in an LSTM

Along the memory path, $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \dots$, so

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \text{diag}(\mathbf{f}_t) \Rightarrow \frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_k} \approx \prod_{t=k+1}^T \text{diag}(\mathbf{f}_t)$$

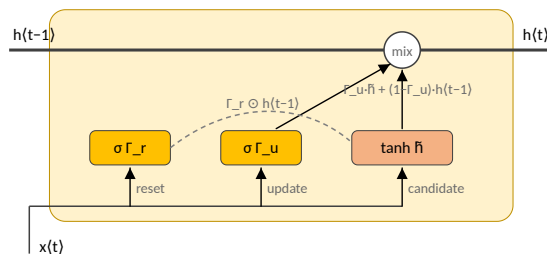
No weight matrix and no squashing derivative in this product: only the forget gates. When the network has learned to remember ($\mathbf{f}_t \approx 1$), the gradient flows back almost unchanged over many steps.

The same idea as the residual connection $\mathbf{h} = \mathbf{x} + F(\mathbf{x})$: an **additive path** with a near-identity Jacobian. Tip: initialize the forget-gate bias to a positive value, so the LSTM starts out remembering.



The experiment of the notes: the label is the **first** token, then up to 99 noise tokens. The tip above is the difference between chance and 100 %.

GRU: a lighter cousin



GRU (Gated Recurrent Unit): 2 gates and no separate cell state.

$$\tilde{\mathbf{h}}_t = \tanh([\Gamma_r \odot \mathbf{h}_{t-1}, \mathbf{x}_t] \mathbf{W} + \mathbf{b})$$

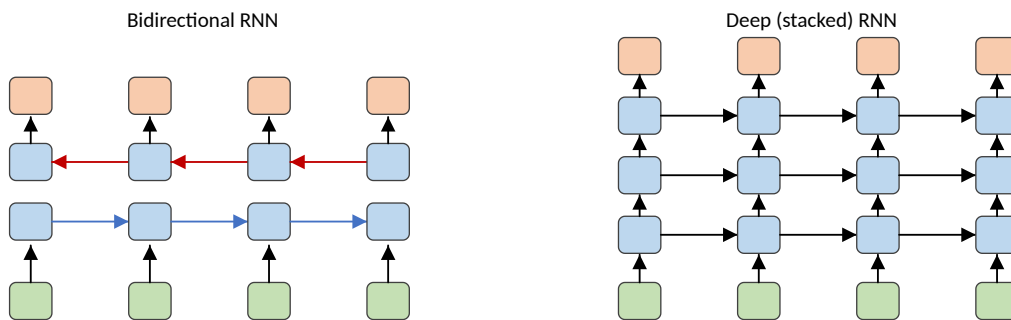
$$\mathbf{h}_t = \Gamma_u \odot \tilde{\mathbf{h}}_t + (1 - \Gamma_u) \odot \mathbf{h}_{t-1}$$

The update gate interpolates between "keep the old state" ($\Gamma_u = 0$) and "replace it" ($\Gamma_u = 1$): forget and input tied together. 3 weight blocks instead of 4.

Similar accuracy in practice; the LSTM is the safe default for long dependencies, the GRU is a bit faster.

Other RNN: bidirectional and deep

- What happens if we model a very non-linear problem? **Deep RNNs**: stack recurrent layers, the outputs of one are the inputs of the next (`num_layers=2`).
- What if the output should not only depend on the past (e.g. S&P 500 values) but also on the future (e.g. classifying words in a sentence)? **Bidirectional RNNs**: one RNN reads left-to-right, another right-to-left; concatenate their states (`bidirectional=True`).



A bidirectional model needs the whole sequence first: fine for classification, impossible for real-time forecasting or text generation.

Practicalities



batch tensor [3, 5] plus lengths [3, 5, 1]: the model must ignore <PAD>

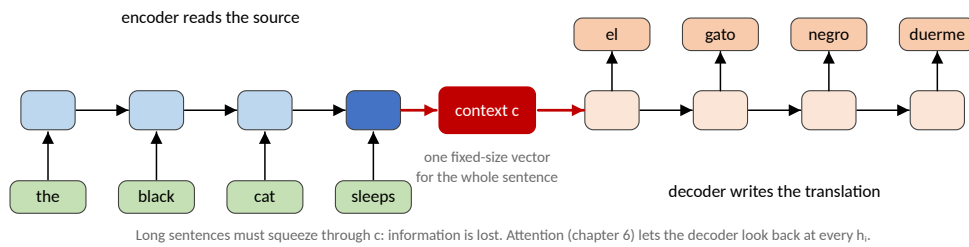
- Batches**: pad sequences to a common length. `pack_padded_sequence` tells the LSTM where each sequence really ends, so the padding is skipped; `padding_idx=0` keeps the <PAD> embedding at zero.
- Many-to-one**: use the hidden state after the **last real token** (what packing returns), or the mean / max over time. Without packing, `h[-1]` is the state after the padding!
- Clip gradients** (norm 1.0) and use Adam.

```
class SentimentLSTM(nn.Module):
    def __init__(self, V, d=64, dh=64):
        super().__init__()
        self.emb = nn.Embedding(V, d, padding_idx=0)
        self.lstm = nn.LSTM(d, dh, batch_first=True)
        self.head = nn.Linear(dh, 1)

    def forward(self, tokens, lengths): # [B, T], [B]
        x = self.emb(tokens)           # [B, T, d]
        x = pack_padded_sequence(x, lengths.cpu(),
                                batch_first=True, enforce_sorted=False)
        _, (h, c) = self.lstm(x)        # h: [1, B, dh]
        return self.head(h[-1]).squeeze(1) # [B] logits

loss_fn = nn.BCEWithLogitsLoss()
# in the loop, after loss.backward():
nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

The limit of recurrence: one vector for everything



Sequential: step t must wait for step $t - 1$. No parallelism over time, slow on GPUs, painful for sequences of thousands of tokens.

Bottleneck: in translation the whole source sentence must fit in one fixed-size vector, and far-away words must survive many multiplications. Next chapter: let every step **look directly** at every other step.

Vocabulary of chapter 5

token, vocabulary	a unit of text and the set of all of them, each with an integer id.
embedding	a learned dense vector per token: row of $\mathbb{E}$, a lookup table.
hidden state h_t	the RNN's running summary of the sequence so far.
unrolling	drawing the RNN as a deep net, one layer per time step, shared weights.
BPTT	backpropagation on the unrolled graph; shared-weight gradients add up.
vanishing gradient	a product of many factors below 1: far steps get no learning signal.

gate	a sigmoid vector in $(0, 1)$ multiplying a signal: a soft switch.
cell state c_t	the LSTM's long-term memory, updated additively.
LSTM / GRU	gated RNNs: 3 gates + cell / 2 gates, no cell.
gradient clipping	rescale the gradient if its norm exceeds a threshold.
padding, packing	fill short sequences; tell the RNN where each one really ends.

Theory notes for this chapter: [notes 04 · Embeddings, RNNs and LSTMs](#) (including the memory experiment).

LAB 11

Sentiment analysis with an LSTM (IMDB)

50,000 movie reviews, each labelled positive or negative. Is word order worth modelling?

- 1 A strong baseline that ignores word order: TF-IDF + logistic regression.
- 2 From text to tensors: truncation, padding and lengths; word embeddings.
- 3 An RNN and an LSTM **from scratch**; backpropagation through time and vanishing gradients, measured.
- 4 Embedding → LSTM → linear, with packing, gradient clipping and early stopping.
- 5 Compare a plain RNN, an LSTM, a GRU and a bidirectional LSTM; read a review word by word; try your own reviews.

[CO Open in Colab](#)

notebook Lab 11 - LSTM sentiment analysis.ipynb

you need this chapter: embeddings, RNN, BPTT, LSTM and GRU, padding and packing

results validation accuracy: plain RNN 80 %, LSTM 87 %, GRU 88 %, and the order-free TF-IDF baseline 90 %. For sentiment, which words appear matters more than their order

runs on CPU works; a GPU is faster

Neural Networks

Máster en Innovación Tecnológica (MINT)

Next: Attention and transformers

imanolgranada@gmail.com

Neural Networks

Máster en Innovación Tecnológica (MINT)

Attention and the transformer architecture

imanolgranada@gmail.com



1

Problems with LSTMs:

- Inputs are **sequential**: no parallelization. Step t must wait for step $t - 1$.
- No explicit modelling of long- and short-range dependencies.
- The distance between positions matters: information from far away must survive many steps.
- The whole input is squeezed into one fixed-size state.

What we want:

- handle inputs of different lengths,
- parallel processing of the entire input,
- shared parameters (like CNNs),
- any position reachable from any other in one step.

Attention Is All You Need

Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin · 2017

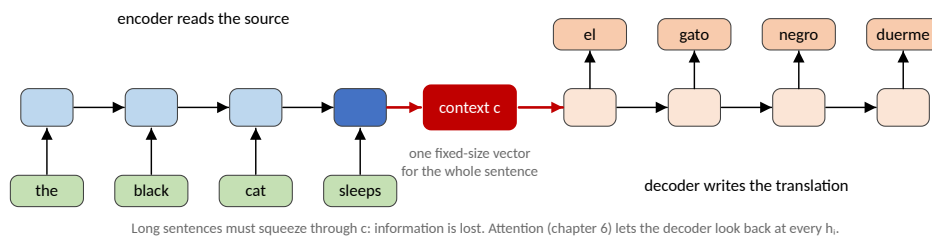
The paper that introduced the transformer: no recurrence, no convolution, only attention. Behind GPT, BERT, translation, speech (Whisper) and vision (ViT).

Two labs close this chapter: Lab 14 (text generation with GPT-2) and Lab 15 (RAG: search and grounded answers).



2

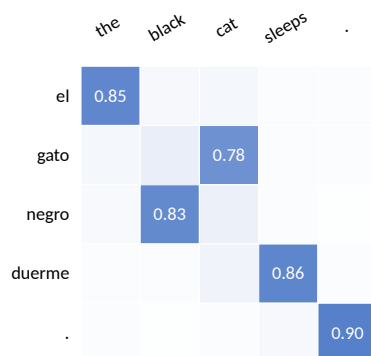
Why attention: the bottleneck



Before attention, sequence models **compressed an entire input sequence into a single fixed-size vector** before decoding it. Long sequences lost information by the time they reached the output.

Attention (Bahdanau 2014) fixes this by letting the decoder **look back at every input position** every time it produces an output, and decide how much each one matters.

Attention in translation: a soft alignment



row = a word being written
column = a source word

each row sums to 1

"gato" looks at "cat",

"negro" at "black":

the order flip is learned
from data, not programmed

When writing each Spanish word, the decoder computes a **weight for every English word**, then reads a weighted average of their encoder states.

$$\mathbf{c}_t = \sum_j \alpha_{tj} \mathbf{h}_j, \quad \sum_j \alpha_{tj} = 1$$

Nobody tells the network that "negro" goes with "black". The weights are **learned** end to end, only from pairs of sentences.

The transformer takes this idea to its limit: attention everywhere, and no recurrence at all.

Attention mechanism: a soft dictionary lookup

A Python dictionary returns the value whose key **exactly** matches the query. Attention returns a **weighted mix** of all the values, weighted by how well each key matches:

- **Query** q : what am I looking for?
- **Keys** k_j : what does each position offer?
- **Values** v_j : what does each position pass on?

$$\alpha_j = \text{softmax}_j(q^\top k_j), \quad \text{output} = \sum_j \alpha_j v_j$$

```
d = {"cat": v1, "dog": v2, "car": v3}
d["cat"]           # hard lookup → v1

# attention, query ≈ "kitten":
# weights  cat 0.80, dog 0.18, car 0.02
# output = 0.80·v1 + 0.18·v2 + 0.02·v3
```

The weights are positive and sum to 1: the output is a **linear combination of the values**.

Everything is differentiable, so the queries, keys and values can be **learned** by backprop.

Attention with numbers

token	key k	value v	$q \cdot k / \sqrt{2}$	softmax α	$\alpha \cdot v$
I	(1, 0)	(1, 0)	1.414		0.284 (0.28, 0.00)
love	(0, 1)	(0, 1)	0.707		0.140 (0.00, 0.14)
pizza	(1, 1)	(0.5, 0.5)	2.121		0.576 (0.29, 0.29)

query of "pizza": $q = (2, 1)$

output = $\sum \alpha v = (0.57, 0.43)$

the new vector for "pizza": mostly its own value, some of "I"

step  4 / 4

Three tokens with 2-dimensional keys and values. We compute the new vector for "pizza", whose query is $q = (2, 1)$:

$$\text{scores} = \frac{1}{\sqrt{2}} (q \cdot k_I, q \cdot k_{\text{love}}, q \cdot k_{\text{pizza}}) = \frac{1}{\sqrt{2}} (2, 1, 3)$$

$$\alpha = \text{softmax}(1.41, 0.71, 2.12) = (0.28, 0.14, 0.58)$$

$$\text{out} = 0.28 \begin{pmatrix} 1 \\ 0 \end{pmatrix} + 0.14 \begin{pmatrix} 0 \\ 1 \end{pmatrix} + 0.58 \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 0.57 \\ 0.43 \end{pmatrix}$$

Every token does this at the same time, each with its own query.

Self-attention:

A sequence of n tokens $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, each a vector of size d_{model} . For each token we create three vectors with learned matrices:

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}_Q, \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}_K, \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}_V$$

$\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ (typically $d_k = d_v = d_{\text{model}}/h$). "Self": queries, keys and values all come from the **same** sequence.

How much token i should focus on token j :

$$\text{score}(i, j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}, \quad \alpha_{ij} = \frac{\exp(\text{score}(i, j))}{\sum_{k=1}^n \exp(\text{score}(i, k))}$$

$$\text{Attention}(\mathbf{x}_i) = \sum_{j=1}^n \alpha_{ij} \mathbf{v}_j$$

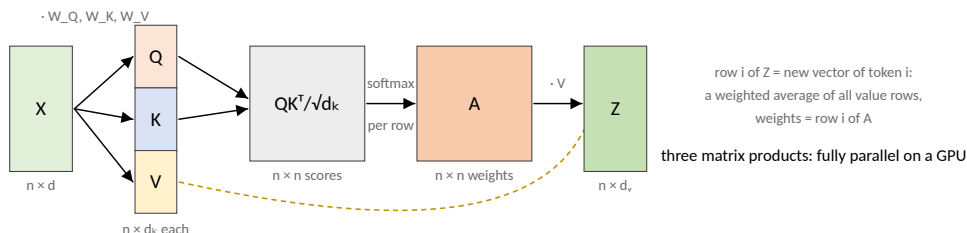
All tokens at once, in matrix form ($\mathbf{Q} = \mathbf{XW}_Q, \mathbf{K} = \mathbf{XW}_K, \mathbf{V} = \mathbf{XW}_V$):

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

$\mathbf{QK}^\top$ is $n \times n$: every token against every token. The softmax is taken row by row. Each token ends up as a combination of all the tokens in the sequence, weighted by the attention scores.

Why separate Q and K? "What I look for" and "what I offer" are different: a verb looks for its subject, a noun offers itself as one. Two matrices let the relation be asymmetric.

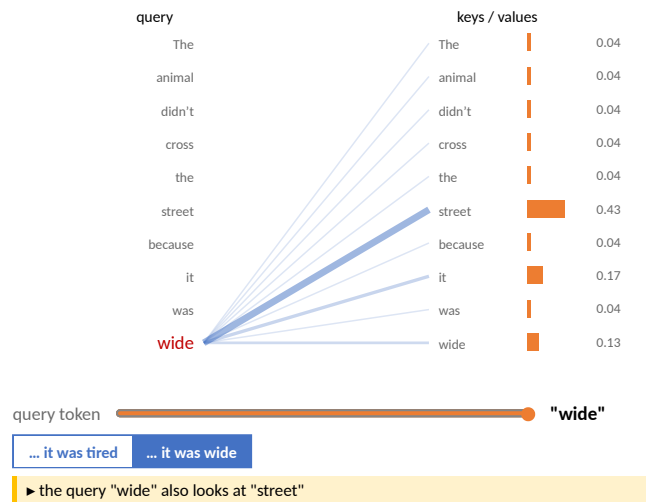
Self-attention as matrices



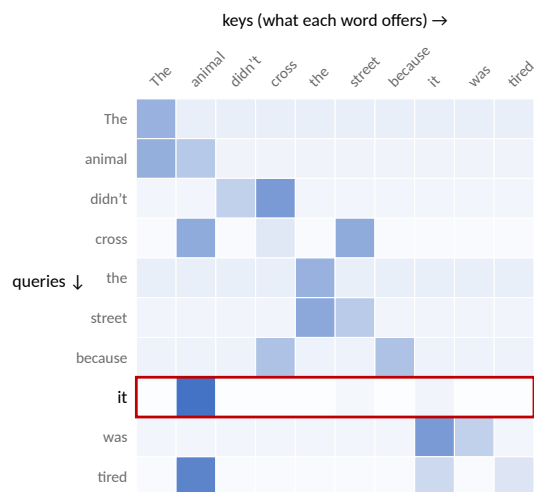
Example: $n = 10$ tokens, $d = 512$, $d_k = 64$. $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are 10×64 , the score matrix is 10×10 , the output is again one vector per token.

Cost: $O(n^2 d)$. Doubling the context quadruples the attention cost: why context windows are limited and why long-context tricks exist (FlashAttention, sparse attention).

Self-attention: what does "it" refer to?



The whole attention matrix



Each **row** is one query: a probability distribution over the sentence. The red row is the one of the previous slide.

Each **column** shows who reads from that word. "animal" is read by "cross", "it" and "tired": the matrix captures who-relates-to-whom in the sentence.

A trained model has one such matrix per head and per layer: GPT-2 small has $12 \times 12 = 144$ of them for every input.

Why divide by $\sqrt{d_k}$?

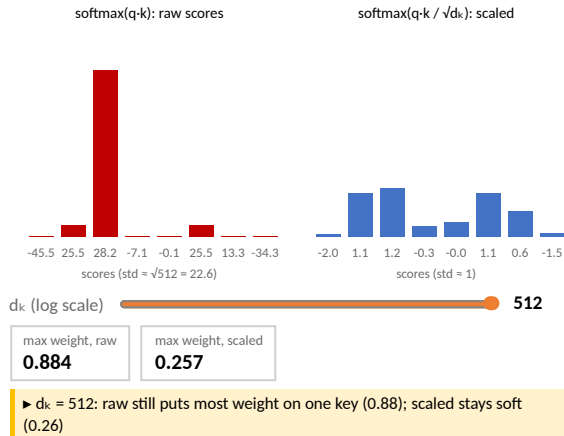
Suppose the components of $\mathbf{q}$ and $\mathbf{k}$ are independent with mean 0 and variance 1:

$$\mathbb{E}[\mathbf{q}^\top \mathbf{k}] = \sum_{i=1}^{d_k} \mathbb{E}[q_i] \mathbb{E}[k_i] = 0$$

$$\text{Var}(\mathbf{q}^\top \mathbf{k}) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \sum_{i=1}^{d_k} \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = d_k$$

Raw dot products have standard deviation $\sqrt{d_k}$ (8 for $d_k = 64$). Large scores push the softmax into **saturation**: nearly one-hot, where its gradient is nearly 0.

Dividing by $\sqrt{d_k}$ brings the variance back to 1, whatever the dimension. ■



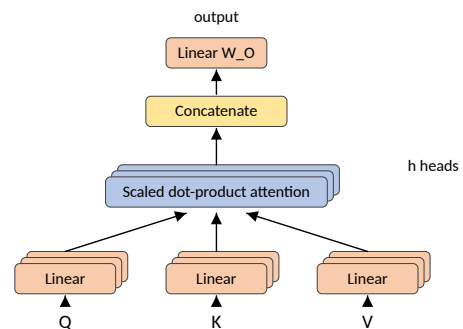
Self-Attention and multihead:

Train several attentions **in parallel** and let each focus on different things (syntax, coreference, position...).

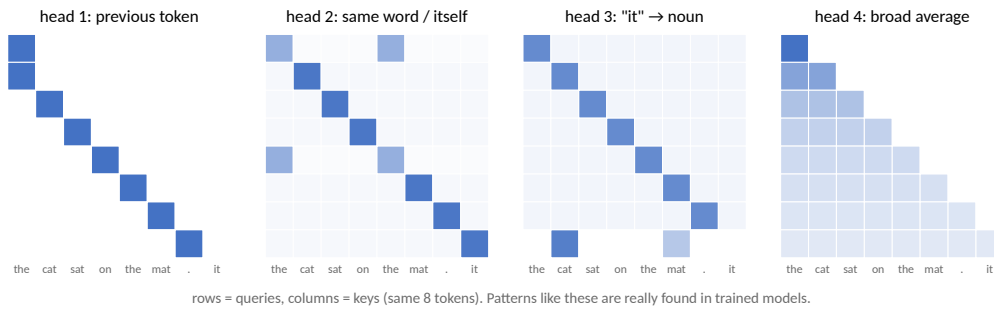
$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_Q^{(i)}, \mathbf{X}\mathbf{W}_K^{(i)}, \mathbf{X}\mathbf{W}_V^{(i)})$$

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O$$

With h heads of size $d_k = d/h$, the cost is about the same as one big head. Example: $d = 768$, $h = 12$, $d_k = 64$.



Different heads, different jobs



One head alone gives one weighted average per token: it cannot look at the previous word **and** at the subject at the same time. With several heads, each builds its own pattern, and $\mathbf{W}_O$ mixes their results. Researchers find heads like these (previous-token, coreference, "induction" heads that copy patterns) in trained language models.

Positional encodings:

Does the order of words matter? Permute the input rows: $\mathbf{QK}^\top$ and the output are permuted the same way. Self-attention is **permutation-equivariant**: "dog bites man" = "man bites dog".

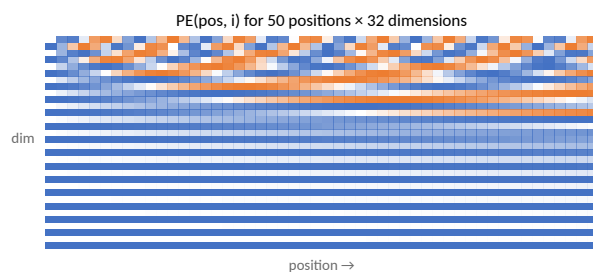
So we **add** an encoding of the position to each embedded input:

$$\text{input}_t = \text{embedding}(\text{token}_t) + PE(t)$$

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Each pair of dimensions is a clock at a different speed. Relative positions become easy to read: $PE(pos + k)$ is a fixed rotation of $PE(pos)$. GPT-2 instead **learns** a position embedding; modern LLMs rotate $\mathbf{q}, \mathbf{k}$ directly (RoPE).



Low dimensions oscillate fast, high dimensions slowly: every position gets a unique pattern.

Self-Attention mask:

The mask is optional, but it is used where it does not make sense for a word to look at **future** words, because they are not available in reality: not for classifying a known sentence, but for **next-word prediction**.

scores $QK^T/\sqrt{d}$	mask: $-\infty$ if $j > i$	softmax: weights
1.1 0.9 -0.3 1.4 -1.4	1.1 $-\infty$ $-\infty$ $-\infty$ $-\infty$	1.00 0 0 0 0
-0.7 1.5 -0.2 -0.6 0.9	-0.7 1.5 $-\infty$ $-\infty$ $-\infty$	0.10 0.90 0 0 0
-0.5 0.2 -0.4 1.8 -1.8	-0.5 0.2 -0.4 $-\infty$ $-\infty$	0.24 0.49 0.26 0 0
1.1 -0.2 1.6 -0.4 -0.2	1.1 -0.2 1.6 -0.4 $-\infty$	0.30 0.09 0.54 0.07 0
1.0 0.6 -0.5 0.3 -0.8	1.0 0.6 -0.5 0.3 -0.8	0.40 0.25 0.08 0.20 0.06

Row i = the token being predicted. It may only look at tokens $1..i$, never at the future.

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V, \quad M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

$e^{-\infty} = 0$: future tokens get exactly zero weight. And one forward pass trains all n next-token predictions of a sequence at once.

Self-attention in code

```
import math, torch

def self_attention(X, Wq, Wk, Wv, causal=False):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv      # [n, dk] each
    S = Q @ K.T / math.sqrt(K.shape[-1])    # [n, n] scores
    if causal:                               # hide the future
        future = torch.triu(torch.ones_like(S), diagonal=1).bool()
        S = S.masked_fill(future, float("-inf"))
    A = S.softmax(dim=-1)                   # each row sums to 1
    return A @ V                           # [n, dv]: new vectors

X = torch.randn(10, 512)                  # 10 tokens, d = 512
Wq, Wk, Wv = (torch.randn(512, 64) / 512**0.5 for _ in range(3))
Z = self_attention(X, Wq, Wk, Wv, causal=True) # [10, 64]
```

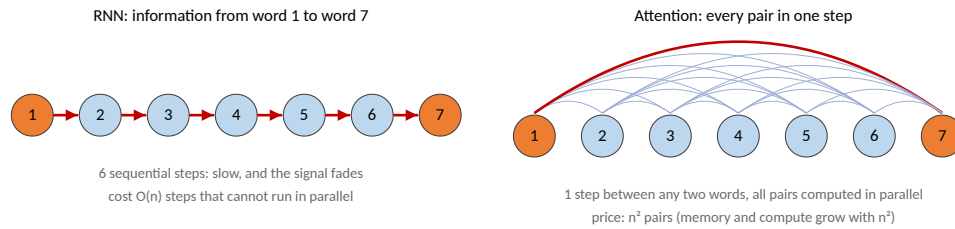
Everything of the last slides fits in a handful of lines: three projections, one scaled product, an optional mask, a softmax per row, and a weighted average of the values.

With a batch, the same code works on tensors $[B, n, d]$ (write `K.transpose(-2, -1)` for K^T). Multi-head: reshape to $[B, h, n, d_k]$ and run all heads at once.

In practice: `nn.MultiheadAttention`, or `F.scaled_dot_product_attention(Q, K, V, is_causal=True)`, which uses fast fused kernels (FlashAttention).

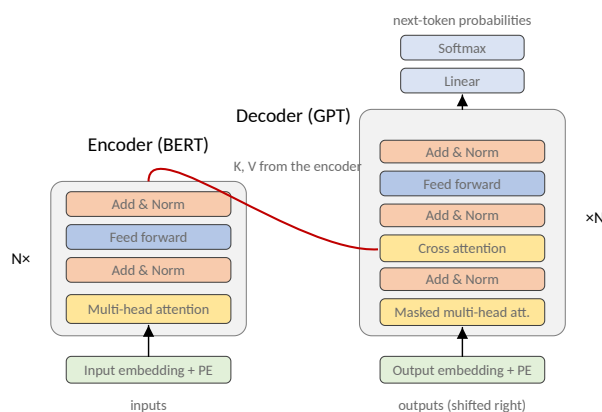
Lab 14 plots the real attention matrices of GPT-2.

Why transformers won



	RNN / LSTM	Self-attention
steps between two far words	n	1
parallel over positions (training)	no	yes: matrix products
cost per layer	$O(n d^2)$	$O(n^2 d)$
memory of the past at generation	one state vector	every previous token (the "KV cache")

Transformer encoder-decoder:



Each block (pre-norm version):

$$\mathbf{X} \leftarrow \mathbf{X} + \text{MHA}(\text{LN}(\mathbf{X}))$$

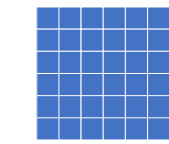
$$\mathbf{X} \leftarrow \mathbf{X} + \text{MLP}(\text{LN}(\mathbf{X}))$$

- **Attention mixes information between positions**; the MLP ($d \rightarrow 4d \rightarrow d$, GELU) processes each position separately: "communicate, then think".
- **Residual connections** and **LayerNorm** make deep stacks trainable (chapters 3–4).
- **Cross-attention**: queries from the decoder, keys and values from the encoder: the alignment of translation.

The figure is the original transformer, with "Add & Norm" after each sub-layer (post-norm). GPT-2 and most modern models apply the LayerNorm before each sub-layer (pre-norm, the formulas above): it trains more stably.

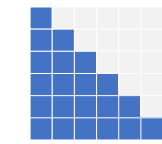
Encoders and decoders: BERT vs GPT

BERT (encoder): sees both sides



fill-in-the-blank, classification, embeddings

GPT (decoder): sees only the past



next-token prediction, generation

	Encoder only	Decoder only	Encoder-decoder
attention	bidirectional	causal	both + cross
trained to	fill in masked words	predict the next token	map input → output
used for	classification, embeddings, search	chatbots, code, text generation	translation, summarization, speech
examples	BERT, RoBERTa, sentence encoders	GPT, Llama, Claude	original transformer, T5, Whisper

Language modelling: the objective behind GPT

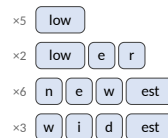
By the chain rule of probability, any sequence factorizes as

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_1, \dots, x_{t-1})$$

So it is enough to learn one thing: the **next token** given the previous ones. A $|V|$ -class classification with softmax and cross-entropy, exactly chapter 1.

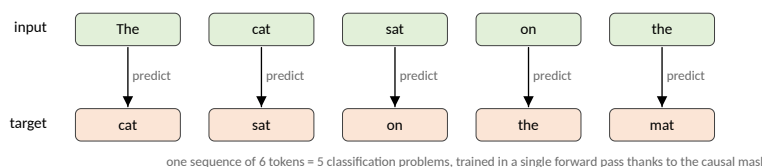
Text is first split into sub-word **tokens** by **byte-pair encoding**: start with characters, repeatedly merge the most frequent adjacent pair. Frequent words become one token, rare ones are split, nothing is ever unknown.

"lo" + "w": 7 times → "low"



the vocabulary grows by one symbol per merge; GPT-2 stops at 50,257 tokens

merges 4 / 4



GPT: counting the parameters

Token + position embedding $\rightarrow N$ identical blocks $\rightarrow$ final LayerNorm $\rightarrow$ linear layer to $|V|$ logits. Per block, ignoring biases and LayerNorms:

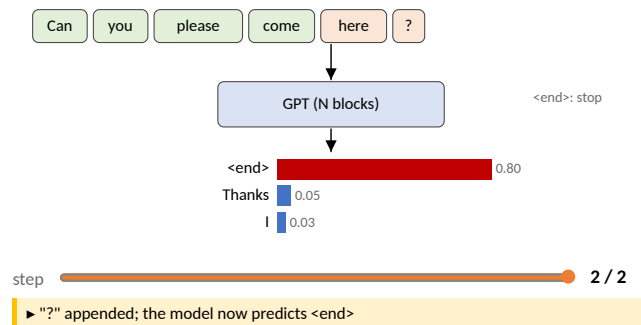
- Attention: $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$, each $d \times d$: $4d^2$.
- MLP with hidden size $4d$: $d \cdot 4d + 4d \cdot d = 8d^2$.
- Total: $12d^2$ **per block**.

GPT-2 small: $N = 12$, $d = 768$, $|V| = 50,257, 1,024$ positions:

$$\underbrace{12 \cdot 12 \cdot 768^2}_{\text{blocks}} \approx 84.9\text{M} + \underbrace{50,257 \cdot 768}_{\text{token emb.}} \approx 38.6\text{M} + \underbrace{1024 \cdot 768}_{\text{positions}} \approx 0.8\text{M} \approx \mathbf{124\text{M}}$$

The output layer usually reuses the embedding matrix (weight tying): $\mathbf{z}_t = \mathbf{h}_t \mathbf{E}^\top$, so it adds no parameters. Modern LLMs: the same recipe with $N \approx 80$, $d \approx 8,000$: tens of billions of parameters.

Generating text: one token at a time



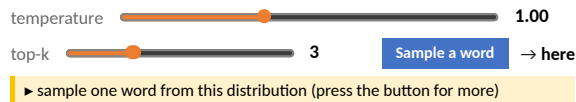
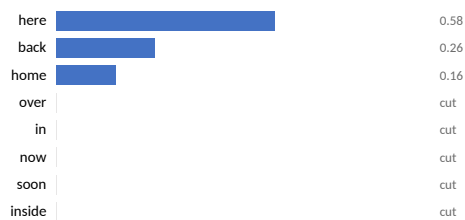
Autoregressive: the model gives a distribution over the next token; one token is chosen, appended to the input, and the model runs again.

Training is parallel (the mask), **generation is sequential**: 500 tokens of answer = 500 forward passes. The keys and values of past tokens are cached (the **KV cache**) so each pass only processes the new token.

How the token is chosen from the probabilities: next slide.

Decoding: from probabilities to text

Can you please come ____ ?



"Can you please come **here**?": the history, and the word being predicted.

- **Greedy**: always the most likely token. Safe, but repetitive (it loops).
- **Temperature T** : sample from $\text{softmax}(\mathbf{z}/T)$. $T \rightarrow 0$ is greedy; $T > 1$ flattens the distribution (more creative, more errors).
- **Top-k / top-p**: keep only the k most likely tokens (or the smallest set with probability p), renormalize, sample.
- **Beam search**: keep the b best partial sentences; used in translation.

Measuring a language model: perplexity

A language model can also **score** a text: look at the probability it gave to each actual next token. The average of minus their logs is the training loss (cross-entropy per token):

$$J = -\frac{1}{T} \sum_{t=1}^T \log p(x_t | x_{<t})$$

perplexity = e^J

Meaning. If the model gave every true next token probability $\frac{1}{4}$: $J = \log 4$ and the perplexity is 4. The model is as uncertain as if it had to pick among **4 equally likely tokens** at every step.

Uniform guessing over GPT-2's 50,257 tokens gives perplexity 50,257. Lower is better.

GPT-2 small, measured in Lab 14:

text	perplexity
The cat sat on the mat and looked out of the window.	21.7
the same words, shuffled	1,536
the same sentence in Spanish	235
Paris is the capital of France.	15.8
Paris is the capital of Germany.	17.8

Word order matters a lot ($\times 70$), GPT-2 saw little Spanish, and it prefers the true fact only slightly: it learned statistics about words, not facts.

LAB 14

Text generation with transformers (GPT-2)

Open up GPT-2 small (124 M parameters) and find every piece of this chapter inside a real model.

- 1 Byte-pair encoding from scratch; how GPT-2 cuts English and Spanish text.
- 2 Count the parameters (the $12d^2$ per block) and run one forward pass: a next-token distribution at every position.
- 3 Score text: log-likelihood, **perplexity**, per-token surprisal.
- 4 Look at the **causal attention** patterns of the heads.
- 5 Decoding from scratch: greedy, temperature, top-k, top-p; the **KV cache**; beam search.
- 6 A base model is not an assistant: prompting and few-shot learning.

[CO Open in Colab](#)

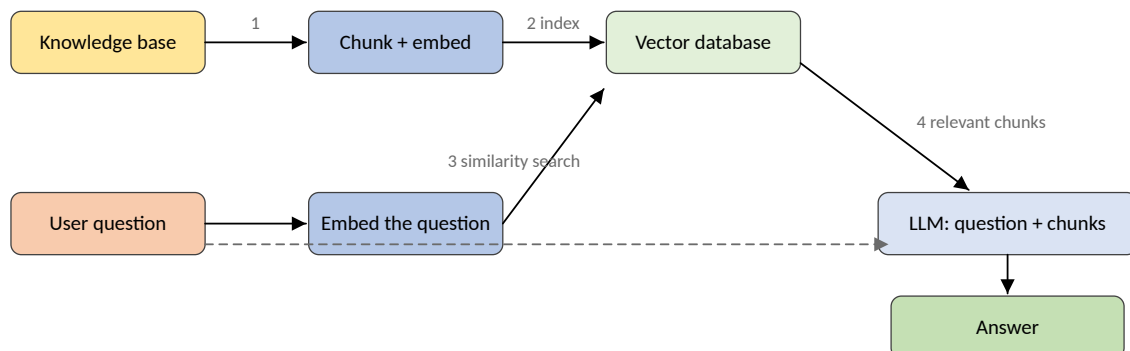
notebook Lab 14 - Text generation with Transformers.ipynb

you need attention, the causal mask, the transformer block, language modelling, decoding and perplexity (this chapter)

runs on CPU works; a GPU is faster

data GPT-2 weights (≈500 MB) from the Hugging Face Hub

How do companies build bots with custom data?



Retrieval-Augmented Generation (RAG): the LLM is not retrained. Instead, the most relevant pieces of **your** documents are found by embedding similarity and pasted into the prompt, so the model answers from them.

The embeddings come from an encoder transformer trained so that texts with similar meaning get similar vectors: chapter 5's word2vec idea, for whole paragraphs.

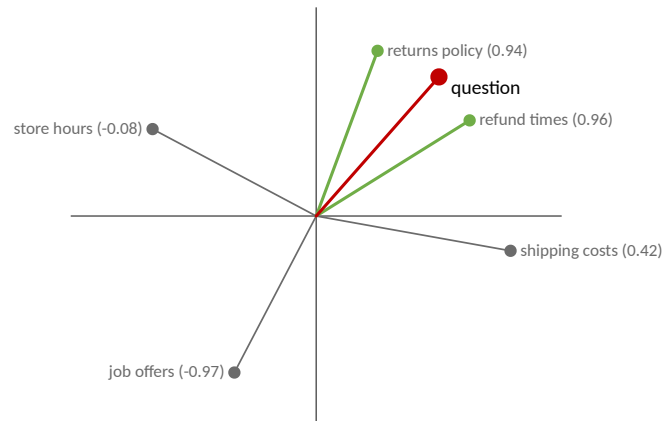
Chunking, embeddings and chunk selection:

- Split each document into **chunks** (a few hundred tokens, with some overlap).
- Turn each chunk into an embedding vector: dim 384, 768, 1,024...
- Cosine similarity** to select the top- k chunks for a question:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

1 = same direction (same meaning), 0 = unrelated, -1 = opposite. With vectors normalized to length 1, the cosine is just the dot product.

Evaluate the retrieval first (next slide), then the generated answers.



Measuring retrieval: hit rate@k and MRR

Before judging the LLM's answers, check the **search**. Take test questions whose relevant chunk is known, and look at the **rank** of the first relevant chunk in the results (1 = top).

- Hit rate@k**: the fraction of questions whose relevant chunk is in the top k (often called recall@k). If it is not among the k chunks given to the LLM, the LLM cannot use it.
- Mean reciprocal rank**: $\text{MRR} = \frac{1}{|Q|} \sum_q \frac{1}{\text{rank}_q}$. It is 1 if the answer is always first, 0.5 if always second.

question	rank of the relevant chunk	1 / rank
q1	1	1
q2	3	0.33
q3	1	1
q4	not found in the top 10	0

- hit@1 = 2/4 = 0.50 (q1 and q3 are first)
- hit@3 = 3/4 = 0.75 (q2 enters at rank 3)
- MRR = (1 + 0.33 + 1 + 0)/4 ≈ 0.58

In Lab 15, on 16 questions: word TF-IDF hit@1 = 0.56; character n-grams and a neural sentence encoder both reach 0.81.

LAB 15

RAG: embeddings, search and grounded answers

A fictional guide to the coffee shops of an invented city (in Spanish), and a small chatbot that answers from it.

- 1 Similarity: dot product, cosine and Euclidean distance.
- 2 Four ways to embed text: word TF-IDF, character n-grams, LSA and a neural sentence encoder, compared with hit rate@k and MRR.
- 3 Search at scale: exact vs approximate nearest neighbours (IVF).
- 4 Chunking long documents.
- 5 An LLM answering with and without the retrieved context; failure modes: retrieval misses, hallucination, prompt injection.

[CO Open in Colab](#)

notebook lab 15 - RAG embeddings and similarity search.ipynb

you need embeddings (ch. 5), cosine similarity, RAG and retrieval metrics (this part), decoding (Lab 14)

runs on CPU works; a GPU speeds up the generation part

data a sentence encoder (~470 MB) and two small Qwen2.5 instruct models (~4 GB) from the Hugging Face Hub

Vocabulary of chapter 6

query, key, value	what a token looks for, what it offers, what it passes on.	transformer block	attention + MLP, each with a residual and LayerNorm.
attention weights	softmax of $\mathbf{q} \cdot \mathbf{k} / \sqrt{d_k}$; one distribution per query.	BPE token	a sub-word unit from repeated merges of frequent pairs.
self / cross-attention	Q, K, V from the same sequence / Q from one, K, V from another.	temperature, top-k	how the next token is sampled from the probabilities.
multi-head	several attentions in parallel, concatenated and mixed by $\mathbf{W}_O$.	perplexity	e^{loss} : the number of equally likely choices the model hesitates between.
positional encoding	added to the embeddings, since attention ignores order.	RAG	retrieve relevant chunks by embedding similarity, give them to the LLM.
causal mask	$-\infty$ above the diagonal: no peeking at the future.	hit rate@k, MRR	how often, and how high, the right chunk is retrieved.

Theory notes for this chapter: [notes 05 · Attention and transformers](#) (with a tiny GPT trained from scratch) · [notes 06 · Embeddings, retrieval and RAG](#).

Neural Networks

Máster en Innovación Tecnológica (MINT)

Thank you!

imanolgranada@gmail.com

